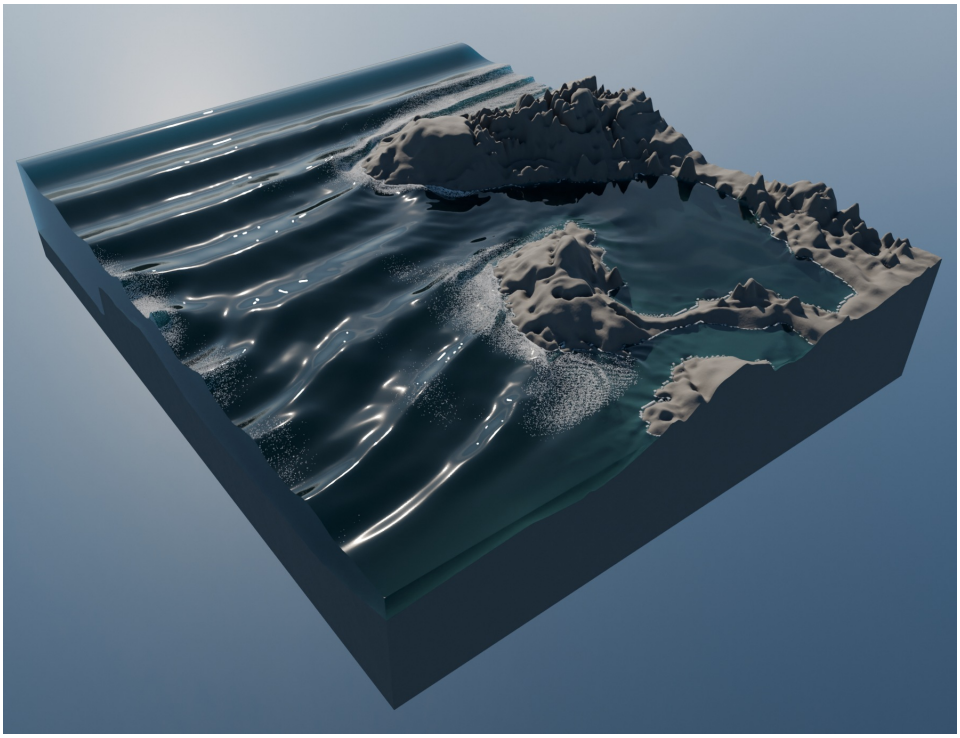
LAGOON

User Guide

Lightweight Animation and Graphics for Open Ocean Numerics



LAGOON 1.0.0rc4, October 2026

Contact

Hans Bihs

Professor
Marine Civil Engineering
NTNU Trondheim

hans.bihs@ntnu.no

AI declaration

This User Guide has been created with the assistance of Claude (Anthropic, model claude-opus-5-5).

Parts of the LAGOON open-source software have been created with the assistance of Claude (Anthropic, model claude-opus-5-5).

Cover

A REEF3D::FNPF run of waves around an island with foam, spray and bubbles, path traced with Blender's Cycles in LAGOON's Diorama look.

Contents

1 Introduction **1**

2 Installation **5**

2.1 The Mac App 5

2.2 With pip 5

2.3 With conda or pixi 5

2.4 From the source code 5

2.5 The four programs 6

2.6 Graphics 6

2.7 Optional parts 6

3 Getting Started **7**

3.1 The Workflow at a Glance 7

3.2 The Main Window 7

3.3 Your First Run 9

4 Opening Runs **13**

4.1 The Case Folder 13

4.2 The Solids of DIVEMesh 15

4.3 Time 16

4.4 Several Runs 16

4.5 A Running Simulation 16

4.6 Cleaning Up a Case Folder 17

4.7 LAGOON Stores 17

5 Case Set-Up **19**

5.1 Set-Up and Results . . . 19

5.2 Starting a Case . . . 19

5.3 A Case That Has Not Run Yet . . . 20

5.4 The Case Set-Up Item . . . 20

5.5 The Points of geo.dat . . . 21

5.6 Editing the Input Files . . . 21

5.7 Porous Structures . . . 22

5.8 Solids . . . 23

5.9 Checking the Input . . . 23

5.10 Running DIVEMesh . . . 24

5.11 DIVEMesh's Grid . . . 24

5.12 The Set-Up Sketch . . . 24

6 River Corridors and Multi-Block Grids **27**

6.1 A River Corridor . . . 27

6.2 A Multi-Block Grid . . . 29

7 Maps and the Place of a Case **35**

7.1 Starting a Case on the Map . . . 35

7.2 Finding a Place . . . 35

7.3 The Georeference . . . 36

7.4 The Map Tab . . . 36

7.5 Bathymetry and geo.dat . . . 37

7.6 Background Maps . . . 39

8 The Sea at the Case **41**

8.1 Sources . . . 41

8.2 Storms, Waves and Boundaries . . . 41

8.3 A River . . . 42

8.4 Tide Gauges . . . 42

9 Scene Items **45**

9.1 Kinds of Items . . . 45

9.2 Colours . . . 46

9.3 Statistics over Time . . . 48

9.4 Vertical Scale . . . 48

9.5 Reading Values: the Probe . . . 49

9.6 Slices, Iso-Surfaces, Iso-Volumes and Streamlines . . . 49

10 The 3D View **51**

10.1 Moving Around . . . 51

10.2 The 2D View . . . 51

10.3 Looks . . . 52

10.4 The Sun . . . 53

10.5 Compare Side by Side . . . 53

10.6 Full Screen . . . 53

10.7 Images and Screenshots . . . 53

10.8 Without a GPU . . . 53

10.9 Speed or Memory . . . 54

11 Timeline and Camera Paths **55**

11.1 Playing . . . 55

11.2 The Play Range . . . 55

11.3 Camera Keyframes . . . 55

11.4 Orbit . . . 56

12 Plots and Free Surface Lines **57**

12.1 Time Series . . . 57

12.2 Measured Data . . . 58

12.3 Free Surface Lines . . . 58

12.4 Wave Analysis . . . 58

13 2D Plots **61**

13.1 Tables . . . 61

13.2 The Data Tab . . . 62

13.3 Live, New Figures and Scripts . . . 62

14 Paper Figures **63**

14.1 Export Figure . . . 63

14.2 The Figure Editor . . . 63

14.3 Templates . . . 64

15 Films **65**

15.1 The Time and the Colour Bars . . . 65

15.2 Films Without a Screen . . . 65

15.3 Path-Traced Films . . . 66

16 Cinematic Looks and Path Tracing **67**

16.1 Cinematic Looks . . . 67

16.2 Path Tracing with Blender . . . 68

16.3 Foam, Spray and Bubbles . . . 70

16.4 Rendering Time . . . 70

17 The LAGOON Format **71**

17.1 Making a Store . . . 71

17.2 Opening a Store . . . 72

18 Scripting **73**

18.1 A Session in Python . . . 73

18.2 The Window Writes the Script . . . 73

18.3 Pictures, Films and Figures . . . 74

19 Reference **75**

19.1 Command Line . . . 75

19.2 Menus and Keyboard Shortcuts . . . 77

19.3 Item Properties . . . 79

19.4 Session API . . . 90

1. Introduction

LAGOON (Lightweight Animation and Graphics for Open Ocean Numerics) is the post-processor of REEF3D and, with its set-up mode, its pre-processor as well. It is a desktop application for setting up, looking at, plotting and rendering ocean and coastal simulations, in the spirit of ParaView and Blender but made for hydrodynamics. LAGOON reads the output of all REEF3D modules directly:

- **REEF3D::NHFLOW**, **REEF3D::FNPF** and **REEF3D::SFLOW**: the free surface, the bed and the σ-grid volume, AMR surfaces, wave gauges, probes and free surface lines,
- **REEF3D::CFD**: the volume on the rectilinear grid, its free surface (the level set $\phi = 0$) and the refined blocks of its adaptive mesh refinement,
- **REEF3D::SEASTATE**: its maps of the sea state and its AMR output,
- the multiphysics output of all modules: floating bodies (6DOF) with their motions, forces and mooring lines, structures, DEM elements, ice floes, rod trees and particles,
- the solids, the bed and the grid of DIVEMesh and the run log REEF3D writes for every run.

Opening a case folder shows its free surface and bed in a GPU-accelerated 3D view, plots its wave gauges and forces on a common time axis under it, and steps through the outputs with a timeline. From there, LAGOON makes paper figures in a journal's style, x-y plots of tables and measurements, films with smooth camera paths, cinematic pictures and path-traced stills and films through Blender's Cycles renderer. Every action in the window is a Python command; a session writes itself out as a script that makes the same figures and films again without a screen, for example on a cluster.

LAGOON also sets cases up. It edits DIVEMesh's `control.txt` and REEF3D's `ctrl.txt` with the commands of both programs at hand, shows what they set up in the 3D view, checks the input before a run, runs DIVEMesh and shows its grid. A case can be started on the map: its place on the Earth, its domain drawn over a map or a satellite image, its bathymetry downloaded for it and written as DIVEMesh's `geo.dat`, the sea at the case (waves, wind, water level, currents, river discharge and tide gauges) read from hindcasts and written as REEF3D's boundary files. River corridors and multi-block grids for coasts and estuaries are laid out in the same window.

LAGOON defines the LAGOON format: a run stored in one folder (Zarr version 3), several times smaller than the VTU, VTP and text files and much faster to open, step through and probe. REEF3D can write it directly during the run (**P 18**), and `lagoon-convert` converts the output of earlier runs.

LAGOON is released under the GNU General Public License, version 3 or later (GPL v3). The LAGOON format library (`lagoon-format`) is released under the MIT license, so that any code can read and write the format. The source code is available from the GitHub repository `https://github.com/REEF3D/LAGOON`, the packages from PyPI (`pip install lagoon-viz`) and the online documentation from `https://reef3d.github.io/LAGOON`.

LAGOON was designed under the following premises:

1. Made for REEF3D and for hydrodynamics

    - a REEF3D case folder opens as it is: the free surface, the bed, the gauges, the bodies and their moorings appear by themselves
    - the parallel output (one file per MPI rank and output) is joined while reading
    - quantities of ocean engineering: vertical scale about the still water level, wave spectra with H_{m0} and T_p, reflection and response analysis, free surface lines with the wave theory, mooring tensions, statistics over time such as H_s maps and the maximum inundation

2. From the set-up to the results in one place

    - the input files of DIVEMesh and REEF3D edited with the command catalogue of both programs, their signs drawn in the 3D view
    - the place of a case on the Earth, its bathymetry and its sea conditions from open data
    - river corridors and multi-block grids drawn and written for REEF3D

3. Fast enough for large runs

    - a 3D view drawn on the GPU (WebGPU: Metal, Vulkan or Direct3D 12)
    - outputs read in the background and kept in a cache, so playing and stepping back are smooth; a streaming mode for cases of a hundred million cells
    - the LAGOON format for runs that are too large for VTK files

4. From looking to publishing in one place

    - paper figures drawn again with matplotlib at the width of a journal column, with vector output and LaTeX labels
    - films with camera keyframes and orbits
    - cinematic looks and path tracing with Blender's Cycles for showcase pictures

5. Reproducible

    - every change is a command that can be undone and appears in the Python console
    - a session is saved as a file and written out as a runnable Python script
    - figure templates make the same figure for the next run of a set-up

6. Open and maintainable

    - Python (3.11 to 3.13), Qt (PySide6), pygfx, NumPy and matplotlib
    - a core without graphics or Qt, used by the window, the scripts and the command line alike
    - an open data format with an MIT-licensed reader, a ParaView plugin and a specification

How this guide is organised

- Chapter 2 (Installation) explains how to install LAGOON and its optional parts on macOS, Linux and Windows.
- Chapter 3 (Getting Started) walks through a first run, from opening it to a figure and a film, and introduces the main window.
- Chapter 4 (Opening Runs) describes what LAGOON reads from a REEF3D case folder, and how running simulations, several runs, case folders and LAGOON stores are handled.
- Chapter 5 (Case Set-Up) describes the set-up mode: the input files, the porous structures and solids, the input check, DIVEMesh and its grid, and the set-up sketch.
- Chapter 6 (River Corridors and Multi-Block Grids) describes the curvilinear grids LAGOON lays out for SFLOW and NHFLOW, including the tide and the rivers of coasts and estuaries.
- Chapter 7 (Maps and the Place of a Case) explains the georeference, the background maps and how `geo.dat` is made from bathymetry.
- Chapter 8 (The Sea at the Case) describes the waves, wind, water levels, currents, rivers and tide gauges read for a case.
- Chapter 9 (Scene Items) describes the items of the scene, their colours and colour maps, statistics over time, the vertical scale, the probe, slices, iso-surfaces, iso-volumes and streamlines.
- Chapter 10 (The 3D View) covers the navigation, the 2D view, the looks, the sun, images and the memory use.
- Chapter 11 (Timeline and Camera Paths) covers playing, the play range and camera keyframes.
- Chapter 12 (Plots and Free Surface Lines) describes the time series, measured data, free surface lines and the wave analysis.
- Chapter 13 (2D Plots) describes the 2D plots mode: tables, expressions, x-y panels and fits.
- Chapter 14 (Paper Figures) explains the figure export, the figure editor and figure templates.
- Chapter 15 (Films) explains how to film the 3D view, in the window and without a screen.
- Chapter 16 (Cinematic Looks and Path Tracing) describes the cinematic looks and path tracing with Blender, including foam, spray and bubbles.
- Chapter 17 (The LAGOON Format) explains how to make, open and read LAGOON stores, from LAGOON, ParaView and Python.
- Chapter 18 (Scripting) explains the `Session` object and the scripts LAGOON writes.
- Chapter 19 (Reference) lists the command line programs, the menus and keyboard shortcuts, the properties of every item and the methods of `Session`.

2. Installation

LAGOON runs on macOS, Linux and Windows with Python 3.11 to 3.13. It is installed as the Python package `lagoon-viz`, which brings the format library `lagoon-format` and all other dependencies with it. On a Mac with Apple silicon, the LAGOON app needs no Python at all.

2.1 The Mac App

Download `LAGOON-<version>-macos-arm64.dmg` from the releases of the GitHub repository (Macs with Apple silicon, M1 and later), open it and drag LAGOON into Applications. Nothing else is needed: Python and LAGOON are inside. If macOS says the app cannot be opened, open it the first time with a right-click, *Open* and *Open* again. The app's program also runs the command line tools:

```
/Applications/LAGOON.app/Contents/MacOS/LAGOON render CASE view.png
/Applications/LAGOON.app/Contents/MacOS/LAGOON convert CASE
```

2.2 With pip

```
pip install lagoon-viz
lagoon
```

It is best to install LAGOON into a fresh environment, e.g. with `python -m venv lagoon` or with `uv venv && uv pip install lagoon-viz`. Release candidates are installed with `pip install --pre lagoon-viz`; without `--pre`, pip installs the last final version.

2.3 With conda or pixi

From the 1.0 release on, LAGOON is available from conda-forge:

```
conda install -c conda-forge lagoon-viz
pixi add lagoon-viz          # in a pixi project
```

2.4 From the source code

```
git clone https://github.com/REEF3D/LAGOON.git
cd LAGOON
pixi run lagoon              # with pixi: the same environment everywhere
pip install -e ./lagoon-format -e ".[dev]"     # or with pip
```

On a Mac, `bash setup_lagoon.command` in the repository installs everything (uv, Python and LAGOON in `.venv`), runs the tests and makes a `LAGOON.app` that starts LAGOON with a double-click. LAGOON installed this way runs directly from the source code: after an update of the sources (`git pull`), restarting LAGOON is enough; there is nothing to compile.

2.5 The four programs

Table 2.1 lists the programs that come with LAGOON. Their options are listed in Section 19.1.

Table 2.1: The programs of LAGOON.

Program	What it does
`lagoon`	the application (also `python -m lagoonviz`)
`lagoon-render`	images and films without a screen, e.g. on a cluster
`lagoon-convert`	REEF3D output (VTU, VTP and text files) to a LAGOON store
`lagoon-setup`	a case's set-up (`control.txt`, `ctrl.txt`) as a paper figure, or its main facts as text

2.6 Graphics

The 3D view draws with WebGPU: Metal on macOS, Vulkan on Linux and Direct3D 12 on Windows. Any GPU of the last ten years is sufficient. Headless Linux machines (clusters, continuous integration) render with Mesa's software Vulkan driver (package `mesa-vulkan-drivers` on Debian and Ubuntu). Without a usable GPU, for example over a remote desktop, the window opens with a note in place of the 3D view; plots, figures and scripts still work. The environment variable `LAGOON_NO_GPU` starts the window without the 3D view on purpose.

2.7 Optional parts

Films MP4 films need `imageio-ffmpeg`, which pip installs with LAGOON; it brings its own ffmpeg. Without it, numbered PNG frames are written instead.

Path tracing needs Blender 4.2 or newer (`https://www.blender.org`). LAGOON finds it in Applications (macOS), on the `PATH`, or where the environment variable `LAGOON_BLENDER` points (Chapter 16).

DIVEMesh The set-up runs DIVEMesh where it finds it: `LAGOON_DIVEMESH`, `DiveMESH` on the `PATH`, or a source build in `~/Codelite/DIVEMesh` (Section 5.10).

Map projections LAGOON transforms longitude and latitude, Web Mercator, UTM and the Norwegian NTM zones itself. For other coordinate reference systems, and to read every GeoTIFF and ICESat-2 file, install the geo extras: `pip install "lagoon-viz[geo]"` (pyproj, rasterio, h5py).

LaTeX Labels in paper figures may hold mathematics without a LaTeX installation (matplotlib's mathtext). With a LaTeX installation, *Labels through LaTeX* sets them in real LaTeX.

The LAGOON format alone Other codes can read and write the format with `pip install lagoon-format`, which needs only NumPy.

3. Getting Started

This chapter takes you from a finished REEF3D run to a first figure and a first film, and introduces the main window. Any REEF3D case folder will do; the pictures show a small wave flume computed with REEF3D::NHFLOW, with three wave gauges and two free surface lines. Chapter 5 starts from the other end: a case that has not run yet.

3.1 The Workflow at a Glance

LAGOON sits on both sides of a REEF3D run (Fig. 3.1). Before the run, it edits the input files in the case folder, makes `geo.dat` and the boundary files, and runs DIVEMesh. REEF3D then writes its results into folders named `REEF3D_<SOLVER>_<output>` in the case folder: the volume as VTU files, the free surface and the bed as VTP files, one file per MPI rank and output, and time series such as wave gauges as text files. LAGOON opens the case folder and joins the pieces of all ranks. Optionally, the output is kept in a LAGOON store instead, written by REEF3D during the run (**P 18**) or by `lagoon-convert` afterwards.

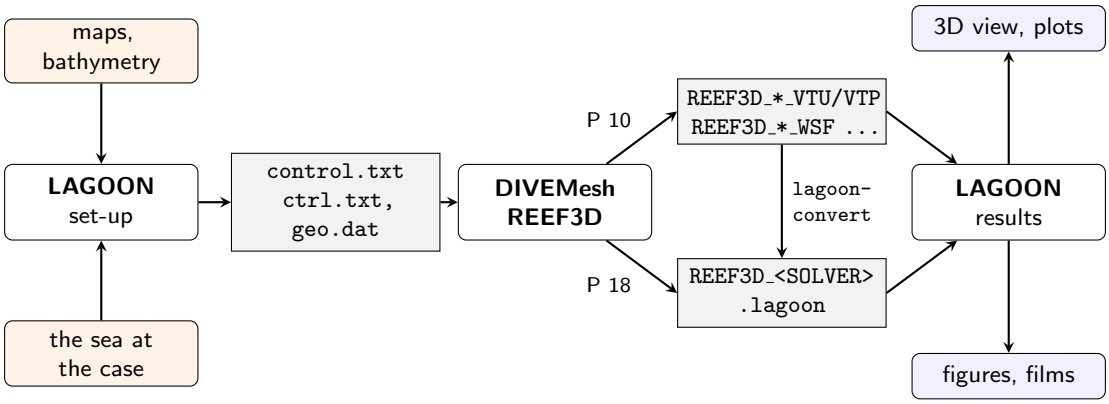


Figure 3.1: From the set-up of a case to pictures. LAGOON writes the input files of DIVEMesh and REEF3D, and reads the VTK and text output of the run, or its LAGOON store when there is one.

3.2 The Main Window

The main window (Fig. 3.2) has the 3D view in the middle and panels around it, which can be moved, tabbed and closed; *View* → *Reset layout* brings them back to their places, and the *View* menu shows or hides each of them. The side panels can be narrowed freely: below the width their content needs they scroll sideways.

LAGOON works in one of three modes, switched with the three buttons in the tool bar:

Set-up (`Ctrl+Shift+U`) the input files of the cases and DIVEMesh's grid (Chapter 5);

Results (`Ctrl+Shift+R`) what the runs wrote, the mode a case with results opens in;

2D plots (`Ctrl+Shift+D`) the figure editor in place of the 3D view (Chapter 13).

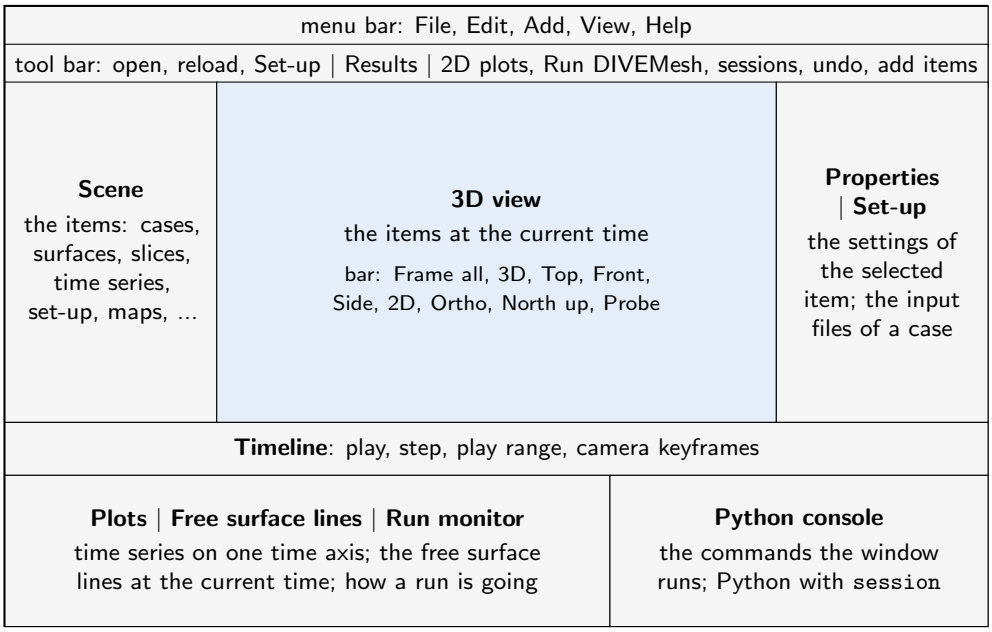


Figure 3.2: The main window of LAGOON and its panels.

Scene lists everything that is shown: the open cases and the items drawn from them. The check box beside an item shows or hides it, a click selects it, and `Del` removes it (Chapter 9).

Properties shows the settings of the selected item: what it is coloured by, its colour map and range, its opacity, its vertical scale and so on. Every change is a command that can be undone with `Ctrl+Z`.

Set-up (tabbed with Properties) edits the input files of a case and holds the tabs for porous structures, solids, river corridors, multi-block grids, the map, the sea and DIVEMesh (Chapter 5).

3D view draws the items at the current time on the GPU (Chapter 10).

Timeline steps through the outputs of the open cases, plays them and holds the play range and the camera keyframes (Chapter 11).

Plots, Free surface lines and Run monitor show the time series (wave gauges, probes, motions, forces) on one time axis, the free surface lines at the current time (Chapter 12), and how the latest run of a case is going (Section 4.5).

Python console shows every command the window runs as a line of Python and runs Python with the current `session` (Chapter 18).

Over a number box, a choice list or a slider in the Properties and Set-up panels, the mouse wheel scrolls the panel; the field takes the wheel only once it is clicked into, so scrolling down a panel never changes a value by accident.

3.3 Your First Run

Step 1: Open the run

Start LAGOON and choose *File* → *Open case...* (Ctrl+Shift+O), then the folder REEF3D wrote its `REEF3D_*` folders into, which is the folder REEF3D ran in. Choosing its `REEF3D_Case` folder opens the case as well; a single output folder, such as `REEF3D_FNPF_WSF`, opens only that output. From a terminal:

```
lagoon ~/runs/flume
```

LAGOON finds what the run wrote and fills the Scene panel: the case, its free surface and its bed, its floating bodies, mooring lines and structures, and, switched off, its volume, wave gauges and free surface lines (Fig. 3.3). Tick an item to show it.

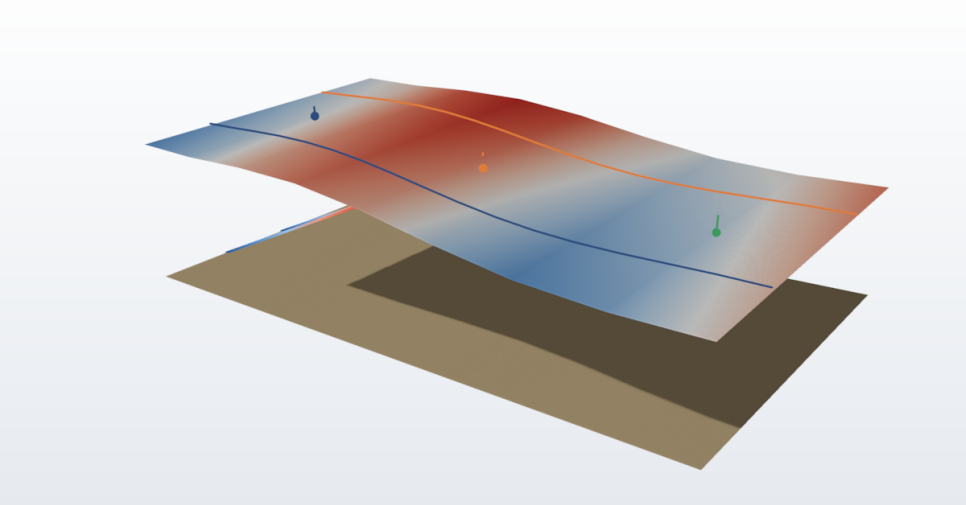


Figure 3.3: The example flume in the 3D view (Studio look): the free surface coloured by its elevation over the bed, the wave gauges as staffs and the free surface lines as lines.

Step 2: Look around

In the 3D view, drag with the left mouse button to turn the view, with the right button to move it, and use the wheel to zoom. The number keys give the standard views: 1 front, 3 side, 7 top, 5 perspective or orthographic; Home frames everything.

Click an item in the Scene panel to see its settings in the Properties panel. Set *Colour by* of the free surface to `velocity`, or raise the *Vertical scale* of the case to make small waves visible. *Probe* in the bar of the 3D view (P), then a click on the surface, lists every value there in a small floating window; the values follow the time (Section 9.5).

Step 3: Play

The Timeline plays the outputs (`Space` in the 3D view), steps through them (`Ctrl+Left`, `Ctrl+Right`) and scrubs with the slider. Outputs are read in the background and kept, so going back is instant.

Step 4: Plot

Tick the wave gauges in the Scene panel: they appear in the Plots panel, with forces and motions, all on one time axis. The orange line marks the current time: click or drag in a plot to move the 3D view to that moment. *View* → *Wave analysis...* gives their spectra and wave parameters (Section 12.4).

Step 5: Make a figure

File → *Export figure...* (`Ctrl+Shift+P`) draws the plots again as a paper figure (Fig. 3.4): choose a style (REEF3D, a serif journal, slides), a width (single, 1.5 or double column) and a file (PDF, SVG, EPS, PNG or TIFF). Chapter 14 describes the figures in detail.

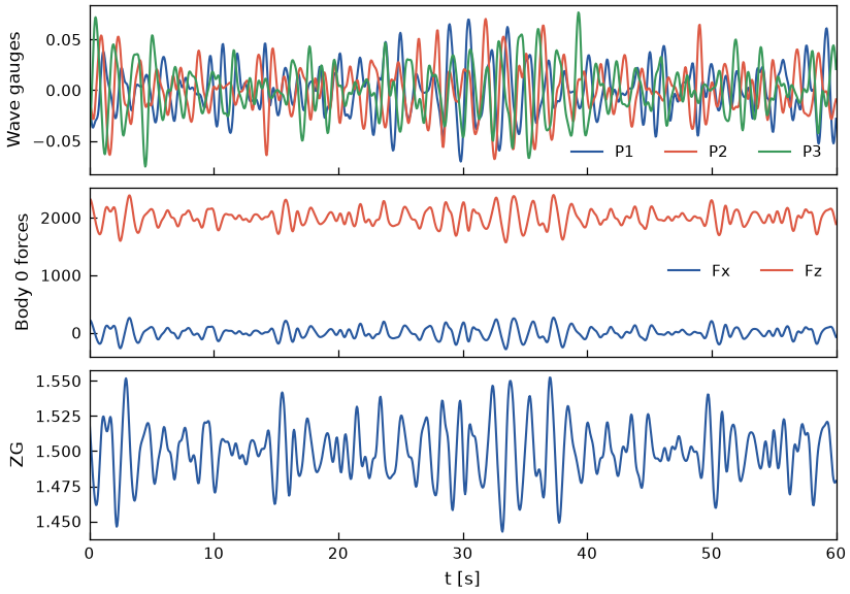


Figure 3.4: The wave gauges of the example as a paper figure.

Step 6: Make a film

File → *Export animation...* (`Ctrl+Shift+E`) films the 3D view as it is over the outputs you choose, into an MP4 file. For a moving camera, add camera keyframes on the timeline first (Chapter 11).

Step 7: Keep it

File → *Save session* (`Ctrl+S`) keeps everything, the scene, the settings, the view and the time, in a `.lagoon-session` file. *File* → *Export script...* writes the session as a Python script that does the same without the window:

```
lagoon-render flume.lagoon-session view.png   # the saved view as an image
python flume.py                                # the script: figures and films
```

4. Opening Runs

4.1 The Case Folder

File → Open case... takes the folder a REEF3D run wrote into, with its `REEF3D_*` folders, one of those output folders, or a LAGOON store. A run can also be opened from a terminal:

```
lagoon ~/runs/flume                          # a case folder
lagoon ~/runs/flume/REEF3D_Case              # the same case
lagoon ~/runs/flume/REEF3D_NHFLOW_VTP_FSF    # one output folder
lagoon ~/runs/flume/REEF3D_NHFLOW.lagoon     # a LAGOON store
lagoon figure.lagoon-session                 # a saved session
```

File → Open output file... opens one output series. It lists a folder's files as ParaView does: the numbered files of a series as one line (`REEF3D-NHFLOW-FSF-..pvtp`, with the number of files, their first and last number and their size), and the pieces of parallel output (the `.vtp` and `.vtu` files next to their `.pvtp` and `.pvtu`) left out. Double-click a folder to go in (*Up* goes out), a series or a file to open it with its case.

The case folder is the folder REEF3D ran in. `REEF3D_Case` inside it holds REEF3D's run log and the copies of the control files; choosing it opens the whole case, named after the case folder. A single output folder opens only that output. Table 4.1 lists the folders LAGOON reads and the scene items they become; <S> stands for the solver: `NHFLOW`, `FNPF`, `SFLOW` or `CFD`. Section 12.1 lists the time series files.

Table 4.1: The output folders of a REEF3D case and what LAGOON shows of them.

Folder	Shown as
REEF3D_<S>_VTP_FSF	Free surface (VTP, one piece per MPI rank)
REEF3D_<S>_VTP_BED	Bed
REEF3D_<S>_VTU	Volume; slices, iso-surfaces, iso-volumes and streamlines of it; for CFD the free surface ($\phi = 0$)
REEF3D_<S>_AMR	Free Surface AMR: the refined patches over the coarse grid (FNPF, NHFLOW, SFLOW); for CFD the refined 3D blocks that join the volume, and the AMR blocks item
REEF3D_<S>_6DOF_VTP	Bodies: floating bodies (6DOF)
REEF3D_<S>_6DOF_Mooring	Mooring lines, coloured by tension
REEF3D_<S>_VRANS_VTP	Structures: porous structures
REEF3D_NHFLOW_FORCING_VTP	Structures: NHFLOW's direct forcing solids
REEF3D_DEM_VTP	DEM elements
REEF3D_<S>_ICE	Ice floes (FNPF)
REEF3D_<S>_RodTree	Rod trees (corals, vegetation)
REEF3D_<S>_Particles, REEF3D_CFD_CPM_Particle	Particles (NHFLOW's Lagrangian particles, CFD's CPM sediment particles)
REEF3D_SEASTATE_VTP, REEF3D_SEASTATE_AMR	Sea state: SEASTATE's maps; its AMR output as a Free Surface AMR

Folder	Shown as
REEF3D_<S>_WSF, _ProbePoint, _Force, _6DOF, ...	Time series: wave gauges, probes, forces, motions, run-up and bed gauges (Table 12.1)
REEF3D_<S>_WSFLINE, _WSFLINE_Y	Free surface lines along x and y, with the wave theory (FNPF)
DIVEMesh_Paraview	Structures: the solids of DIVEMesh (REEF3D_Solid.vtp); the bed of the set-up (REEF3D_Topo.vtp)
REEF3D_Case	the run log (REEF3D_<S>_run.jsonl) and the copies of the control files: times, runs
REEF3D_<S>.lagoon	a LAGOON store, read in place of the VTK files it holds

Each MPI rank's piece of an output is read and the pieces are joined; the parallel files (.pvtu, .pvtp) say which pieces belong together. Opening a case adds the case and its main items to the scene: the free surface (or the Free Surface AMR, when the run wrote one) and the bed, the objects of the same run (bodies, mooring lines, structures, DEM elements, ice floes, rod trees, particles), and its time series and free surface lines. The run's volume, its time series and its free surface lines are there too, switched off: tick them in the scene (or *Volume* in the tool bar) to show them. Items that are switched off are not read at all, which keeps large cases quick to open.

When a folder holds the output of several runs, the run of the solver that ctrl.txt sets up (**A 10**; without it CFD, REEF3D's default) and, of that, the run written last is shown. The other runs stay selectable as the *Dataset* of an item. Slices, iso-surfaces and iso-volumes are in the tool bar, streamlines and statistics in the *Add* menu.

CFD

A REEF3D::CFD run has no free surface file: it opens with its free surface as an iso-surface of the volume, *Free surface*, where the level set ϕ is 0 (coloured by the velocity), the volume itself switched off, and its floating bodies.

With adaptive mesh refinement, CFD writes the refined blocks of every output in REEF3D_CFD_AMR: an index per output (REEF3D-CFD-AMR-<output>.vtm) and its blocks, 3D grids of a half, a quarter, ... of the volume's cell size. LAGOON joins them to the volume (its label says *with AMR*): the volume's cells under a block are left out, and so are a block's cells under a finer block. The free surface, slices, iso-surfaces and iso-volumes are then as fine as the refinement, e.g. around a floating body; *Colour by* level shows where (0: the volume's own cells), and the *Grid lines* of the free surface draw where it crosses the cells' faces, of every level. The item *AMR blocks* (in the scene, switched off) draws the outlines of the blocks, coloured by their level. Only the blocks an output's index lists are read, so a file left over from an earlier run is not. Streamlines and time series at a point come from the volume's own cells. With **P 18**, REEF3D writes the blocks into the LAGOON store as well; LAGOON reads them from there the same way.

NHFLOW's direct forcing solids

NHFLOW's direct forcing solids (**A 581** – **A 589** in `ctrl.txt`: boxes, cylinders, jacket members, spheres, wedges) are a *Structures* item of the case: from the file NHFLOW writes (`REEF3D_NHFLOW_FORCING_VTP`, with its STL and DIVEMesh's solids), or, before a run or when `ctrl.txt` was changed since, built from `ctrl.txt`.

SEASTATE

REEF3D's spectral wave model writes maps of the sea state (`REEF3D_SEASTATE_VTP`). They open as a *Sea state* item, coloured by the significant wave height `Hs` on the rainbow map, as SEASTATE's own maps, with arrows where the waves go and dry land left out. *Colour by* offers `Tm01`, `Tm-10`, `Tp`, `dir`, `spread`, `depth` and whatever the run wrote besides (the wind `U10x`, `U10y`; coupled to a flow solver, the wave forces `Fx`, `Fy` and the Stokes transport `MStx`, `MSty`; with rollers `roller`, `Dr`, `Dw`). *Vectors of* offers `wave_direction` (unit arrows), `wave_vector` (as long as H_s), `wind`, `wave_force` and `stokes_transport`, which LAGOON makes from them. SEASTATE's AMR output (`REEF3D_SEASTATE_AMR`) opens as a Free Surface AMR coloured by H_s, its refined patches outlined, at the times SEASTATE wrote.

4.2 The Solids of DIVEMesh

The solids of a case, such as a monopile, a breakwater or the blocks that shape a beach, come from DIVEMesh. LAGOON reads `DIVEMesh_Paraview/REEF3D_Solid.vtp`, which DIVEMesh writes with every solid of `control.txt`, STL geometries included. Without it, the common solids are built from `control.txt` itself: boxes, cylinders, cones, spheres, wedges and polyhedra (**S 10**, **S 32**, **S 33**, **S 41** – **S 43**, **S 51**, **S 61** – **S 63**, **S 81** – **S 84**). They are shown as one structure, *DIVEMesh solids*; the faces that lie in the bed REEF3D writes are left out.

Only the case's own solids are opened. DIVEMesh leaves `REEF3D_Solid.vtp` alone when a case has no solids, so in a folder where several cases run the file can be another case's. LAGOON passes it over when

- `control.txt` has no solid commands (**S**): the case has none;
- DIVEMesh has run since without writing it (its partition and grid preview in `DIVEMesh_Paraview` are more than an hour newer);
- the solids lie outside the domain of the run shown (from the run log).

When `control.txt` has changed since DIVEMesh wrote the file, its solids are built from `control.txt` instead. DIVEMesh's bed (`REEF3D_Topo.vtp`) left over from an earlier DIVEMesh run is passed over in the same way.

4.3 Time

The timeline steps through REEF3D's outputs. Their times come from the run log when REEF3D wrote one (`REEF3D_Case/REEF3D_<S>_run.jsonl`), else from the output interval (**P 30**) of the copy of `ctrl.txt` the run was started with (`REEF3D_Case/ctrl-<run>.txt`, else `ctrl.txt`); without either, the outputs are numbered. The case's *Output interval* property sets the interval by hand. Objects with their own output interval (DEM elements, ice floes, rod trees, particles) take their times from their files.

4.4 Several Runs

More cases can be opened into the same session; each item has a *Case* property naming the case it shows. *View* → *Compare side by side* (Ctrl+Shift+2) shows two cases in two 3D views with linked cameras and time (Section 10.5).

4.5 A Running Simulation

File → *Reload case* (F5) reads the case folders again: new outputs of a running simulation appear, while the scene, the time and the camera stay, and the results stay on the screen until the new frames are read. *File* → *Follow running simulation* reloads by itself whenever the simulation writes, and keeps the timeline at the newest output. A LAGOON store is read while REEF3D appends to it.

The run monitor

The *Run monitor* (a tab beside the plots) shows how the latest run of a case is going, read every few seconds from its run log (`REEF3D_Case`) and `REEF3D_Log`:

- whether it is running, finished, stopped, or **not responding** (no end written and no iteration for much longer than one takes: killed, crashed or hanging);
- the iteration and the simulation time, against **N 45** and **N 41** of the control file the run was started with, as a progress bar;
- the time step, the speed (simulated seconds per wall hour, wall seconds per iteration over the last 50), the wall time so far, and the time to the end with the clock time it will be done;
- the outputs written and the largest velocity;
- small plots of the time step and the largest velocity over the last iterations.

Warnings say when the time step drops to a tenth of the usual, the largest velocity rises to five times the usual, a value is NaN, or `ctrl.txt` was changed after the run started: the run goes on with the copy in `REEF3D_Case`, and the changed lines are listed (− as the run has it, + as `ctrl.txt` has it now). *Follow* is *File* → *Follow running simulation*.

4.6 Cleaning Up a Case Folder

A case folder collects the output of every run started in it: REEF3D overwrites only the files it writes again, so a shorter run leaves the later outputs of the one before, another solver's run leaves its folders, and a new LAGOON store renames the one before to `*.lagoon-earlier`. *File→ Clean up case folder...* lists what is not the run shown, by run, with its size: output left over from earlier runs and earlier stores come ticked, another solver's run is listed but not ticked. *Move to LAGOON_archive* moves the ticked groups into `LAGOON_archive/<date>/` of the folder, their paths kept (to be put back by hand); *Delete* deletes them. The run shown and DIVEMesh's grid and input files are never offered.

4.7 LAGOON Stores

A run converted to the LAGOON format (*File→ Convert to LAGOON format...* or `lagoon-convert`, Chapter 17) is several times smaller and opens faster. Open the case folder as usual: a store in it (`REEF3D_<S>.lagoon`) is read in place of the VTK files it holds, unless the VTK files have newer outputs (a run that went on and was not converted again). The items then say *(LAGOON)* in their names. A store folder can also be opened on its own; it then shows only its solver's run, and the solids of DIVEMesh and `control.txt`, which are read from the case folder, are missing. Opening the case folder is therefore usually better than opening the store alone.

REEF3D writes the store itself during the run with **P 18 1** (instead of the VTU/VTP files) or **P 18 2** (as well as them). It adds an output to the store when it writes the next one: a store opened just after the run started has no outputs yet; reload once the run has written more. A store REEF3D started on top of an earlier run's shows the latest run only.

5. Case Set-Up

LAGOON shows a case before it has run: open a folder with DIVEMesh's `control.txt` and REEF3D's `ctrl.txt` (*File* → *Open case...*) and you see what they set up. The input files are edited in the Set-up panel, checked before a run and handed to DIVEMesh, whose grid LAGOON then shows. The same signs can be laid over the results of a run, and drawn as a paper figure in plan and side view.

5.1 Set-Up and Results

LAGOON works in one of two modes, switched with the buttons in the tool bar (*View* → *Set-up*, `Ctrl+Shift+U`, and *View* → *Results*, `Ctrl+Shift+R`):

Set-up the input files of the cases. The *Case set-up* items and, once DIVEMesh has run, its grid are shown; what the runs wrote is left out (listed greyed in the scene) and the timeline rests. Each case with input files gets a *Case set-up* item when it has none. The Set-up panel comes forward.

Results what the runs wrote. The *Case set-up* items are there too, switched off: tick one to see where the gauges and zones were. The Set-up panel makes way for *Properties*.

Set-up items keep a check box of their own in each mode, so switching back and forth shows each mode as you left it. A case that has not run yet opens in the set-up. The switch is a command like any other: it can be undone, and a script switches with `session.set_mode("setup")` or `session.set_mode("results")`. The third button, *2D plots* (`Ctrl+Shift+D`), puts the figure editor in place of the 3D view without changing the mode of the session (Chapter 13).

5.2 Starting a Case

File → *Open case...* opens a folder that has input files or output. For a case that does not exist yet, *File* → *New case...* asks for a folder (make one in the dialog with *New Folder*), starts its `control.txt` and `ctrl.txt` with a comment line, and opens it in the set-up. Add DIVEMesh's commands in the `control.txt` tab of the Set-up panel (the domain **B 10**, the cell size **B 1**, the sides **C 11** – **C 16**), then REEF3D's in the `ctrl.txt` tab, and *Save*. In a script: `session.new_case("~/runs/flume")`.

File → *New case from map...* starts a case at a place on the Earth instead: a draft with a domain and a background map, which you move and turn on the map, give a bathymetry from the web and then put in its folder with *Create case...* (Section 7.1). With no case open, the *Map* tab of the Set-up panel starts one at Trondheim.

5.3 A Case That Has Not Run Yet

A case folder without output opens as its set-up:

- the **domain** (**B 10** of `control.txt`), its grid (**B 1** or **B 2**) and the kinds of its sides (**C 11** – **C 16**): inflow, outflow, wave generation, numerical beach, symmetry plane, wall;
- the **still water level** (**F 60** of `ctrl.txt`), the **relaxation zones** of wave generation and numerical beach (**B 96**, or **B 107** and **B 108**), the wave direction (**B 105**), the focus point (**B 81**) and wavemakers (**X 170** – **X 172**);
- **gauges** and **probes** (**P 50** – **P 67**, **P 88**, **P 121** – **P 148**), numbered as in the output files, discharge **sections** (**P 167**, **P 168**), **force** boxes (**P 81**) and Morison cylinders (**P 85**);
- floating **bodies** (**X 110** – **X 164**), their centre of gravity (**X 23**) and moorings (**X 311**, **X 312**), nets (**X 322**);
- direct forcing and porous **structures** (**A 581** – **A 589**, **B 210** – **B 291**), and other regions (**A 276**, **A 372**, **F 72**, **S 74**);
- DIVEMesh's **solids** (**S**), as a structure item, and the **bed**: its topo objects (**T**);
- the points of `geo.dat`, as an item of their own (Section 5.5).

The vertical scale of the case applies to the set-up as to any other item (Section 9.4), and the 2D button draws it flat from above. A long, flat domain such as a flume is easiest to read from the side (the 2D view's front side) with a vertical scale.

5.4 The Case Set-Up Item

A case that has run gets the same item when its folder has input files, switched off: tick it in the scene to see where the gauges and zones were over the results. `ctrl.txt` may also be the copy REEF3D keeps in `REEF3D_Case`, and without `control.txt` (or **B 10** in it) the domain is the extent of the results. *Add* → *Case set-up* adds another one. Each group can be switched off in its properties (Table 5.1).

Table 5.1: The groups of the Case set-up item.

Property	Shows	Commands
Domain	the domain's box	B 10
Sides	the kinds of the domain's sides, labelled	C 11 – C 16
Still water	the still water level, a faint plane	F 60
Relaxation zones	wave generation and numerical beach zones	B 96, B 107, B 108
Waves	wave direction, focus point, wavemakers	B 105, B 81, X 170 – X 172

Property	Shows	Commands
Gauges	wave gauges and free surface lines, numbered	P 50 – P 58, P 121 – P 147
Probes	point, line and velocity probes, numbered	P 61 – P 67, P 88, P 144 – P 148
Sections	discharge sections and flow files	P 167, P 168, P 230, P 240
Forces	force boxes and Morison cylinders	P 81, P 85
Floating bodies	bodies, centre of gravity, moorings, nets	X 23, X 110 – X 164, X 311 – X 322
Structures	direct forcing and porous structures	A 581 – A 589, B 210 – B 291
Regions	refinement boxes, wind areas, prescribed water levels	A 276, A 277, A 372, F 72, S 74
Solids	outlines of DIVEMesh's solids (off: they are a structure item)	S
Bed	DIVEMesh's topo objects	T
Curvilinear grid	the grid of `curvgrid.dat` or `blocks.dat` where its nodes are	G 50
River drawn	the river drawn on the map (Section 6.1)	
Labels	the numbers of gauges and probes, the kinds of the sides	

Commands LAGOON cannot place (an unknown shape, too few values) are listed in the set-up's notes (`lagoon-setup CASE --describe`).

5.5 The Points of geo.dat

A case with a `geo.dat` gets a *geo.dat points* item in the scene, beside *Case set-up* and *DIVEMesh grid*: every point of the file as DIVEMesh reads it (scaled with **G 12**, z turned with **G 23**), a dot each, coloured by its height (*Colour by* `elevation` or `depth`, any colour map). Zoom in, in the 2D view from the top, to see how far apart they lie; *Point size* makes the dots smaller or larger. Its properties say how many points there are, their spacing (the grid's dx and dy for a regular grid such as the one *Make geo.dat* writes, else the median distance to the nearest point, for soundings), and their extent and heights. Like the other set-up items it has a check box of its own in each mode: on in the set-up, off over the results until ticked. *Make geo.dat* (Section 7.5) adds it and draws it again whenever it writes the file.

5.6 Editing the Input Files

The Set-up panel (beside *Properties*; it comes forward in the set-up) edits a case's input files: one tab for `control.txt` (DIVEMesh), one for `ctrl.txt` (REEF3D). Its further tabs are *Porous*, *Solids*, *River*, *Blocks*, *Map*, *Sea* and *DIVEMesh*. Each file's tab lists the commands of its file by section (*B · Boundary*, *P · Printer*, ...); a command given several times (gauges, solids) has a line for each, numbered as in the output files.

Select a command to see its values, ready to change, with what the user guide says about it: its default, which REEF3D modules use it, whether it may be given several times. Its signs light up in the 3D view (yellow), so a gauge or a zone can be checked where it is. Choices (**C 11** – **C 16**, **A 10**, ...) are picked from a list.

Add finds any command of the program by its number (`P 61`) or by words (`wave gauge`, `relaxation`); it is added with its default values, after the other commands of its letter. *Add another* adds one more line of a command that may be given several times.

Remove takes the selected line out.

Text edits the whole file as text (*Apply text*).

Commands the program would stumble over are marked: red for what stops it or is misread (a missing value, text with a capital letter and no `//`), orange for what it ignores (an unknown command, one given twice, one the module does not use); the tooltip says why.

Every change is an undo step and a line of the session's script, and the 3D view follows at once:

```
session.set_input("ctrl.txt", "P 51", [3.0, 2.5], occurrence=1)
```

The files themselves are written with *Save* (or *File* → *Save input files*); a tab with edits not saved yet has a *, and LAGOON asks before closing. Comments, blank lines and the order of the file are kept: only the lines that changed are rewritten.

5.7 Porous Structures

The *Porous* tab of the Set-up panel holds the porous structures of a case (VRANS: breakwaters, rubble mounds, porous cylinders), as the `ctrl.txt` lines that make them (Table 5.2).

Table 5.2: The porous structures of the Porous tab.

Module	Structures	Material
CFD	box (B 270), vertical cylinder (B 274), wedge along x (B 281) or y (B 282), plate along x (B 291)	on each line: n, d_{50}, α, β
NHFLOW	box (B 210), cylinder along y (B 212), vertical cylinder (B 213), sphere (B 215), wedge along x (B 217) or y (B 218)	one for all: B 201 n d_{50} α β

Add puts the kind chosen in the middle of the domain (in a 2D case across it), from the bed to the still water level (**F 60**), of rubble: $n = 0.4$, $d_{50} = 0.05$ m, $\alpha = 500$, $\beta = 2.0$. For NHFLOW, **B 201** comes with the first structure.

Draw drags its plan in the 2D top view of the 3D view (Esc stops): a box or wedge over the area, the largest cylinder or sphere in it, snapped to the cells (**B 1**). It reaches from the bed to the still water.

Select one to see its shape and material, ready to change; it lights up in the 3D view. *Remove* takes it out.

A wedge's top runs from its first height at its start to its second at its end, its base at the lower of the two (a slope rising along x: z start at the bed, z end at the crest). A plate is a band of the thickness given above that line. n is the porosity, d_{50} the mean stone diameter, α and β the linear and nonlinear resistance coefficients of the Forchheimer terms. SFLOW and FNPF have no porous structures. Each change is an undo step, saved with the input files; in a script (no values: the default place and material):

```
session.add_structure("B 270", [30, 40, 0, 1, 0, 0.6, 0.4, 0.05, 500, 2])
```

5.8 Solids

The *Solids* tab works the same way for solids: DIVEMesh's box (**S 10**), vertical cylinder over the whole height (**S 33**), cylinder along y across the domain (**S 32**), sphere (**S 51**) and wedges along x (**S 61**) or y (**S 62**) in `control.txt`, for every module; and NHFLOW's direct forcing box (**A 581**), cylinder along y (**A 583**), vertical cylinder (**A 584**), sphere (**A 586**) and wedges along x (**A 587**) or y (**A 588**) in `ctrl.txt`. *Add* and *Draw* make a solid from the bed to the domain's top (for NHFLOW, whose domain ends at the still water, up to half the water depth above it). DIVEMesh's solids are meshed by *Run DIVEMesh*; NHFLOW takes its solids as **A 580** says (0: they raise the bed, 1: immersed). In a script: `session.add_structure("S 33", [12.0, 2.5, 0.4])`.

5.9 Checking the Input

Check (in the Set-up panel) looks at the input before a run, as it is saved:

- each file's commands, as for the editor (unknown commands, missing values, a command with no effect in the solver);
- the number of processes (**M 10**) in both files, and DIVEMesh's grid file for each (`DIVEMesh_Grid/grid-000001.dat`, ...): a grid made for fewer processes, none at all, or a `control.txt`, `geo.dat` or `solid.stl` changed after DIVEMesh made it;
- no 3D output (neither **P 20** nor **P 30**), no end (neither **N 41** nor **N 45**);
- what the run will cost: the cells, per process (a note for very few or very many), and, when the last run in the folder had the same grid, the time per iteration and for **N 41**.

Errors (REEF3D or DIVEMesh would stop) come first, then warnings and notes. `session.check_input()` gives the same in a script (`print(report.text())`).

5.10 Running DIVEMesh

Run DIVEMesh (in the Set-up panel, in the tool bar in the set-up, and *File* → *Run DIVEMesh*) saves `control.txt` and runs DIVEMesh in the case folder; its output shows in the panel's *DIVEMesh* tab and is kept in `DIVEMesh_Log/DIVEMesh-run.log`. When it has finished, the set-up shows what it made: its grid, the solids (`REEF3D_Solid.vtp`) and the bed (`REEF3D_Topo.vtp`, as a *Bed* item of the set-up).

LAGOON looks for the program as `LAGOON_DIVEMESH`, `DiveMESH` on the `PATH`, then in a source build (`~/Codelite/DIVEMesh/bin/DiveMESH` from make, `~/Codelite/DIVEMesh/src/Release/DIVEMesh` from CodeLite); if none is there, it asks for it once and remembers it.

5.11 DIVEMesh's Grid

The *DIVEMesh grid* item draws the grid lines on the faces of the domain: by default the bottom and the two faces at the back of the usual view (*Faces*: bottom and back, bottom, back, outside). A fine grid is drawn with every n-th line, so a face has a few hundred lines at most (*Every n-th line*: 0 for that, 1 for every line); the planes between the MPI ranks of DIVEMesh's partition (**M 10**) are drawn in orange. Before DIVEMesh has run, the item shows the uniform grid `control.txt` sets up (**B 10** with **B 1** or **B 2**); a stretched grid (**B 101** – **B 103**) shows once DIVEMesh has made it. The item's properties give the number of cells and the smallest and largest cell sizes along x, y and z.

FNPF and NHFLOW scale DIVEMesh's heights to σ (0 – 1) and stretch them from the bed to the water. Their grid, and the domain, are drawn from the bed's lowest point (`geo.dat`, else the bottom of **B 10**) to the still water level (**F 60**), with the levels spaced as DIVEMesh spaced them: as ParaView shows the results at the start, whatever heights **B 10** gives.

5.12 The Set-Up Sketch

File → *Export figure...* → *Case set-up* draws the set-up as a paper figure (Fig. 5.1):

- the **plan view**: the bed in colour (`geo.dat`, or the bed of the results) with its scale and the coastline at the still water level; the sides (inflow with its discharge **W 10**, outflow written beside them), relaxation zones hatched, gauges and probes numbered, DIVEMesh's solids filled; for a refined grid (**G 1**) the refinement boxes (**G 10**) and faint lines of the level-0 grid;
- the **side view**: a section through the middle of the domain, the bed filled with sand and the water above it, the zones it crosses hatched; a long, flat domain gets a vertical scale, given in the title (only the side view for a 2D case);
- a **table of the main facts**: module, domain, cells, still water, waves, generation and absorption, inflow, simulated time, output interval, refinement, processes;

- for a refined grid, **the levels**: their cell size and about how many columns each has (level 0 the whole domain, the finest the boxes, those between with a nesting buffer of **G 3** cells), against a uniform grid as fine as the finest level; cells the run flags (**G 12**, **G 20** – **G 23**) come on top;
- the legend.

For FNPF, NHFLOW and SFLOW, whose σ-grids reach from the bed to the free surface, the heights come from the bed (`geo.dat`) and the still water level, whatever heights **B 10** gives. A set-up figure is at least a column and a half wide (140 mm); narrow figures take the facts and the legend in one column.

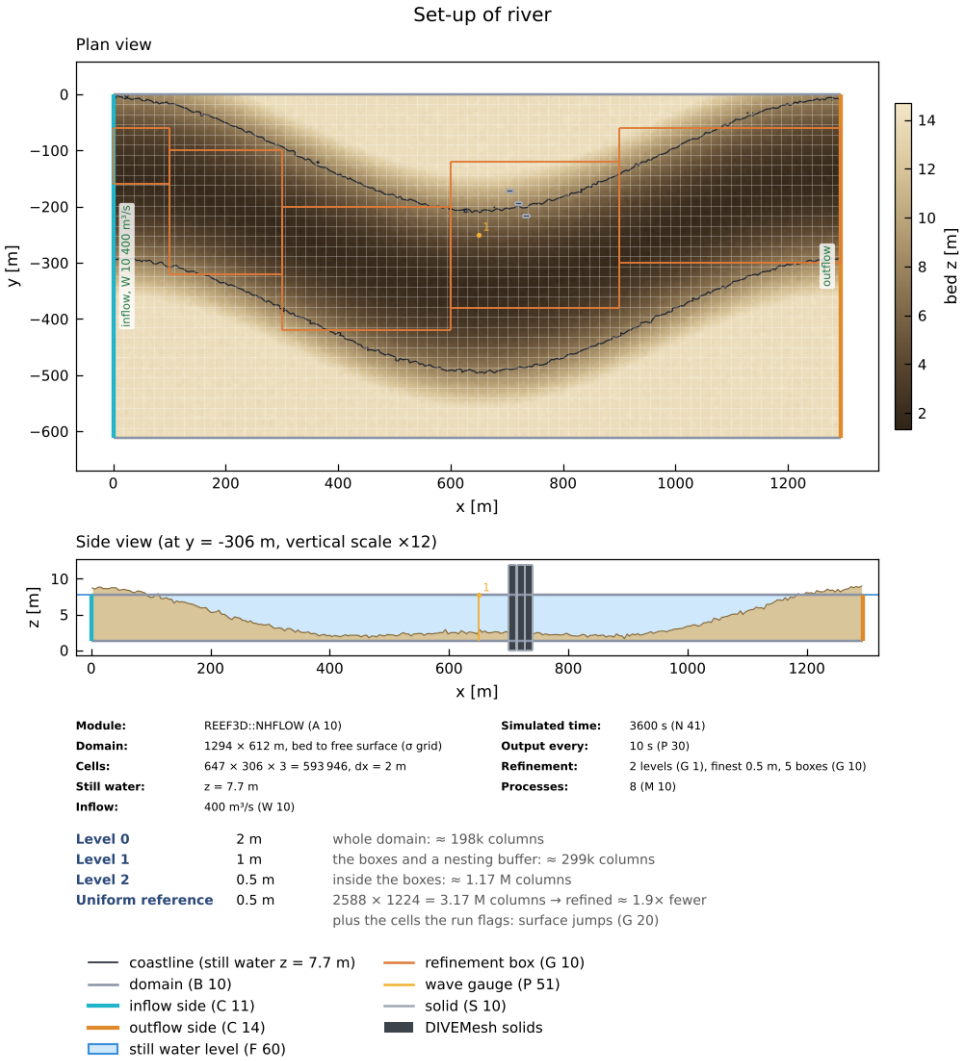


Figure 5.1: The set-up sketch of a river case computed with REEF3D::NHFLOW on a refined grid: the plan view with the bed, the refinement boxes and a wave gauge, the side view, the main facts and the levels of the refinement.

The same from the command line:

```
lagoon-setup ~/runs/flume                  # flume-setup.pdf beside the case
lagoon-setup ~/runs/flume sketch.png --width 190
lagoon-setup ~/runs/flume --describe       # the main facts as text
```

`--describe` lists the files found, the module (**A 10**), the domain and grid, the kinds of the sides, the still water level and waves, how many gauges, probes and other signs there are, and what does not fit together (e.g. **M 10** differs between `control.txt` and `ctrl.txt`: REEF3D would stop).

In a script, `read_setup` gives the set-up of a folder and `setup_figure` its sketch, and a session edits the input files and runs DIVEMesh the same way the panel does:

```
from lagoonviz.core.data.setup import read_setup
from lagoonviz.render.figure import FigureSpec, save_figure
from lagoonviz.render.setup_sketch import setup_figure

setup = read_setup("~/runs/flume")
print(setup.describe())
save_figure(setup_figure(setup, FigureSpec(width_mm=140)), "flume-setup.pdf")

from lagoonviz.core import Session
session = Session()
session.open_case("~/runs/flume")                    # not run yet: the set-up
session.set_input("control.txt", "B 1", [0.025])      # finer cells
session.add_input("ctrl.txt", "P 51", [12.0, 0.5])    # one more wave gauge
session.save_inputs()
session.run_divemesh(on_line=print)                   # its grid, solids and bed
```

6. River Corridors and Multi-Block Grids

REEF3D's SFLOW and NHFLOW compute on curvilinear grids as well as on DIVEMesh's rectilinear one: a single curvilinear grid (**G 50 2**, `curvgrid.dat`) for a river reach, or several blocks of curvilinear cells that meet along their faces (**G 50 1**, `blocks.dat`) for a coast, an estuary or a river running into a harbour. The *River* and *Blocks* tabs of the Set-up panel lay them out over the bed, write them and set the input files up for them.

6.1 A River Corridor

The *River* tab lays a river corridor: two banks shaped by points you drag, and the curvilinear grid between them.

Laying it out

Start lays a straight corridor along the domain (a tenth in from each side across), **Draw** one over an area you drag in the 2D top view; it flows along the area's longer side, with 12 points on each bank.

Follow thalweg (with `geo.dat`) lays the corridor along the river in the bed: the thalweg, the path along the deepest channel, from the deepest point of the inflow side (**C 1x** $= 1$) to the deepest of the outflow side (**C 1x** $= 2$; without them, from end to end of the domain's longer axis), smoothed. Its banks lie where the water reaches across it (*Water level*: **F 60** by default; without **F 60**, 0 when the bed's heights reach below and above 0, as at a coast or an estuary, else halfway up the banks) plus the *Margin*, never more than 2.5 times the river's usual half width from the thalweg (a marsh or a harbour basin does not widen it), smoothed along the river, with *Points per bank* points to drag afterwards. The bed of its grid comes from `geo.dat`.

Draw the river (before there is a bed: a new case started on the map): click the river in the 2D top view over the map as a line, point by point from upstream; click the last point again (a double click) or press `Enter` to end, `Backspace` takes the last point back, `Esc` stops, and dragging away from the points pans the view as always. The line is drawn in yellow, "upstream" at its start, and kept in `river.json`. *Continue drawing* takes it up again: a click adds a point at the downstream end, a click on a segment puts a point into it, a point is dragged to move it, and a right click (or `Alt` + click) on a point removes it.

Domain and geo.dat sets the domain (**B 10**'s x and y) around the river drawn, a *River width* beyond its line and on DIVEMesh's cells, and makes `geo.dat` there with the Map tab's source (Section 7.5).

Create river then makes the corridor along the line, *River width* wide, and its grid (20 sweeps of smoothing); with `geo.dat` it follows the river in the bed, as *Fit to the bed* does. Draw or continue the river again and *Create river* makes the corridor anew.

Fit to the bed (with `geo.dat`) makes the corridor as it is (created along the river, laid over an area or dragged) follow the river in the bed: the thalweg within the corridor, from the deepest point of its upstream end to the deepest of its downstream end, the banks where the water reaches (*Water level*) plus the *Margin*, at most half as far again as the corridor was wide. Its points per bank, curve and grid's cells stay.

Drag points (on after Start, Draw, Fit to the bed and Follow thalweg): press on a point of a bank in the 2D top view and drag it; the bank follows at once. Pressing anywhere else pans the view; Esc stops. Each point moved is an undo step.

Points per bank (4 – 60) spaces that many points along the banks as they are now: more for a winding river, fewer for a calm one.

Curve *through the points* (a centripetal Catmull-Rom spline: the bank passes every point, without loops or overshoot between close points) or *smooth* (a clamped cubic B-spline the points pull on; it passes the first and the last). The banks and their points are drawn in yellow.

Reverse flow swaps upstream and downstream. The bank on the left looking downstream is the left bank; "upstream" is written at the corridor's start.

Its grid

The grid has *Cells along* × *Cells across* cells, its nodes on straight cross-sections between the banks. *Cross-sections*: **square to the river** (from points spaced equally along the line halfway between the banks, straight across it to both banks; where they would cross on the inside of a tight bend they are spaced evenly there), or **equal lengths** along both banks, paired (skewed in bends where one bank is longer). Along the banks, square cross-sections are evened out so that no cell is much shorter or longer than its neighbour (at most 1.2 times) or than an even one (0.6 to 1.6 times). A corridor along the thalweg starts so; *Tidy the grid* (under *Follow thalweg*) makes the grid even again after points were dragged: these cross-sections and 20 sweeps of smoothing.

Smoothing runs that many sweeps of Winslow's elliptic grid equations [Winslow, 1966] over the inner nodes (the banks and the two ends stay): smoother, more even cells in bends. The set-up draws the grid as it changes, and the tab says how large its cells are, their smallest angle and whether any fold (REEF3D stops on a folded cell: move the points, use fewer cells along or more smoothing). The bed (z_b at the nodes) slopes down from the upstream end (*Bed at upstream end*, *Bed slope* in m/m), or comes from `geo.dat`'s points. While there is a corridor, the set-up draws its grid only; the River tab says when `curvgrid.dat` holds another grid.

Make grid writes `curvgrid.dat` (`ni nj`, then x y z_b of every node, i fastest), sets **G 50 2** in `ctrl.txt` and makes DIVEMesh's grid the logical box of the curvilinear one (**B 10**'s x and y and **B 1**: ni × nj cells; its heights kept), one undo step. Then *Save* and *Run DIVEMesh*. The sides keep their numbers: 1 is the upstream end ($i = 0$), 4 the downstream end, 3 and 2 the banks. With **G 50 2**, the set-up draws the domain, its sides and the grid (*Curvilinear grid*, on the bed) where the nodes are, not DIVEMesh's logical box.

The River corridor item

The corridor is an item of the scene, *River corridor* (*Add* → *River corridor* lays one out at once: along the thalweg with `geo.dat`, else along the domain). It draws the banks and their points in yellow and the grid it makes; *Points* and *Grid* in its properties switch them off. Selecting it in the set-up brings the River tab forward. *Remove* takes the corridor away with its `corridor.json`; *Reset* lays it out again the way it was made (along the domain, over the area drawn, along the river drawn on the map, along the thalweg with its margin, or fitted to the bed from any of these), dropping the dragged points and keeping the grid's settings. The corridor is kept in `corridor.json` in the case folder. In a script:

```
session.start_corridor([0, 200, 0, 20], points=12)   # or along the thalweg:
session.auto_corridor(margin=5.0, points=12)
# or the river drawn on the map first, the domain and the bed made for it:
session.draw_river([(0, 92), (125, 106), (250, 96), (400, 90)])
session.corridor_domain(width=60.0)   # B 10 around it; then make_geo_dat("best")
session.create_river(60.0, points=12, margin=2.0)   # fitted to the bed
session.move_corridor_point("left", 5, 90.0, 28.0)
session.set_corridor_options(points=16, curve="bspline", cells_along=100,
                             cells_across=10, smoothing=100, slope=0.001)
session.reset_corridor()     # as it was made, the dragged points dropped
session.make_corridor_grid()
```

6.2 A Multi-Block Grid

The *Blocks* tab lays a multi-block grid: blocks of curvilinear cells that meet along their faces, node on node, for REEF3D's SFLOW and NHFLOW (**G 50 1**, DIVEMesh **R 20 1**, `blocks.dat`). Block 0 is the grid of the run (DIVEMesh's logical box); the blocks 1, 2, ... lie beside it: a river reach and the harbour it runs into, say, or the sea, the narrows and the bay of an estuary (Fig. 6.1).

The topology

The topology is kept in `blocks.json` in the case folder: the *corners*, the *edges* between two corners (cells, a grading ratio, straight or curved through their points) and the *blocks*, each with its four faces `i0`, `i1`, `j0`, `j1` ($-i$, $+i$, $-j$, $+j$) as chains of edges (`"-e"` runs edge `e` backwards). Blocks that share an edge share its nodes: that is the interface. Each block is meshed by transfinite interpolation from its faces, then smoothed by Winslow's equations: on grids half as fine first, then by red-black over-relaxed sweeps until no node moves more than a ten-thousandth of a cell (*Smoothing*: the most sweeps on each grid). The nodes of a wall slide along it for the first 200 sweeps, so cells stay square at the banks (not on a face that is partly an interface, a river joining it: there they stay even). A change to one block makes only that block again.

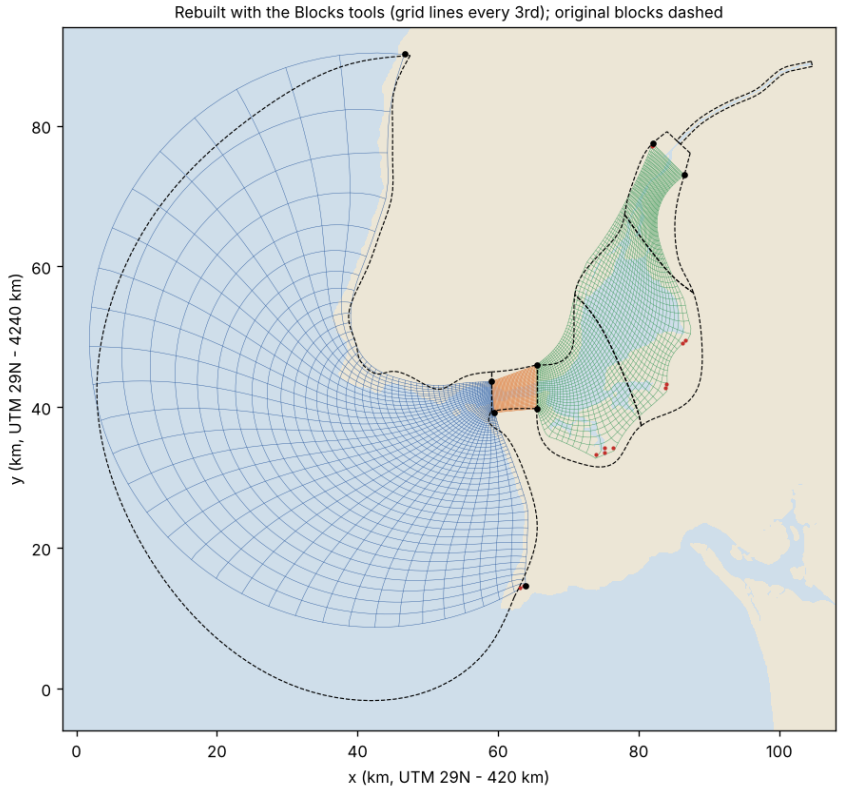


Figure 6.1: A multi-block grid of the Tagus estuary laid out with the Blocks tab (every third grid line drawn): the sea block with an arc as its open side and edges along the water, spaced from fine at the mouth to coarse offshore, the narrows and the bay. The blocks of an earlier grid of the estuary are dashed.

Building the blocks

Start lays one block over the domain, **Draw** one over an area you drag in the 2D top view (a further one when there are blocks), **From corridor** makes the River tab's corridor block 0. **Remove** takes the blocks away with `blocks.json`.

In the 2D top view *Drag points* moves corners (the points of their edges go along, the more the nearer they are) and the points of curved edges; *Cut edge* cuts the edge clicked in two at that point (a new corner); *Add block* puts a block outside the edge clicked, *Depth* deep with *Cells* across, sharing that edge. Esc stops. Each change is an undo step.

Block its name, smoothing and four faces. A face shows its edges, cells and interfaces; a face without an interface is a wall, or open: *Inflow* (a discharge in m^3/s) on the $-i$ face, *Outflow* (a water level in m) on the $+i$ face. *Block 0* makes the block chosen the grid of the run; *Remove block*.

Edge cells (they go on to the opposite face and through the blocks beyond, so the counts always match), curve and grading. *Finer* and *Coarser* double or halve every edge. *Pick edge* (in the 2D view) chooses the edge clicked.

Import blocks.dat a grid made elsewhere (a script, another program) as the case's blocks, to go on with here. Each face is cut where another block's face starts or ends on it; the pieces are edges through their nodes (the nodes kept exactly; more or fewer cells

keep their grading), shared where two faces have the same nodes: the interfaces. The boundaries come from `control.txt` (**C 11** – **C 14** of block 0: an inflow with **W 10**, an outflow with **F 62**, other codes open) and the **G 54** lines of `ctrl.txt`. The inside of each block is made again (2000 sweeps of smoothing). Without `geo.dat`, the nodes' bed is written as `geo.dat`, so *Make grid* gives it back.

Coasts and estuaries

The faces of the blocks of a coast or an estuary lie on land a little beyond the water, smooth, and the shore inside them is wetted and dried. Three tools lay them:

Circular arc (an edge's *Curve*): through the edge's corners and one point, the open sea's side of a block round a mouth. Made an arc, a straight edge bulges out of its block by a quarter of its length; drag the point.

Spacing *By the cells at its ends* spaces an edge's nodes by the length of its first and last cell (*At the start*, *At the end*), growing evenly from one to the other (60 m at a mouth to 1.5 km offshore, say), or, with a *Largest* cell, by *Growth* from each end up to it (fine at both ends of a bay). The cells follow from the edge's length; *And the edges across the blocks* gives the edges whose cells go with it the same spacing, turned their way, and they all take the most cells any of their spacings asks for. After corners were moved, *Cells from spacing* sets the counts again. The tab says how long the end cells are now.

Along the water (with `geo.dat`): *Follow the water* lays the edge on land, *Margin* m from the water below *Water below* (**F 60**), bending no tighter than *Bend radius* (land spits and inlets narrower than twice it are passed over). It runs along the shore between the points nearest its corners, the way round nearer to where the edge is now; *Its corners onto the shore* moves them there (off: they stay, the edge joining the shore from them). Dragging one of its corners lays it again, so a corner slides along the coast; dragging a point of it shapes it by hand from then on (*Stop following* too).

A face of block 0 can also be **open**: its condition is set in `ctrl.txt` by hand; DIVEMesh's side keeps its code (**C 11 2**), no **W 10** or **F 62**. The other blocks' open faces are an inflow or an outflow (**G 54**).

The tide at the open sea

A face of block 0 can take the tide: REEF3D drives it as an open edge (**B 520**, method 3, a Riemann edge) with background 1, the water level the *Tide* box writes:

- *From*: *Tidal constituents* (a line each: name, amplitude in m and Greenwich phase in degrees, as tide tables, FES2014 or TPXO give them: M2, S2, N2, K2, K1, O1, P1, Q1, M4, MS4), with the nodal corrections and astronomical arguments of *Start*; *A tide gauge, observed* or *predicted* (*Find* lists the gauges nearest to the case, as the Sea tab; its mean over the run taken away, since gauges have datums of their own); or *A file of levels* (time and level a line: seconds from the start, or dates and times in UTC).

- *Start* (UTC, $t = 0$ of the run), *Length* and *Every* make the series `background-1.dat` (`t eta 0 0`: no background current, as for a standing tide at a coast; *With the long-wave current*: `t eta`, and REEF3D adds $\eta\sqrt{g/h}$ along the direction). *Ramp*: the tide rises from nothing over that time (**B 510**).
- *As constituents in REEF3D* writes the constituents as **B 511** instead (REEF3D sums them; the nodal corrections of the start held over the run).
- *Travels along the coast* (**B 515**): the tide holds at a point (*Holds at x, y*) and travels along *Direction* at the long-wave speed of *At the speed of* metres of water: a tide running along a coast past the open side.

Write the tide writes `background-1.dat` and **B 510** (**B 511**, **B 515**) in `ctrl.txt`, one undo step. Then give the face the boundary *Tide*; *Make grid* writes DIVEMesh's side (**C 1x 2**) and an open edge for each tide face: **B 520** (method 3), **B 521** (the face's edge of block 0: $-i$ x^-, $+i$ x^+, $-j$ y^-, $+j$ y^+), **B 523** (background 1); zones of other kinds are kept, **B 90 1** (and **B 92 0** without waves) set. REEF3D takes tide edges on block 0 only.

Rivers

Attach river (the *River* box) puts a river block on a part of the chosen edge (one on the outline): along the river drawn in the River tab (*Draw the river*; else the corridor's line), from upstream to where it meets the edge. The edge is cut there, *Cells across* of its cells wide (or as many as make *Width*), and the river block runs upstream from the cut, its banks parallel to the river and bending onto the cut's ends, its cells *Cell length* long (twice those across when not given). Its i runs downstream: the *Discharge* comes in at its $-i$ face (**G 54**). Its banks are edges to drag, space or lay along the water like any other.

A block 1, 2, ... takes its discharge (or its water level) **over time** in the *Over time* box, for the block chosen in the *Block* box:

- *Face*: the discharge in at its $-i$ face (`discharge-<b>.dat`, REEF3D's **G 54 b 3 b**) or the water level at its $+i$ face (`waterlevel-<b>.dat`, **G 54 b 4 b**); `b` is the block's number in the Blocks tab.
- *From*: *GloFAS*, the river's daily discharge (Copernicus, 5 km, from 1984, via Open-Meteo) at the middle of the face, hourly between the days, from *Start* over *Length* (needs the case's georeference). Or *A file*: `time value` a line, the time in seconds from the start of the run or a date and time (UTC, taken from *Start*); a header and # comments are skipped. A gauge record from SNIRH, the USGS or NVE as exported will do.
- *Length*: the series cut to the run (its values at 0 and at the end interpolated); *the file's* keeps the file's times. *Times* scales the values (a tributary's share, a gauge upstream of the face).

Write the series writes the file (# lines saying where it came from) and makes the face's boundary the series; with the grid made (**G 50 1** in `ctrl.txt`) the **G 54** lines follow at once, else at *Make grid*. The face's row then shows the file; a value typed there makes it constant again. REEF3D interpolates linearly in time and holds the end values. The level at an outflow still rises from **F 60** over **F 64**'s steps. Block 0 has no **G 54** face: its river is the Sea tab's *River* (`hydrograph.dat`, **B 60 2**, Section 8.3). **G 54 b 3 n** and **b 4 n** need a REEF3D with the multi-block **G 54** time series.

Pieces for the processes

REEF3D runs each block 1, 2, ... on one process, and shares block 0 out over all of them (DIVEMesh). A large block beside a small block 0 then keeps one process busy while the others wait. *Pieces* writes a block as that many blocks in `blocks.dat`, cut in equal parts along i or j; the pieces meet node for node, the **G 54** lines name the pieces at the faces (an inflow on the first piece, an outflow on the last). The blocks stay one block in LAGOON, to edit. A block with an inflow or outflow is cut along i; pieces need 5 cells at least along the cut.

Balance cuts the blocks 1, 2, ... for the processes of **M 10**. Each block gets about as many pieces as fit a process beside its share of block 0, then one more cut at a time where it helps most, until the busiest process has at most 5 % more cells than the mean (each piece also costs some exchange). Under *Rules* the tab says how the processes share the cells, as REEF3D does it: block 0 with its ring evenly over all, then the pieces, the largest first, each onto the process with the fewest cells. The set-up draws the cuts as lines and names a block's pieces.

So a coast with an estuary is laid out in a few steps: one block over the sea, its corners where the coast meets the open sea and on the banks of the mouth, its open side an arc, its two sides along the water, spaced from fine at the mouth to coarse offshore; then *Add block* at the mouth for the narrows and again for the bay, its far corners on the river, its sides along the water; *Attach river* on the bay's far side; *Balance* for the processes.

Rules and quality

The tab lists the rules the grid breaks, and the set-up marks them where they are (red: errors, *Make grid* waits; orange: warnings):

- the edges of a face must join up, corner to corner, and opposite faces have the same number of cells; no folded or left-handed cells;
- block 0's interfaces cover whole faces (blocks 1, 2, ... may meet several blocks on a face), and the block behind an interface is at least 5 cells deep (8 next to block 0: DIVEMesh **R 21**, a warning below that);
- inflow only on a $-i$ face, outflow only on a $+i$ face, never on an interface, and with a value; block 0 has at most one of each;
- warnings: corners with 3 or 5 blocks around them, poor cells.

The set-up draws the blocks in yellow with their corners and names, the grid of each and the quality of its cells (*good*: angles of 45° or more and a size jump of at most 1.3 to the neighbour; *fair*: 30°, 2; *poor*; *folded*).

Make grid

Make grid writes `blocks.dat` (the number of blocks, then per block `ni nj` and x y z_b of every node, i fastest; the bed from `geo.dat` or flat), **R 20 1** and block 0's box and sides (**B 10**'s x and y, **C 11** – **C 14**) in `control.txt`, and in `ctrl.txt` **G 50 1**, one **G 54 b t v** per open face of the blocks 1, 2, ... and, for an open face of block 0, **B 60 1** with **W 10** or **F 62**: one undo step. Then *Save* and *Run DIVEMesh*.

The blocks are an item of the scene, *Multi-block grid* (*Add* → *Multi-block grid*); *Points*, *Grid*, *Quality* and *Rules* in its properties switch the marks off. In a script:

```
session.start_multiblock([0, 100, 0, 50], cells=(20, 10))   # or:
session.multiblock_from_corridor()
session.extrude_block("b0_e", 60.0, cells=12)     # a block outside edge b0_e
session.split_block_edge("b0_n", 50.0, 50.0)
session.move_block_vertex("b0_ne", 102.0, 55.0)
session.set_block_edge("b0_e", cells=16, curve="through", ratio=1.0)
session.set_block_face(0, "i0", "inflow", 50.0)
session.set_block_face(1, "i1", "outflow", 2.5)
session.set_block_face(0, "i0", "open", 2)        # C 11 2, its condition by hand
session.write_tide("2026-01-19T00:00", days=30,
                   constants="M2 1.00 61\nS2 0.35 89\nN2 0.21 44\nK1 0.07 52",
                   direction=90.0, reference=(43761.1, 42647.5), depth=5800.0)
session.set_block_face(0, "i0", "tide")      # B 520 3, B 521, B 523 at Make grid
session.set_block_edge("b0_w", curve="arc")
session.set_block_spacing("b0_s", 1500.0, 60.0)   # first and last cell (m)
session.set_block_spacing("b1_n", 150.0, 150.0, largest=300.0, growth=1.05)
session.follow_water("b0_n", margin=800.0, radius=3000.0)   # with geo.dat
session.fit_block_cells()
session.import_blocks(smoothing=2000)              # the case's blocks.dat
session.attach_river("b2_n", cells=16, step=100.0, discharge=336.0)
session.write_block_series(5, "2026-01-19T00:00", days=30, glofas=True)
session.write_block_series(2, file="levels.txt", outflow=True)
session.set_block_options(2, pieces=3, split="i")  # written as 3 blocks
session.balance_blocks()                           # for M 10's processes
print(session.multiblock_balance().text())
issues, quality = session.multiblock_check()
session.make_multiblock_grid()
```

7. Maps and the Place of a Case

REEF3D computes in model coordinates. A **georeference** says where they lie on the Earth; with it LAGOON lays a map or a satellite image under the results and the set-up, the domain can be drawn on the map, and a bathymetry survey or a download becomes DIVEMesh's `geo.dat`.

7.1 Starting a Case on the Map

With no case open, the *Map* tab of the Set-up panel starts one at Trondheim on Kartverket's map; *Find* takes it anywhere else. *File* → *New case from map...* asks where: a place name (*Nidelva*, *Trondheim havn*; Section 7.2) or a latitude and longitude (`63.44, 10.40`). LAGOON starts a draft case there: a domain of 1000 × 500 m centred on the place, in its UTM zone, with a background map, in the set-up's 2D view and the Map tab. Then

1. drag the domain where it belongs (*Draw on the map*; `Shift` + drag turns it),
2. add the bathymetry (*Make geo.dat*) and the commands of `control.txt` and `ctrl.txt`,
3. *Create case...* asks for a folder (new or empty) and puts the case there: its input files, `georef.json`, `geo.dat` and what DIVEMesh made. From then on the case is that folder.

Until then the draft lives in LAGOON's drafts folder (`~/Library/Application Support/LAGOON/Drafts` on a Mac; `LAGOON_DRAFTS` moves it), so nothing is lost when LAGOON closes: open it again from there. In a script:

```
case = session.new_case_from_map("Nidelva", length=2000.0)
session.place_case((63.44, 10.40))   # move it
session.create_case("~/runs/nidelva")
```

7.2 Finding a Place

A place is found by its name or by its latitude and longitude:

- **Names** come from Kartverket's place names in Norway (the central place name register: rivers, bays, islets, skerries, farms, towns) and from OpenStreetMap's Nominatim for the whole world. When the name as typed is not there, LAGOON looks for names beginning so (*Munkh* finds Munkholmen) and then for names alike (*Trondhem* finds Trondheim).
- **Name, municipality** narrows it: *Nidelva, Trondheim* (or *Nidelva, Arendal*) when several places have the name.

- The likeliest place comes first: the name as typed before names beginning so, towns and fjords before farms, and the places near the case before those far away (near Trondheim when there is no case yet). The list under the field (or *Which one?* for a new case) offers the others.
- **Coordinates**: `63.44, 10.40`, `63.44N 10.4E`, `lat 63.44 lon 10.40`, `63,44 10,40`; latitude first unless N/S/E/W or *lat*/*lon* say otherwise.

One request goes to each service per search (none while typing), with LAGOON's name, as their terms ask. Offline, type the latitude and longitude.

7.3 The Georeference

A case keeps its georeference in `georef.json` in its folder:

```
{"version": 1, "crs": "EPSG:25833", "origin": [270000.0, 7040000.0],
 "rotation_deg": 30.0, "vertical_datum": "EPSG:5941", "z_offset": 0.0}
```

crs the coordinate reference system of the map, an EPSG code (`EPSG:25833` is ETRS89 / UTM zone 33N) or WKT;

origin where the model's $(0, 0)$ lies in it;

rotation_deg the angle from east to the model's x-axis, counter-clockwise (the panel also shows it as a compass bearing);

vertical_datum, z_offset the datum of the heights (`EPSG:5941`, NN2000; or `LAT`, `MSL`) and the height of the model's $z = 0$ in it.

REEF3D copies the file into its run log and into the LAGOON stores it writes (`lagoon-convert` too), and DIVEMesh into its grid (`grid-geometry.dat`), so the results keep their place when they leave the case folder. LAGOON transforms WGS 84 and ETRS89 longitude and latitude, Web Mercator, UTM and the Norwegian NTM zones itself; for other CRSs install pyproj (`pip install "lagoon-viz[geo]"`).

7.4 The Map Tab

The *Map* tab of the Set-up panel holds:

Find the place a place name or a latitude and longitude, then *Find*: the centre of the domain goes there (in the place's UTM zone when the case has no CRS yet), and the case gets a background map. When several places have the name, the list under the field offers the others, those near the case first.

Georeference the CRS, the origin, the rotation, the vertical datum; *Apply* writes `georef.json`, *Remove* takes it away. The rotation slider turns the georeference about the middle of the view as you drag it (what is in the middle of the screen stays there on the map): the map and north turn with it at once, and it is written when you let go (the arrow keys: 1° at a time, page up and down: 15°). *North up* turns the 3D view so north points up the screen; from the top the view keeps north up while you drag, so the map stays and the model turns.

Domain *Draw on the map* turns the view to 2D from the top; drag the domain's rectangle there (**B 10**, snapped to the cell size **B 1**), drag an edge of the domain to move that side, or a corner to move two: it stays a rectangle. `Shift` + drag turns the model on the map about the domain's centre: the domain turns, the map stays north up (**B 10** keeps its numbers: the model's own axes turn). `Esc` stops. Or type the extent (*Set B 10*) or the rotation (*Turn*), or drag the slider under it: the model turns on the map about the domain's centre as you drag, and is written when you let go.

Background map *Show map* adds a *Background map* item for the case (Section 7.6). The case keeps it (`LAGOON_map.json` in its folder, as last set), so opened again, its results in a new window or from a script, it has its map. A georeferenced case that keeps none opens with the place's best map (Kartverket's in Norway, else OpenStreetMap); one whose map was taken away opens without.

Bathymetry the source of `geo.dat` and how it is made (Section 7.5).

Every step is a command of the session: it can be undone, and a script does the same:

```
session.set_georef("EPSG:25833", origin=[270000.0, 7040000.0], rotation_deg=30.0)
session.set_domain(0.0, 2000.0, 0.0, 800.0)
session.turn_model(35.0)  # about the domain's centre
```

7.5 Bathymetry and geo.dat

Downloaded

From: Download needs no file: LAGOON asks the web for the bathymetry of the domain and its margin. *Best available* takes, the finest first, each filling what the ones before had no data for:

1. Kartverket's national topobathy terrain model in Norway (land and sea in one model, 1 m where surveyed, else 50 m; heights in NN2000);
2. EMODnet's Digital Terrain Model at sea in Europe (about 115 m, heights to LAT);
3. the Copernicus DEM GLO-30 on land anywhere (30 m, heights to EGM2008; its 1° tiles are some 30 MB each, fetched once);
4. NOAA's ETOPO 2022 for the rest of the world, land and sea (15 arc-seconds, about 450 m, EGM2008; at sea built on GEBCO 2022);

5. GEBCO 2020 (15 arc-seconds, public domain) last.

Each can be chosen alone too. The coast of *Best available* lies where Copernicus' 30 m land has it (land above 0), not where a coarse sea source smears it over its 115 or 450 m cells: land takes Copernicus' heights; the sea (low ground joined to the open sea) takes the sea sources' depths, at least 0.1 m deep at the coast and 2 cm deeper per metre out, to 1 m. Low ground behind dikes (polders) keeps Copernicus' heights. Kartverket's model, where it has data, goes over all of them. EMODnet's depths are below the lowest astronomical tide: *LAT below MSL* (from tide tables or the chart's datum notes, about half the spring tidal range) puts them on the land's zero.

Near the coast

Near the coast a coarse sea source is a guess: its first 1.5 cells out (about 700 m for ETOPO and GEBCO, 170 m for EMODnet) mix land and sea, and ETOPO and GEBCO give tidal flats, harbours and creeks a flat 1 or 2 m. *Near the coast* makes the bed there again, between the coastline and the sea's depths further out:

A beach profile (Dean) the default [Dean, 1991]: $h = A\,x^{2/3}$ deep at x metres from the coast (*Dean's A*: about $0.21\,D^{0.48}$ for a grain size D in mm, 0.1 for 0.2 mm sand), bent to meet the sea's depths;

The smoothest bed from 0 m at the coast to the sea's depths, for creeks, harbours and mudflats, whose bed is no beach;

At most 1 m deep keeps the coarse depths, held 0.1 to 1 m deep next to the coast.

Trust the sea from also makes the sea source's shallows again (e.g. 2.5 m, over ETOPO's flat 1 and 2 m).

Files of one's own

Over the download go files of yours, the lower in the list over the higher, blended in over five cells at their edges, and met by the profile near the coast: surveys, chart soundings (*depths*; *Its zero below MSL*: the chart's datum, about half the spring tidal range), an intertidal terrain model from satellite waterlines, DeltaDTM (30 m coastal terrain without buildings and trees). ICESat-2 files (NASA's ATL24, `.h5`) and *ICESat-2 soundings*, downloaded for the domain (a free NASA Earthdata login: `EARTHDATA_TOKEN`, or `~/.netrc` with `machine urs.earthdata.nasa.gov`; reading them needs h5py), give the sea floor along the satellite's tracks to 20 to 40 m in clear water: too sparse to lie over the sea, they guide the profile near the coast.

Or *A file of mine...*: a GeoTIFF, an Esri ASCII grid (`.asc`) or an x y z text file (soundings: spaces, tabs, commas or semicolons; a header line is skipped). A points file with a header names its columns: x, y and z are found by their names (longitude, latitude and the average depth of EMODnet's survey files, `.emo`), or given as *Columns* (their names, or numbers from 1); the dialog shows what it reads and the range of z, so the sign is plain. Soundings many times as many as the grid's cells (a multibeam survey of millions of points) are averaged in each cell first. Give its CRS if the file does not name one (`EPSG:4326` for longitude and latitude, `model` for model coordinates) and tick *depths* when the values are positive down.

Make geo.dat

Make geo.dat asks where to store `geo.dat` (a draft case started on the map goes to that folder, with its input files and `georef.json`; for a case in its own folder the dialog starts there, and a place elsewhere gets a copy, since DIVEMesh reads the one in the case folder). It carries the bathymetry onto a grid over the domain (two cells beyond it) as fine as *Resolution* says:

- as DIVEMesh's cells (**B 1**, the default),
- *the finest the data has* (Kartverket's 1 m in Norway, Copernicus' 30 m on land, EMODnet's 115 m, ETOPO's and GEBCO's 450 m; a file's own cells or the distance between its points; the tab says what that is over the domain),
- or every so many metres,

at most *Most points* (4 million by default, 0: no limit), coarser where the domain would need more (DIVEMesh reads, bins and interpolates every point; `geo.dat` takes some 35 bytes a point). It is written in model coordinates, with **G 10 1** in `control.txt`. A case with blocks (Section 6.2) gets `geo.dat` over its blocks and 5 % of their size beyond them, whatever **B 10** says. A raster is sampled bilinearly; scattered points are interpolated linearly on their triangulation (*TIN*), by inverse distance or from the nearest point; outside the data the nearest height is taken. A progress bar shows how far it is; *Cancel* keeps the `geo.dat` there was. Downloads are kept in the cache. The *geo.dat points* item then shows the points written (Section 5.5). Then *Run DIVEMesh*.

`geo.dat` is in model coordinates, the bed as it lay when it was made. Turning the model, moving it on the map, another CRS or a larger domain leave it as it was, and DIVEMesh would mesh the bed of before. LAGOON keeps how `geo.dat` was made (`LAGOON_geo.dat.json` beside it: its source, spacing, interpolation, and the georeference and domain it was made for), so the Map tab then says it no longer fits, with *Make geo.dat again* (the same source and settings, for the model as it lies now), and *Run DIVEMesh* asks to make it again first. A `geo.dat` from elsewhere is not checked. In a script:

```
session.make_geo_dat("best")      # downloaded; or a file: "~/surveys/nidelva.tif"
session.make_geo_dat("best", spacing="finest")   # as fine as the data is
session.make_geo_dat("best", nearshore="smooth", trust_depth=2.5)   # creeks
session.make_geo_dat(
    "best",
    overlays=[{"path": "~/charts/soundings.xyz", "crs": "EPSG:4326",
               "depths": True, "below_msl": 2.6}, "~/icesat2/"],
    icesat2=True,     # and ICESat-2 downloaded for the domain
)
session.geo_dat_state()    # why geo.dat no longer fits the case, "" if it does
session.remake_geo_dat()   # from the same source, for the model as it lies now
```

7.6 Background Maps

A *Background map* item shows a map round its case's domain, under the results and the set-up, in the 3D view, the 2D view, paper figures, films and the set-up sketch. Each mode keeps its

Table 7.1: The properties of the Background map item.

Property	
Map	the source: Kartverket's topographic map, grey map or nautical chart (Norway), Sentinel-2 cloudless (satellite: 2017 in Europe, 2016 elsewhere), OpenStreetMap, a GeoTIFF of yours (an orthophoto, a chart) or a tile server (an XYZ URL with `{z}`, `{x}`, `{y}`)
File or URL	the GeoTIFF, or the tile server's URL
Margin	how far the map reaches round the domain, in domain sizes
Placed	below the model (a floor under the bed), or at the still water level round the domain; in 2D from the top it lies under everything
Opacity, Resolution	how strongly, and with how many pixels along the longer side

own view: the set-up its 2D map, the results their 3D view, the map under both. Its properties are listed in Table 7.1.

The 2D view from the top shows north up: the map lies as on a map, and a turned model is drawn turned. Turning the model turns the domain and its grid, never the map, in 2D and in 3D. *North up* in the bar of the view (or `N`) turns any view, 2D or 3D, back so that north points up the screen (in 3D: away from you, the tilt kept). North is true north at the middle of the domain: a CRS's grid north is true north only on its central meridian, so a model lined up with the CRS's grid lies a little turned in the north-up view, as on the map. Finding a place far from the case's CRS gives the case the place's UTM zone (WGS 84 UTM outside Europe, ETRS89 UTM in it); the Map tab says so.

In the 2D view from the top the map follows the view: zoom in and it is fetched again in finer detail, zoom out or pan and it covers the new area (once the view rests; the map before stays until the new one is there). In 3D it covers the domain and its margin. While a map is fetched, a bar above the view counts its tiles. Map tiles are fetched once and kept in a cache (`~/Library/Caches/LAGOON/tiles` on a Mac; `LAGOON_CACHE` moves it). LAGOON asks the servers with its own name and only for the tiles over the area shown, as their terms ask. In a script: `session.add_background_map("Sentinel-2 (satellite)", margin=1.0)`.

Every map shows the credit its source asks for: under the 3D view, in the corner of films and in the corner of figures. Kartverket's maps are CC BY 4.0 (Kartverket), the Sentinel-2 cloudless 2016/2017 mosaic is CC BY 4.0 (EOX, modified Copernicus Sentinel data), OpenStreetMap is ODbL (OpenStreetMap contributors). Images whose terms do not allow such use are not offered; a GeoTIFF of your own is drawn with no credit.

8. The Sea at the Case

The *Sea* tab of the Set-up panel reads the sea at a georeferenced case, for its waves and its boundaries, and the water measured nearby, to compare a run with. It reads at the middle of the wave generation side (**C 1x** $= 6$), else at the domain's centre; the tab says where and from which source.

8.1 Sources

Source chooses where the sea comes from (*Automatic*: MET Norway in its waters, Open-Meteo elsewhere):

MET Norway (thredds.met.no; Norwegian waters): **NORA3** (waves and wind, 3 km, hourly, from 1959, a month or two behind, with 2D spectra) and **NorKyst** (water level and current, 800 m, hourly, from 2017-02-20: v2 to 2023, v3 from 2024).

Open-Meteo (open-meteo.com; anywhere): **ERA5** waves and wind (ECMWF's reanalysis, 0.5°, hourly, from 1940, a few days behind; no 2D spectra) and **Météo-France's** sea level with the tide and the surge and its surface currents (SMOC, 8 km, hourly, from 2022). Free for non-commercial use; for commercial use give the key of an Open-Meteo plan in *API key*.

Open-Meteo, CERRA wind as Open-Meteo, with the wind from **CERRA** (the Copernicus European Regional Reanalysis, 5.5 km, hourly, Europe, 1985 to mid 2021 on Open-Meteo) instead of ERA5's 0.5°: closer to the coast and in fjords and estuaries. CERRA has no waves (they stay ERA5's); where or when it has no wind, ERA5's stands in, and the table says for how many hours.

8.2 Storms, Waves and Boundaries

Find storms the largest wave events at the case in the years chosen (H_s, at least three days apart), with T_p, where they come from and the wind. Choosing one sets the time window (a day before its peak to 12 hours after) and the hour of the spectrum (its peak).

Show the conditions H_s, T_p, direction and wind and the water level and current over the window, as time series items. They are tables in the case's `LAGOON_Sea` folder, time 0 at the window's start (UTC, written in the header); `beta` is where the waves go in the model (from its x-axis, as **B 105**), `u`, `v` the current in the model's frame, `level` the still water level (**F 60**) plus ζ. NorKyst's current is depth-averaged, Open-Meteo's at the surface.

Write spectrum-file-2d.dat (MET Norway): NORA3's 2D wave spectrum (30 frequencies, 24 directions) at the hour chosen as REEF3D's `spectrum-file-2d.dat` in the case folder: the directions within $\pm 90°$ of the mean, turned into the model's frame, the energy per rad/s and rad. With *Set ctrl.txt to them*, the wave generation follows it: **B 85 11**, **B 92 31**, **B 93** H_s T_p, **B 105** the mean direction, **B 130 1**.

Set the waves (JONSWAP) (Open-Meteo): ERA5's H_s, T_p and mean direction at the hour; with *Set ctrl.txt to them* a JONSWAP spectrum: **B 85 2**, **B 92 31**, **B 93** H_s T_p, **B 105** the direction (**B 88** and **B 130** stay as they are). Where Open-Meteo serves no peak period, ERA5's mean period stands for T_p.

Write boundary hydrographs the water level and discharge along the sides of the domain over the window: per side `hydrograph_FSF_<n>.dat` (time, water level) and `hydrograph_Q_<n>.dat` (time, discharge into the domain), $n = 1$ x min, 2 y max, 3 y min, 4 x max; the sides more than half in the water, or those chosen. The discharge is the current across the side times the water depth, the level the still water level plus the mean ζ along it. NorKyst gives its own depth; with Open-Meteo the depth comes from the case's bed, and its sea level and current, asked about every 4 km along the sides, are interpolated between. A *Boundaries* time series shows them all. With *Set ctrl.txt to them*: **B 440 n n** (the side as a patch), **B 421 n 1**, **B 422 n 1**.

8.3 A River

River (GloFAS): the daily discharge of the river at the case from **GloFAS** (the Copernicus flood service, 5 km, from 1984; through Open-Meteo, anywhere), hourly between the days over the window, time 0 at its start. It is read at the upstream end of the case's river corridor, else at the middle of the inflow side (**C 1x** $= 1$), else at the domain's centre; the tab says where, and which GloFAS cell answered and how far away it is. It is written as `hydrograph.dat` for ioFlow (**B 60 2**), or as `hydrograph_Q_<n>.dat` for a side as a patch (**B 440**, **B 421**). A *GloFAS river* time series shows it. GloFAS is a model on a 5 km river network: small rivers may not be in it, and a point near a confluence may take the neighbour.

8.4 Tide Gauges

Find gauges lists the tide gauges nearest to the case (300 km at most) of **Kartverket** (Norway: observations and the tide predicted, relative to mean sea level, every 10 minutes), **NOAA CO-OPS** (USA: observations every 6 minutes and predictions, relative to MSL) and the **IOC** Sea Level Station Monitoring Facility (the world: raw observations of many agencies, without a datum, given about their mean over the window, not quality-controlled). *Show the measured water level*: the observed, the predicted and the surge (observed minus predicted) over the window, and `level` the still water level plus the observed, as a time series to compare the run's gauges with. *Use it as the water level at the sides*: the observed level (gaps filled linearly) as `hydrograph_FSF_<n>.dat` at the sides chosen above (**B 440**, **B 422**).

Changes to `ctrl.txt` are one undo step each and are saved with the input files. In a script:

```
session.find_storms(2015, 2024, source="Open-Meteo")
session.sea_conditions("2020-01-05T00:00Z", "2020-01-06T00:00Z")
session.write_spectrum("2020-01-05T04:00Z", apply=True)
session.write_waves("2023-03-10T06:00Z")
session.write_boundaries(start, end, sides=[1, 4])
session.river_discharge(start, end, target="B 60")
session.find_gauges()
session.gauge_levels("NOAA:8518750", start, end)
session.gauge_boundaries("Kartverket:TRD", start, end, [4])
```

Name the data when you publish results made with them: MET Norway (NORA3, NorKyst; NLOD 2.0 / CC BY 4.0); Open-Meteo.com (CC BY 4.0) with Copernicus/ECMWF (ERA5, CERRA, GloFAS) and Météo-France / Copernicus Marine (sea level, currents); Kartverket (NLOD 2.0), NOAA CO-OPS, the IOC Sea Level Station Monitoring Facility.

9. Scene Items

Everything shown is an item in the Scene panel: the cases, the surfaces and volumes drawn from them, the time series and free surface lines plotted from them, the set-up, grids and maps of a case. The *Add* menu (and the tool bar, for the common ones) adds items; Del removes the selected one (a case with all the items that show it), and *Edit* → *Undo* (Ctrl+Z) takes back any change. The check box beside an item shows or hides it; a clicked item brings its properties to the front.

Select an item to change it in the Properties panel. Each change runs as a command: it can be undone, it shows in the Python console, and it is in the script the session writes (Chapter 18). Section 19.3 lists all properties of all items.

9.1 Kinds of Items

Table 9.1: The kinds of scene items.

Item	Shows
Case	a REEF3D run; its *Vertical scale* applies to all its items
Case set-up	the input files' domain, sides, relaxation zones, gauges, probes, bodies (Section 5.4)
DIVEMesh grid	the grid lines on the faces of the domain, the MPI partition (Section 5.11)
River corridor	the banks of a river corridor and its curvilinear grid (Section 6.1)
Multi-block grid	the blocks of a multi-block grid, their grids and quality (Section 6.2)
geo.dat points	the points of `geo.dat`, coloured by height, with their number and spacing (Section 5.5)
Background map	a map or satellite image round a georeferenced case, under the set-up and the results (Section 7.6)
Free surface	the water surface, coloured by a variable or plain
Free Surface AMR	the free surface of an adaptive-mesh run, finer patches over coarser ones, with the patch outlines; *AMR patches* off shows the base grid (level 0) alone, *Grid lines* draws the cells of every level
Bed	the bathymetry (sand coloured by default)
Statistics	a map of an output over a time window: maximum, minimum, mean, time of the maximum, H_s, arrival and duration of the water, maximum inundation (Section 9.3)
Volume	the outside of the volume output
Slice	the volume cut by an x, y or z plane
Iso-surface	where a variable of the volume has a given value; on the free surface or the bed, the line where it has it

Item	Shows
Iso-volume	the part of the volume where a variable lies in a range (the water of a CFD run: ϕ from 0 up); on the free surface or the bed, the part of the surface in the range
Streamlines	lines along a vector field, through the volume, in a plane or on the free surface
Sea state	SEASTATE's map of the sea state: H_s, periods, direction, with arrows where the waves go (Section 4.1)
Time series	wave gauges, probes, 6DOF motions or forces, plotted under the 3D view
Free surface line	the surface along the lines of `WSFLINE`, with the wave theory (FNPF)
Gauge	a gauge position to show in the 3D view and the plots
Bodies	floating bodies (6DOF)
AMR blocks	the outlines of CFD's refined blocks, coloured by their level
Mooring lines	the mooring lines of floating bodies, coloured by tension
Structures	fixed structures: porous and direct forcing structures, the solids of DIVEMesh
DEM elements, Ice floes, Rod trees, Particles	the objects of those modules, with trails for DEM elements and particles

9.2 Colours

Surfaces, slices and objects share these properties:

Colour by a variable of the output (a vector's magnitude, or one of its components, e.g. `velocity.x`), or `none` for a plain colour.

Colour map one of the maps of Table 9.2.

Automatic range is fitted to every frame shown, as *Fit range to this frame* would be: playing, the colours follow the values of each output, and *Range min* and *Range max* show the frame's range. For the same colours at every time (a film to compare outputs), switch it off: *Range min* and *Range max* then hold. *Fit range to this frame* (`Ctrl+F`) sets them to the current output.

Range from the 1st to the 99th percentile, symmetric around zero when the values change sign (the default: strong colours, still water white in the diverging maps, a single spike does not wash them out), or the lowest to the highest value, as ParaView (every extreme in range, e.g. the deepest scour). *Range min* and *Range max* show six significant digits (`1.15e-06`, `0.000156909`, `2.448`), so the tiny values of k, ε and the eddy viscosity can be read and typed; their arrows step by a tenth of the range's order of magnitude.

Range [%] scales the automatic range: 100 % runs from the lowest to the highest value, 95 % from $0.95\times$ min to $0.95\times$ max (values beyond take the colours at the ends), 50 % brings out the small waves. *Fit range to this frame* uses it too.

Opacity a transparent free surface shows the bed under it.

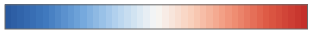
Colour bar whether the item has one, under the 3D view, on films and in figures. *Bar title* (empty: name · variable; `$\eta$ [m]` for symbols), *Bar numbers* (auto, integer, 1 to 3 decimals, scientific) and *Bar ticks* (0: the ends under the view and on films, round numbers in figures; 1: the ends and the middle; 2 to 11: that many, evenly spaced) are the bar's in all of them.

Grid lines off, dark or white: the edges of the cells of the free surface, the bed, the volume, a slice or an iso-surface, drawn just off the surface (Section 9.6).

Contour lines of a variable, and how many contour lines.

Vectors of a vector field and the arrows' colour: `velocity (white)`, `velocity (black)` or `velocity (grey)`; then how many arrows, and their length. They are spread over the area the surface covers, so a river in a wide domain gets them all on its water; the longest reach about the next arrow. The bed has the same options: NHFLOW's and SFLOW's velocity at the bed.

Table 9.2: The colour maps.

Name		Use
`reef3d`		REEF3D's look: troughs in REEF3D blue, still water white, crests in warm red (the free surface)
`ocean`		sequential for magnitudes (speed, depth): white to deep REEF3D blue
`tension`		slack to taut in the REEF3D colours, without the white middle (mooring lines)
`lagoon_blue`		REEF3D blues: deep water to foam
`balance`		diverging, after cmocean's balance: troughs blue, crests red, zero light
`viridis`		perceptually uniform, from dark blue over green to yellow
`rainbow`		Google's Turbo [Mikhailov, 2019], as in SEASTATE's maps, without its darkest ends and with yellow and orange widened; smooth lightness without false bands
`sand`		sediment: deep bed dark, shallow bed light sand (the bed)
`greys`		dark to mid-light grey: never paper white
`white`		mid grey to white; plain, pure white shaded only by the light

Older names lead to these maps: `turbo`, `jet` and `cividis` give `rainbow`.

Bodies, structures and the other objects have a *Material* (steel, concrete, rock, ice, wood): its colour, and how the cinematic looks draw it (Chapter 16). An item coloured by nothing takes the colour of its kind or of its material; with a colour map other than its kind's own, it takes the middle colour of that map (a bed with the `white` map is white).

9.3 Statistics over Time

Add → *Statistics* makes a map of a surface output over a time window (the free surface, else the bed; the Add menu's list chooses another). It reads every output of the window once (a progress dialog, which can cancel) and keeps the map in the case's `LAGOON_Statistics` folder, so it opens again at once, in this session or the next. The map is the same at any time; it lies at the mean height of the surface over the window. Per point it holds the fields of Table 9.3.

Table 9.3: The fields of a statistics map.

Field	What
`<v>_max`, `<v>_min`, `<v>_mean`	of each variable (a vector: its magnitude), e.g. `velocity_max`, `eta_max`
`<v>_time_of_max`	when the maximum came [s]
`<v>_change`	the last output minus the first: a bed's `elevation_change` is its erosion (< 0) and deposition (> 0)
`Hs`	the significant wave height $H_{m0} = 4\,\sigma(\eta)$ over the window: wave penetration, harbour agitation (FNPF, NHFLOW, SFLOW free surfaces)
`eta_range`, `setup`	the highest crest to the lowest trough, the mean water level
`first_wet`	when the water first arrived [s] (empty where it never did)
`wet_duration`	how long it stayed [s]
`ever_wet`	1 where the water ever came: the maximum inundation (*Hide never wet* leaves the rest out)
`max_depth`, `max_level`	the deepest water, the highest water level: run-up

Wet means a water depth above *Wet from* (0.01 m; the depth is NHFLOW's `WL`, SFLOW's `waterlevel` or FNPF's `depth` + `eta`), else `wetdry`. Waves and velocities count only while wet. Each output stands for the time half-way to its neighbours. *From* and *To* set the window (leave out the start-up; *To* 0: the last output), *Variables* which variables; the defaults are `eta`, `velocity` and the water depth, for a bed its elevation. Changed, the map is made again with *Compute*. The outputs must keep their points (not AMR surfaces, whose patches move). In a script:

```
session.add_statistics(source="nhflow_free_surface", start=60.0)
```

9.4 Vertical Scale

Waves are small next to the length of a domain. The case's *Vertical scale* stretches the heights of all its items about the still water level: surface, bed, bodies, slices and gauges alike. With *Same for all items* off on the case, each item gets its own scale. The free surface can leave out dry land (*Hide dry land*, on by default), so only the wet free surface shows: FNPF's surface where it has no water depth, NHFLOW's and SFLOW's over their dry cells (`wetdry`), the Free Surface AMR likewise. The bed can leave out what is under still water (*Hide wet land*).

9.5 Reading Values: the Probe

Probe in the bar of the 3D view (P) lists every value at a point, like ParaView's point inspection: click on a surface and a small floating window shows the item, the point (x, y, z), the time and output, and all the arrays of its dataset there, not only the one it is coloured by (vectors as magnitude and components). A white cross marks the point; the values follow the time. *Copy* puts them on the clipboard as tab-separated lines. The window floats over LAGOON, so the 3D view keeps its size and perspective; move and resize it as you like (it opens at the view's top right and stays where you put it). Closing it or Esc ends the probe. Drags still turn the view, and without the probe a click does nothing. For values over time, use REEF3D's gauges and probes (Chapter 12). In a script, `session.data.probe(...)` gives a `Probe` with `rows()` and `text()`.

9.6 Slices, Iso-Surfaces, Iso-Volumes and Streamlines

Add → *Slice* cuts the volume in the middle; set the *Normal* (x, y or z) and the *Position* in metres (Fig. 9.1). *Grid lines* (dark or white) draws the edges of the cells it cuts, so the slice shows the grid the solver computed on (on a σ-grid the lines follow the bed and the free surface).

Add → *Iso-surface* draws where a variable has a given *Value*; its *Grid lines* show where it crosses the cells' faces (for CFD's free surface with AMR, of every level). *Add* → *Iso-volume* (and its button in the tool bar) draws the part of the volume where a variable lies between *From* and *To*, as a closed body: for a CFD run it starts with the level set ϕ from 0 up, the water itself; else the upper half of the velocity. The free surface, the bed and the volume have *Grid lines* too: the edges of their cells (the triangles REEF3D writes, two per cell, without the diagonals).

On a surface, an iso-surface is the line where the variable has the *Value* ($\eta = 0$: the still water line on the free surface; a depth contour on the bed), and an iso-volume the part of the surface where the variable lies between *From* and *To* (the shallows of the bed, the crests of the waves). In 2D they lie just over their surface.

Add → *Streamlines* starts lines on a seed plane (*Seeds on plane*, *Plane at*) and follows a vector field through the volume, only in the seed plane, or on the free surface; *Lines* sets about how many. New streamlines start where you are looking: in the plane of a selected slice, or on a selected free surface.

Which data

With several cases open, a new slice, iso-surface, iso-volume or streamlines cut the case of the item selected in the scene (else the first case that has a volume); *Case* in its properties puts it on another case, through the middle of that case's volume if its plane would miss it. They are made of the data of the item selected: with a volume, a slice or an iso item selected, of that volume (a case may have several); with the free surface or the bed selected, an iso-surface or iso-volume of that surface. *Dataset* in the properties chooses another (slices and streamlines: the volumes; iso items: the volumes and the surfaces).

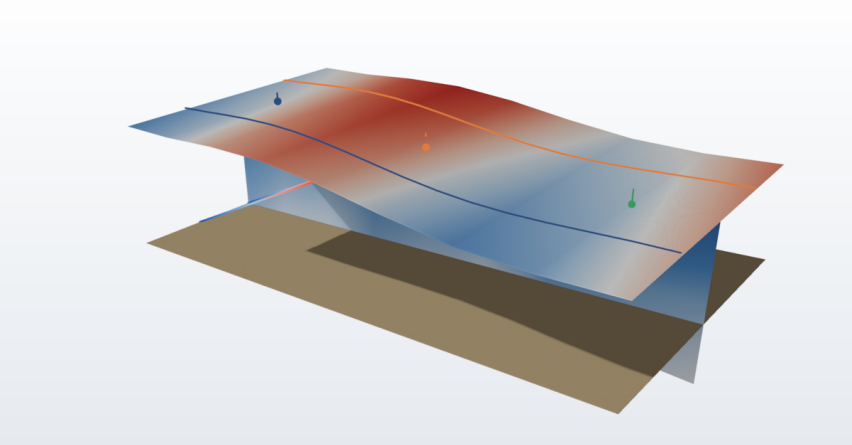


Figure 9.1: A slice through the volume of the example flume, coloured by the velocity.

Or choose the data as you add the item: *Add* → *Iso-surface* (and Iso-volume, Free surface, Bed, Slice, Streamlines, Statistics) lists *Automatic* and every output of the open cases the item can show: an iso-line on SFLOW's bed, an iso-volume of the NHFLOW volume, the SFLOW free surface beside the NHFLOW one. In the tool bar, the arrow beside the button opens the same list. In a script:

```
session.add_of("iso_surface", "sflow_bed")      # case= with several cases open
```

10. The 3D View

10.1 Moving Around

The view turns about the middle of what it shows, all the way round. Table 10.1 lists the mouse and trackpad controls; the Blender-style controls on the middle button work as well.

Table 10.1: Mouse controls of the 3D view.

	Mouse or trackpad	**Blender style**
Turn	left drag	middle drag
Move	right drag	Shift + middle drag
Zoom	wheel or pinch	Ctrl + middle drag

The bar of the 3D view holds *Frame all*, the views *3D*, *Top*, *Front* and *Side*, *2D*, *Ortho*, *North up*, *Probe* and, with several cases open, the case the view shows. The keys of Table 10.2 work with the mouse over the view.

Table 10.2: Keys of the 3D view.

Key	**View**
1, 3, 7	front, right side, top (with Ctrl: back, left, bottom)
5	perspective or orthographic
2	the 2D view (Section 10.2)
N	north up
P	probe: a click lists every value at a point (Esc ends it)
Home or .	frame everything
arrow keys	move the view (with Shift: further)
Space	play or pause

10.2 The 2D View

2 switches to a flat, orthographic view without rotation, in which any drag moves the view and the wheel zooms. In it, 1, 3 and 7 choose the side it looks from. From the top, every item is drawn flat, the free surface as a plane coloured by its values over the bed, but lit as in the 3D view, in the same look and light, so the waves and the bed's relief stay shaded (Fig. 10.1). From the front or a side, the 2D view is an elevation of the shapes as they are, with the vertical scale. 2 again goes back to 3D; the buttons in the bar of the view do the same.

North up (N) turns the view so that north points up the screen, in 2D and in 3D (the tilt kept): north of the case's georeference (Chapter 7), else the model's y. A background map under the case follows the 2D view, fetched again in finer detail when you zoom in and for the new area when you pan; a bar above the view counts its tiles while they come.

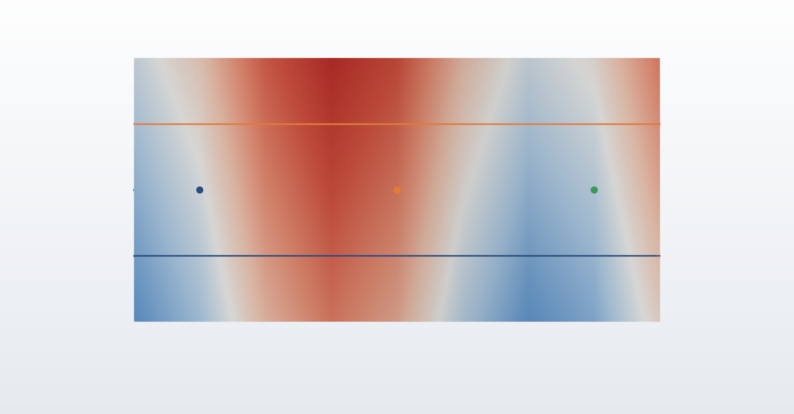


Figure 10.1: The 2D view from the top, with the wave gauges and the free surface lines.

10.3 Looks

View → *Viewport look* sets how the 3D view is drawn:

Studio (light grey, the default) and **Ocean** (dark blue, Fig. 10.2): plain, even lighting for reading values.

Open ocean, Wave flume, Underwater, Diorama, Diorama classic the cinematic looks, with physically based materials, reflections and, under water, fog. Open ocean, Wave flume and Diorama show what Blender renders: the sky picture of ÆgirFrames [NTNU, 2026] around the model, lighting it and reflected by the water, with its sun where the look's sun is, and structures in their default colours in ÆgirFrames's red; the bed stays LAGOON's light sand (path traced: dark rock). Diorama classic keeps the earlier Diorama look: a blue backdrop and a block of sand. Chapter 16 describes the cinematic looks.

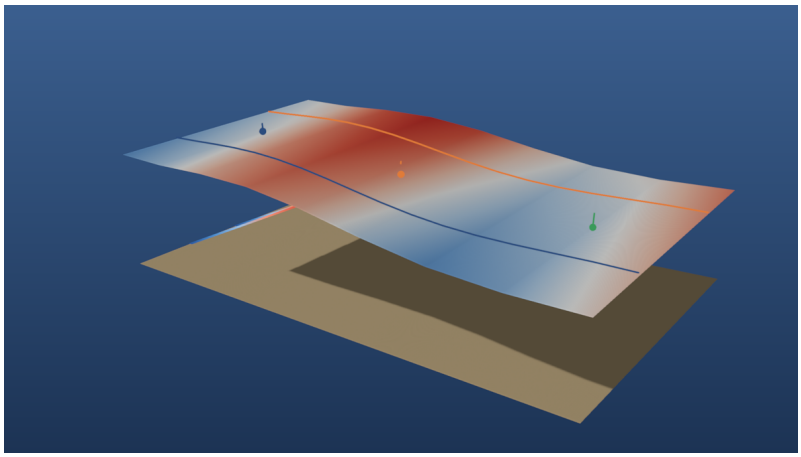


Figure 10.2: The Ocean look.

View → *Theme* switches the window between a light and a dark theme.

10.4 The Sun

View → *Sun...* moves the light: *Direction* turns it around the vertical (0° from $+x$, 90° from $+y$) and *Height* lifts it from just over the horizon (long shadows) to straight overhead. The 3D view follows while the sliders move. The sun is saved with the session and used by films and by path tracing too; *Look's own* gives the sun back to the look. In a script:

```
session.set_sun(azimuth=200, elevation=20)  # degrees
session.set_sun()                           # the look's own sun again
```

10.5 Compare Side by Side

View → *Compare side by side* (Ctrl+Shift+2) splits the view in two, one case in each (with two cases open). The cameras and the time move together, so the same wave can be compared in two runs.

10.6 Full Screen

View → *Full screen viewer* (Ctrl+Shift+F) shows only the 3D view and the player on the whole screen, for presentations; Esc goes back.

10.7 Images and Screenshots

File → *Export image...* (Ctrl+Shift+I) draws the 3D view alone again, off screen, as large as you choose: as on the screen, twice as sharp, Full HD, 4K or 8K wide, a width in pixels of your own, or a width in centimetres at a print resolution (17 cm at 300 dpi: a journal's page width; the resolution is kept in the file). The image frames what the view shows, and lines, markers and text keep their share of it: a 4K image is the view drawn sharper, not smaller. The time and the colour bars go where the legend layout puts them (*View* → *Legend...*, Section 15.1). PNG, JPEG or TIFF, by the file's end; the GPU draws up to 8192 pixels a side. In a script:

```
from lagoonviz.render.offscreen import render_image
render_image(session, "view.tif", (2008, 1130), scale=2.5, legend=True, dpi=300)
```

File → *Save screenshot...* (F12) saves a quick picture of the whole window with the 3D view in it. For a paper figure with axes, see *File* → *Export figure...* → *3D view* (Chapter 14).

10.8 Without a GPU

Without a usable GPU, or over a remote desktop without one, the window opens with a note in place of the 3D view; plots, figures and scripts still work, and `lagoon-render` can draw with Mesa's software driver on Linux.

Table 10.3: The modes of *Memory use*.

Mode	Frames kept	Read ahead	
Performance (in memory)	a quarter of the computer's memory (2 – 16 GB)	the whole play range, 6 threads	runs from memory from the second time round; a volume's cells kept for slices and iso-surfaces
Balanced (the default)	1 GB	8 frames, 4 threads	
Low memory	256 MB	2 frames, 2 threads	the cells read again for every slice: large volumes fit, playing is slower
Streaming (very large cases)	768 MB	6 frames, 4 threads	surfaces of more than 2.5 million points shown coarser while playing or dragging the timeline, at full detail as soon as the time stands still

10.9 Speed or Memory

View → *Memory use* chooses how much LAGOON keeps in memory to be fast (Table 10.3).

Frames played once come back from memory without reading the files again, as long as they fit; films read ahead in the same measure. Whatever the mode, LAGOON never reads further ahead than the frames fit. The choice is kept for the next start and applies at once. For `lagoon-render` and scripts, `LAGOON_MEMORY=low` (or `performance`, `streaming`) does the same.

Very large cases

Cases of 100 million cells and more (a free surface of 10 million points and more) open in a moment: LAGOON reads a frame when it is shown, not before.

- *Streaming* is for playing them. While the timeline plays or is dragged, a free surface from a LAGOON store is read at every 2nd, 3rd, ... grid line, to about 2.5 million points; when it stops, the frame comes in at full detail. The bed and other surfaces that do not change are always at full detail. Films and stills are always made at full detail, whatever the mode. Opening a case of more than 5 million points a frame in another mode, the status bar says so.
- Items that are switched off (the volume a case opens with, for one) are not read at all, neither while playing nor for films.
- Fields that are the same in every output (FNPF's still water depth, its coastline) are read once.
- The automatic colour range of a large surface comes from tiles spread over it, not from the whole surface.

Help → *Record a performance report* counts what every thread of LAGOON does while you use it; stop it to save the report, for example to send it with a question about a slow case.

11. Timeline and Camera Paths

11.1 Playing

The Timeline panel steps through the outputs of the open cases:

- the buttons go to the start of the play range, the previous and the next output, and the end of the play range, and play or pause (`Space` in the 3D view);
- `Cmd+Left` and `Cmd+Right` on a Mac (`Ctrl+Left` and `Ctrl+Right` on Windows and Linux) go one output back or forth, wherever the keyboard is (*View* → *Previous frame*, *Next frame*); `Cmd+F` (`Ctrl+F`) fits the colour range of the selected item to the frame shown (*View* → *Fit range to this frame*);
- the slider jumps to where it is clicked and follows the mouse while held;
- the frame number can be typed into its box;
- the number box sets the playback speed in frames per second (up to 240).

The label shows the time in seconds when it is known, and the output number. Playing waits for each output to be read and reads the next ones ahead, so it never skips one; a long run plays as fast as the disk allows.

11.2 The Play Range

The play range is marked on the slider by two thick stripes beside the handle, at its ends to begin with. Drag them to play only a part of the run: playing starts at the left stripe and goes up to the right one, and the slider outside is dimmed. With the *repeat* button on (the default) the range plays again and again; off, playing stops at its end. The range is saved with the session; *Export animation* starts from it, and so do films made by `lagoon-render` or a script:

```
session.set_play_range(1.0, 2.5)   # seconds
session.set_play_range()           # all of it
```

11.3 Camera Keyframes

A film can move the camera. A camera keyframe keeps the 3D view as it is at the current time:

1. Go to a time, set up the view, and press the key button on the timeline (or *View* → *Camera path* → *Add camera keyframe*, `Ctrl+K`).
2. Go to a later time, change the view, add another keyframe. And so on.

The slider marks the keyframes with small diamonds. With the camera path on (the button beside the key button, or *View* → *Camera path* → *Follow camera path*), the 3D view and every film made from it follow a smooth path through the keyframes: the camera eases in at the first, passes the others without a jolt and comes to rest at the last. Before the first keyframe and after the last, the camera stays put.

Remove camera keyframe takes away the keyframe at the current time, *Clear camera keyframes* all of them. Keyframes are saved with the session and written into its script.

11.4 Orbit

View → *Camera path* → *Orbit around the scene* makes the keyframes of one turn around the middle of what is shown, from the first output to the last, starting from the view as it is: a quick showcase film. In a script, `session.orbit_camera(degrees=90)` turns by any angle.

12. Plots and Free Surface Lines

12.1 Time Series

Each *Time series* item is a row in the Plots panel under the 3D view, all rows on one time axis: wave gauges, probes, 6DOF motions and velocities, forces, run-up and bed gauges. A case opened with such files gets them as items by itself, switched off: tick them in the scene to show their rows and their gauges in 3D. *Add* → *Time series* adds more. They are named as the REEF3D User Guide names the print-outs (Table 12.1).

Table 12.1: The time series LAGOON reads.

Name	Print-out	File
Wave gauges, Wave gauges (theory)	P 51, P 50	`REEF3D_*_WSF/...-WSF-HG(-THEORY).dat`
Velocity probe n, ... (theory)	P 65, P 66	`REEF3D_*_ProbePoint/...-Vel-Probe(-Theory)-n.dat`
Point probe n	P 61	`REEF3D_CFD_ProbePoint/...-Probe-Point-n.dat`
Depth-averaged point probe n	P 63	`REEF3D_SFLOW_ProbePoint/...`
Pressure probe n	P 64	`REEF3D_CFD_PressureProbe`, `REEF3D_NHFLOW_ProbePoint/...-Probe_Press-n.dat`
Force box n	P 81	`REEF3D_*_Force/..._Force-n.dat`
Morison force n	P 85	`REEF3D_*_Force_ALE/..._ALE_Force-n.dat`
Run-up gauges, Maximum run-up gauges	P 133, P 134	`REEF3D_NHFLOW_RUNUP`
Cylinder run-up n	P 140	`REEF3D_FNPF_Runup`
Bed level gauges, Bed shear stress gauges, Maximum ...	P 121, P 125, P 122, P 126	`REEF3D_*_Sediment`
Theoretical wave time series n	P 58	`REEF3D_Log-Wave` (not shown by itself: REEF3D always writes one)
Body n forces, Body n motion, Mooring line n	6DOF	`REEF3D_*_6DOF`

Tables headed `t` (the probes), headed without the time (the run-up gauges) or not headed at all (CFD's pressure probes) are read too: the time is their first column. In the item's properties:

File which file of the case (the gauges, a body's forces or motion, ...).

Columns which of its columns, e.g. `P1, P4` or `Fx, Fz` (empty: all).

Gauges in 3D the gauges as staffs in the 3D view, with a dot at the water surface that follows the time.

Line names e.g. `P1 = gauge 1; P4 = gauge 4`; also in the figures made from the plots.

Legend above the plot, in the plot (its upper right corner) or off, and *Values at the cursor*.

The orange line marks the current time. In a plot, the mouse works as in Table 12.2. The legend shows each line's value at the current time.

Table 12.2: Mouse controls of the plots.

click or drag	move the time (and the 3D view) there
wheel	zoom the time axis
right or middle drag	move along the time axis
double-click	show everything

12.2 Measured Data

A time series item takes a *Measured file* (... to choose it; a path relative to the case folder is kept relative): a text file with the time in the first column and one column per gauge, separated by spaces, tabs, commas or semicolons; lines starting with # or % and a header line of names are allowed. Its columns are drawn dashed in the colour of the line they go with: the column of the same name (`P1`), else in order (the first value column for the first line plotted). *Measured columns* says otherwise (`P1 = 2; P4 = G4`), *Measured time shift* moves the measured times (to the moment the wave maker started, say).

In the lower left corner of the row, each line is compared with its measurement over the time shown (zoom in to compare a part): the RMSE, the bias (computed minus measured), the correlation r, Willmott's index of agreement d (1: perfect) and the lag (the shift of the computed line that matches best; positive: it comes later). Figures made from the plots draw the measured lines dashed too.

12.3 Free Surface Lines

Free surface line items show the surface elevation along REEF3D's free surface lines at the current time: lines along x (**P 52**, `WSFLINE`), along y (**P 56**, `WSFLINE_Y`, *Free surface lines in y*) and NHFLOW's time-averaged lines (**P 146**, **P 147**: one item per line, at the end of its averaging window). A case opens with them switched off; ticked, they are drawn in the *Free surface lines* panel (a tab beside the plots), and as lines in the 3D view (*Lines in 3D*). *Lines* picks some of the lines (e.g. `1, 3`); with *Wave theory* on, the theory FNPF wrote is drawn dashed. The value axis stays fixed while playing, so the waves move through a steady frame.

12.4 Wave Analysis

View → *Wave analysis...* works over a time window (the Plots panel's when it opens; *From*, *to*) with Welch's method [Welch, 1967] (*segments*, half overlapping):

Spectra the lines ticked (measured ones dashed) and a table of their wave parameters: $H_{m0} = 4\sqrt{m_0}$, T_p, $T_{m01} = m_0/m_1$, $T_{m02} = \sqrt{m_0/m_2}$ from the spectrum; $H_{1/3}$, H_{max}, $T_{1/3}$ and the number of waves from zero down-crossings.

Reflection two or three gauges along the wave direction (their x from the gauge file) and the water depth (the case's still water level to start with): the incident and reflected spectra fitted per frequency (Mansard and Funke, 1980; with two gauges Goda and Suzuki, 1976), K_r and the incident and reflected H_{m0}. Frequencies at which a gauge spacing is near a multiple of half a wave length cannot be separated: they are shaded and left out of K_r.

Response (RAO) a wave gauge in, a motion out (a 6DOF column): the gain $|S_{xy}|/S_{xx}$, the phase (positive: the motion comes later) and the coherence; the gain is drawn strong where the coherence reaches the value asked for, and the largest RAO is given with its period.

Copy table puts the numbers on the clipboard (tab separated, for a spreadsheet), *Save figure* writes the tab's figure as PDF, SVG or PNG. In a script: `lagoonviz.core.waves` (`zero_crossing`, `reflection`, `transfer`) and `lagoonviz.core.analysis` (`spectrum`, `wave_parameters`, `compare`), drawn with `lagoonviz.render.wave_figures`. Spectra and comparisons with measured data are also made as paper figures (Chapter 14).

13. 2D Plots

View → *2D plots* (Ctrl+Shift+D, the third mode button next to *Set-up* and *Results*) fills the window with the figure editor: the figures of a project made in LAGOON rather than in a separate plotting program. The panels make way; the timeline stays, so free surface lines and 3D views in the figure follow its time. *Set-up* or *Results* bring back the 3D view with the panels as they were. The figure shown is kept for the next start. What is drawn:

- the time series of the open runs (gauges, forces, motions), with measured files next to them;
- tables of columns: data files (measurements, published results, a spreadsheet), a time-series file as columns, the free surface lines at a moment, or values collected over all open runs (one row per run);
- new columns from expressions of the others;
- x-y panels: any column against any other, as lines, points, steps, bars or a filled area, a right y axis, logarithmic axes, and fitted curves with their equation and R^2 in the legend.

The figure is written as for a paper (Chapter 14): journal widths, fonts in points, vector PDF, SVG or EPS, or PNG and TIFF, and kept as a template that draws the same figure for the next run (Section 14.3).

13.1 Tables

Add table (under the list of sources, on the left) takes the tables of Table 13.1.

Table 13.1: The tables of the 2D plots.

Table from	
Data file	text columns (spaces, tabs, commas or semicolons; a header line of names, units in brackets such as `Fz [N]`; `#` comments; decimal commas with semicolons) or an Excel file (`.xlsx`, the first sheet)
Time-series file	a REEF3D time-series file of the run (`nhflow_wsf_hg`, the forces, ...): its time and columns
Free surface lines	the lines at the current time: x and one column per line
Values over all open runs	one row per open run, a value per line written `name = file: expression`, e.g. `Hm0 P3 = nhflow_wsf_hg: hm0(P3)` or `Fz max = nhflow_6dof_forces_0: max(Fz)`

The table's columns are then in the list of sources: drag one onto the figure (or double-click it) and it is drawn against the table's first column on a new x-y panel. The line's settings choose another *x column*, how it is drawn (*Draw as*: line, scatter, step, bar, fill), the *y axis* (left or right) and a *Fit* (linear, quadratic, cubic, exponential, power, logarithmic). A data file's path is kept relative to the template when it is saved, so a folder of template and data can be moved together.

13.2 The Data Tab

Beside the figure, the *Data* tab shows a table's rows. *Add column* takes `name = expression` in the table's columns by name:

```
eta_cm = 100 * P1
U = sqrt(u**2 + v**2)
H = max(eta) - min(eta)
late = where(t > 20, eta, nan)
smoothed = smooth(Fz, 11)
Kr2 = Kr**2
```

The expressions know `+ - * / ** %`, comparisons, `&`, `|`, `x if c else y`, the constants `pi`, `e`, `g` (9.81), `nan`, `inf`, and the functions `sin`, `cos`, `tan`, `arcsin`, `arccos`, `arctan`, `arctan2`, `sinh`, `cosh`, `tanh`, `exp`, `log`, `log10`, `sqrt`, `abs`, `sign`, `floor`, `ceil`, `round`, `deg`, `rad`, `where`, `clip`, `isnan`, `minimum`, `maximum`, `interp`, `cumsum`, `diff`, `gradient`, `smooth`, `mean`, `median`, `std`, `var`, `min`, `max`, `rms`, `hm0`. A name with spaces or brackets is written `col("Fz [N]")`. A column that cannot be computed is named under the table with the reason. *Remove column* takes away a computed column, *Remove table* the table and its lines, *Copy* puts the table on the clipboard (tab separated, for a spreadsheet).

13.3 Live, New Figures and Scripts

Live (under the figure) draws the figure again when its data change: a table's file written again, the time series of a running simulation. With *Follow running simulation* (Section 4.5) the plots show a run as it goes. *New* starts an empty figure and keeps the tables, which are often drawn again; *Open template...* and *Save template...* switch between figures. In a script:

```
from lagoonviz.core.tables import TableSpec, fit, read_table

flume = TableSpec("flume", "file:flume.dat",
                  formulas={"Kr2": "Kr**2"}).load(session)
line = fit(flume.columns["T"], flume.columns["Kr"], "linear")
print(line.equation, line.r2)
```

`lagoonviz.core.tables.evaluate(expression, columns)` evaluates an expression on a dictionary of arrays.

14. Paper Figures

LAGOON draws figures for papers again with matplotlib, not as screenshots: at the width of a journal column, with fonts in points, axis labels with units, vector output and LaTeX in labels.

14.1 Export Figure

File → *Export figure...* (Ctrl+Shift+P) makes one figure of the kinds of Table 14.1.

Table 14.1: The figures of *Export figure*.

Figure of	
Plots (time series)	the rows of the Plots panel, stacked, sharing the time axis, measured lines dashed
Wave spectra of the plots	a spectrum per row, H_{m0} and T_p in the legend, over a time window
Compare with measurements	a computed line against a column of a measured file
Free surface lines	the free surface lines at the current time
3D view	the 3D view as it is, rendered at the figure's resolution, with its colour bars
Case set-up	the set-up sketch: plan and side view, the main facts (Section 5.12)

Then:

Style *REEF3D* (sans serif, the colours of the REEF3D group), *Journal (serif)* (black first, serif text), *Presentation* (large text, thick lines, a light grid).

Width single column (90 mm), 1.5 columns (140 mm), double column (190 mm) or any; the height follows the content unless it is set.

Font size, Font, Resolution the resolution applies to PNG and TIFF files and to the pixels of a 3D view.

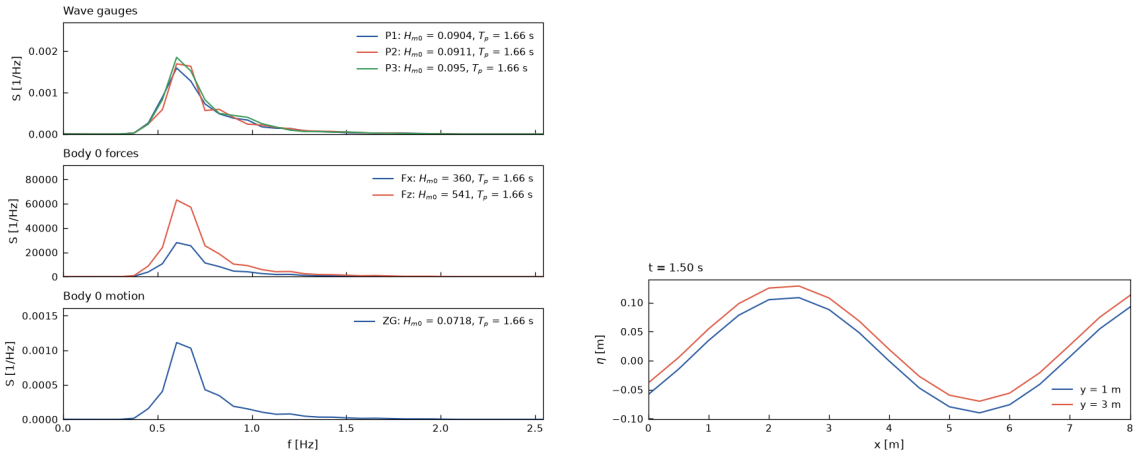
Labels through LaTeX with a LaTeX installation; without it, labels may still hold `$\eta$`-style mathematics.

Title, File the file type follows the name: `.pdf`, `.svg`, `.eps`, `.png` or `.tif`.

14.2 The Figure Editor

The figure editor builds figures of several panels. It is the window's 2D plots mode (*View* → *2D plots*, or *File* → *Figure editor...*, Chapter 13):

Left what can be drawn: the time-series files and free surface lines of the open cases, measured files added with *Add measured file*, and the columns of the figure's tables (*Add table*).



(a) Wave spectra

(b) Free surface lines

Figure 14.1: Paper figures of the example flume: the spectra of the wave gauges and the free surface lines.

Middle the figure's panels (time series, spectra, free surface lines, x-y panels, 3D views) and their lines. Drag a column onto a panel, or double-click it, to add it; onto empty space for a new panel. Below them, the settings of what is selected: of the figure (style, size, a grid of columns, panel letters (a), (b), ...), of a panel (title, how many columns it spans, notes, its moment in time; a 3D view panel takes the 3D view as it is when asked), of a line.

Right the figure as it will be written, drawn again after each change; the *Data* tab shows the tables and adds columns from expressions.

Notes are placed with `x, y: text` (fractions of the panel) and may hold `$LaTeX$`.

A panel of lines has its *Legend*: inside where it covers least (by default only with several lines), in a corner or at a side inside the axes, outside right, above or below the axes, or off; its columns, text size and a frame. A line's *In the legend* leaves it out. A 3D view panel puts its *Colour bars* under the image, right of it, or in a corner of it, with their *Bar length*; each bar's title, numbers and ticks are the item's (Section 9.2).

14.3 Templates

The editor keeps a figure as a template (`*.lagoonfig`): what it shows, by file and column names such as `nhflow_wsf_hg` and `P1`, not the run it came from. Open the next run of the same set-up, open the template, and the same figure is drawn for it. In a script:

```
from lagoonviz.core.figure_template import FigureTemplate
from lagoonviz.render.figure import (save_figure, template_figure,
                                     template_spec)

template = FigureTemplate.load("gauges.lagoonfig")
spec = template_spec(template, dpi=300)
save_figure(template_figure(session, template, spec), "gauges.pdf", spec)
```

Every figure made in the window is also in the session's script (Chapter 18), so it can be made again without the window.

15. Films

File → *Export animation...* (Ctrl+Shift+E) films the 3D view over the timeline into an MP4 file. The frames are drawn off screen at the size of the film, so the film is smooth whatever the window does. A progress dialog shows how far it is; *Stop* ends the film early and keeps what is done.

Table 15.1: The settings of *Export animation*.

Setting	
File	where the `.mp4` goes (by default next to the case)
Size	the 3D view's shape at Full HD, HD or 4K width, or 16:9 sizes up to 4K
Frames	from, to, and every how many, numbered as on the timeline; by default the play range
Frames per second	of the film
Show	the time and the colour bars on the frames; *Legend...* places them
Renderer	LAGOON's own (fast, as the 3D view draws), or Blender Cycles (path traced, Chapter 16)
White water	Cycles only: how much foam, spray and bubbles where the surface breaks (Section 16.3)

The film shows what the 3D view shows: its items, look and camera, or the camera path when it is on (Chapter 11). A size with the 3D view's shape frames exactly what the view shows; a different shape shows more at the sides or above and below. From 2560 pixels wide, a Mac encodes the film with its video hardware, which keeps 4K films fast. Without `imageio-ffmpeg`, numbered PNG frames are written instead of the MP4 file.

15.1 The Time and the Colour Bars

View → *Legend...* (also *Legend...* in the export dialog) says where films and exported images put them and how they look: the corner of the colour bars and the corner of the time (or no time), the bars' length (of the film's width) and thickness, the text size, and a light box behind them. Sizes follow the film: a 4K film gets them twice as large as Full HD. Each bar is the item's: on or off, its title, numbers and ticks (Section 9.2). The same dialog switches off the colour bars under the 3D view. The settings are kept with the session; in a script:

```
session.set_legend(bars="lower right", time="upper left", length=0.35, box=True)
write_movie(session, "wave.mp4", legend=True)
```

15.2 Films Without a Screen

```
lagoon-render flume.lagoon-session flume.mp4 --size 1920x1080 --fps 25
lagoon-render ~/runs/flume flume.mp4 --start 100 --end 400 --every 2 \
    --style ocean
```

A saved session brings its items, view, play range and camera path; a case folder is shown as it opens. Section 19.1 lists all options.

15.3 Path-Traced Films

With *Renderer: Blender Cycles*, each frame is path traced by Blender: light reflected and refracted by the water, soft shadows, foam and spray. This takes seconds to minutes per frame instead of a fraction of a second (Chapter 16).

16. Cinematic Looks and Path Tracing

The cinematic looks and path tracing are made for showcase pictures rather than for reading values.

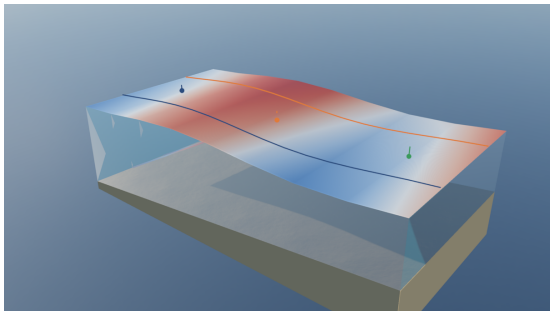
16.1 Cinematic Looks

View → *Viewport look* has five cinematic looks besides Studio and Ocean (Fig. 16.1, Table 16.1). They use physically based materials: smooth water that reflects its surroundings, sand and rock with a fine grain, concrete, steel, ice and wood (the *Material* of bodies, structures and other objects). The surroundings are made by LAGOON; nothing is downloaded. Water coloured by a variable keeps its colours; coloured by nothing, it takes the look's water colour.

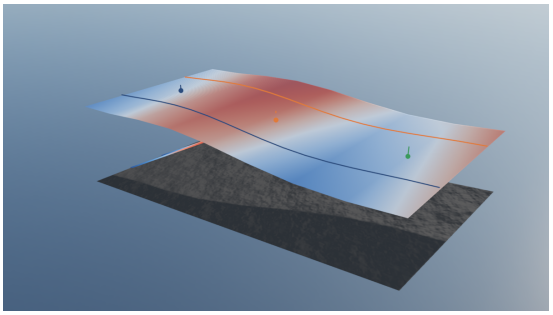
Table 16.1: The cinematic looks in the 3D view and path traced.

Look	**3D view**	**Path traced (Cycles)**
Open ocean	the sky picture around the model, reflected by the water	the model in the sky picture; the bed dressed as land by height and slope, cut off in rock at the sides; ripples and shore foam
Wave flume	the sky picture around the model; the bed in LAGOON's light sand	as a wave tank of ÆgirFrames: the sky picture, glass water, a dark rock floor
Underwater	the camera below the surface: blue depth fog, light from above	LAGOON's own deep-water surroundings, water that absorbs and scatters light
Diorama	the model on a stand in the sky picture: a solid block of light sand, see-through water with blue sides down to the bed	the same, the bed dark rock, the water a clear blue body of glass
Diorama classic	the earlier Diorama: a blue backdrop, LAGOON's own soft sky, a block of sand	the sky picture, a block of sand

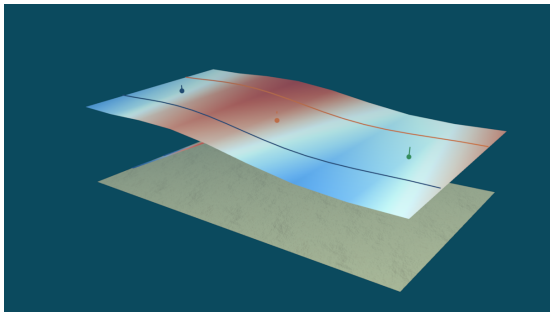
In the Open ocean, Wave flume and Diorama looks, the 3D view shows the same sky picture as the path-traced pictures, tone mapped as Blender shows it, so the 3D view previews what Blender renders. Structures and bodies in their default colours are ÆgirFrames's red (half metallic, matte); the bed keeps LAGOON's light sand in the 3D view, while path traced a bed in its default colour is ÆgirFrames's dark rock in the Wave flume and Diorama looks. Colours and materials you choose are kept. In the 3D view the water stays a see-through surface rather than a body of glass.



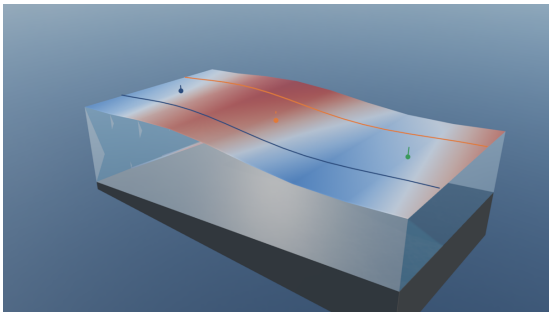
(a) Open ocean



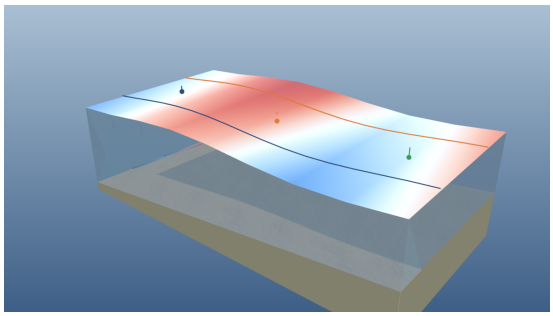
(b) Wave flume



(c) Underwater



(d) Diorama



(e) Diorama classic

Figure 16.1: The cinematic looks in the 3D view.

16.2 Path Tracing with Blender

Blender's Cycles path tracer [Blender Online Community, 2026] renders the same scene with light that bounces: reflections and refraction through the water, soft shadows, light through the surface onto the bed. LAGOON runs Blender in the background as a separate program; Blender's window never opens (Fig. 16.2).

1. Install Blender 4.2 or newer. LAGOON finds it in Applications (macOS), on the `PATH`, or where the environment variable `LAGOON_BLENDER` points.
2. Choose a cinematic look. With Studio or Ocean, Cycles uses Wave flume or Open ocean.
3. *File → Render still with Cycles...* renders one picture of the 3D view as it is; *File → Export animation... → Renderer: Blender Cycles* renders a film.

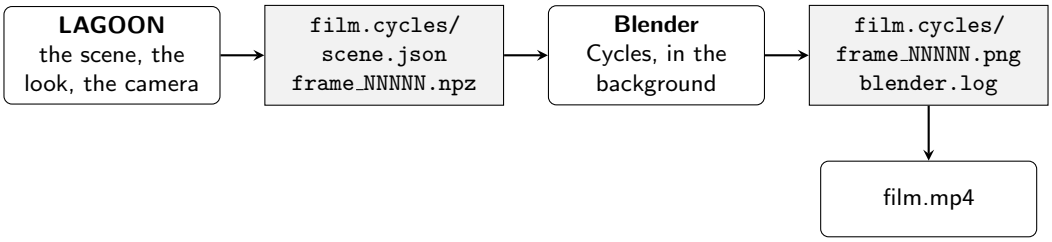


Figure 16.2: Path tracing: LAGOON writes the scene of every frame into a folder beside the film, Blender renders the frames, LAGOON joins them into the film.

```
lagoon-render flume.lagoon-session still.png --engine cycles \
    --style open_ocean
lagoon-render flume.lagoon-session film.mp4 --engine cycles \
    --samples 64 --size 1920x1080
```

Water

Clear water, a free surface coloured by nothing, goes to Blender as a body of water: the surface, walls along the sides of the domain and the shore, and a floor. Above water it is rendered as ÆgirFrames renders the ocean [NTNU, 2026]: glass in the look's water colour that refracts and reflects what the camera sees, while light, shadows and reflections pass through it, so the bed and the structures under it are lit as in air and cast their shadows on the bed; a faint haze fills it. The surface is subdivided when rendered, for smooth crests: twice, unless its mesh is already as fine as the picture's pixels. In the Open ocean look it carries small wind ripples that break the reflections into glitter. Under water, the water absorbs light the deeper it goes and scatters it. Water coloured by a variable stays an opaque, coloured surface, so its colours read. Where the water thins out to nothing at the shore it fades away, so the sand shows instead of a sheet of glass.

The sky picture

The Open ocean, Wave flume and Diorama looks are path traced as ÆgirFrames shows a case: the domain as a model floating in ÆgirFrames's sky picture, a panorama of a clear sky over the sea with a low sun, which lights the model and is seen behind it. The picture is turned so that its sun stands in the sun's direction (*View* → *Sun*...); its sun's height is the picture's own, about 30°. The picture is rendered with Blender's AgX view transform as it is.

Coasts

In the Open ocean look, a coast looks like one: the bed, coloured by nothing, is dressed by its height above the still water level and its slope, with sand under water, wet sand at the water line, dry sand, grass and heath higher up, and rock where it is steep, all with a fine relief. At the sides of the domain the land is cut off in rock down to a base, and the water ends in clear walls, with small wind ripples on it and broken foam just off the beach.

16.3 Foam, Spray and Bubbles

Path-traced films and stills show white water where the free surface breaks: spray thrown off the crests, foam drifting on the surface and fading, bubbles under it rising to the top. The model follows the diffuse particles of Ihmsen et al. [2012]. White water is born where REEF3D flags breaking (NHFLOW's `breaking`), on sharp crests whose water runs forward fast, and where the surface flow converges and traps air; it moves with the free surface's velocity from output to output. Its size and lifetime scale with the case (the cells and the water depth), so a flume and a coast look alike. A still runs the foam from a few wave periods before its time, so that it has formed.

Foam needs the free surface's velocity on its grid (NHFLOW and FNPF write it) and the output times in seconds. How much there is: *View → White water in Cycles* (*None*, *Light*, *Medium*, *Heavy*; Medium by default, Heavy as much as before October 2026), also in the export dialog, or `--white-water light` on the command line (`--no-foam`: none). In a script: `path_trace(..., foam="light")`.

16.4 Rendering Time

Path tracing takes from seconds to minutes per frame, depending on the size, the samples and the GPU. The progress dialog shows what Blender is doing; its first run on a computer compiles its GPU code, which takes a few minutes once.

Samples per pixel (default 64): the images are denoised, so 32 to 64 are usually enough; 16 for a quick look.

Size HD takes half the time of Full HD, 4K four times as long.

GPU Blender uses the GPU (Metal, OptiX, CUDA, HIP or oneAPI) when there is one; the first lines of its log say which.

Meshes whose triangles stay the same from frame to frame are moved in place instead of being built again, and similar materials are shared, so the time of a frame is spent on light. Each frame's line in `<film>.cycles/blender.log` says how long building and rendering took, for example:

```
LAGOON frame 3/5 399189 triangles 4.2 s (building 0.0 s, rendering 4.2 s)
```

A stopped film goes on where it stopped: the scene and the frames are kept in `<film>.cycles/` beside the film, and when the film is made again, frames whose data, camera and settings have not changed are kept. `--fresh` renders every frame again.

17. The LAGOON Format

A LAGOON store (REEF3D_<SOLVER>.lagoon) holds a whole REEF3D run in one folder: the volume, the free surface and the bed, time series, free surface lines, floating bodies, particles and objects, AMR surfaces and CFD's AMR blocks, each stored once in a compact, chunked form. It replaces the many VTU, VTP and .dat files of a run: 4 to 9 times smaller for grids, often a thousand times for floating bodies, and much faster to open, step through and probe. It is Zarr version 3 [Zarr Developers, 2023], so other tools can read it too.

17.1 Making a Store

While REEF3D runs: P 18

Table 17.1: The LAGOON store in ctrl.txt.

P 18	
0	no store (default)
1	the store *instead of* the VTU/VTP files of what it holds
2	the store *as well as* the VTU/VTP files

REEF3D writes ./REEF3D_<SOLVER>.lagoon next to its other output, for FNPF, NHFLOW, SFLOW and CFD:

- the volume (FNPF and NHFLOW: σ-grid; CFD: Cartesian grid) when the VTU output is on (**P 10**); each MPI rank writes its block in the background, so the run does not wait for it;
- the free surface and the bed (FNPF, NHFLOW, SFLOW; CFD: the topography as bed) when their VTP output is on;
- curvilinear grids (**G 50**, NHFLOW and SFLOW), which the converter cannot read from the VTK files;
- floating bodies (6DOF): the mesh once and the motion of each output;
- NHFLOW particles, DEM elements and FNPF ice floes;
- the AMR surface of FNPF, NHFLOW and SFLOW with refinement: level 0 and the refined patches of every output; CFD's refined 3D blocks.

A run never adds to the store of an earlier run: that store is renamed REEF3D_<SOLVER>-<run>.lagoon-earlier when the new run starts. If the store cannot be written, that output stops going into the store on all ranks and REEF3D writes its VTU and VTP files again for the rest of the run.

From the VTK files: lagoon-convert

In LAGOON, *File → Convert to LAGOON format...* converts the open case in the background and opens it again from the store. On the command line:

```
lagoon-convert CASE                          # everything there is
lagoon-convert CASE --outputs volume,bodies  # some outputs only
lagoon-convert CASE --out other.lagoon       # another place
```

Converting again appends what a running simulation has written since. A store of another run is not appended to: move it away first. Fields are kept bit for bit, heights and body vertices to float32 precision. The converter also adds what REEF3D does not write into the store itself: CFD's free surface, CPM particles, mooring lines, rod trees, the time series files and the `REEF3D_CFD_AMR` files.

17.2 Opening a Store

LAGOON Open the case folder as usual (Section 4.7).

ParaView *Tools → Manage Plugins → Load New* and pick the plugin `lagoon-format/paraview/lagoon_reader.py`; then open the store's top `zarr.json` and choose *LAGOON store*. Grids come as structured blocks, bodies as triangles, particles as points, objects as polygons or lines, AMR surfaces as rectilinear grids per level and patch, all on the time bar.

Python with the package `lagoon-format` (`pip install lagoon-format`, NumPy only):

```
from lagoon_format.store import Store

store = Store("REEF3D_NHFLOW.lagoon")
print(store.solver, store.run_id, list(store.outputs))

volume = store["volume"]
volume.times                        # time of every output [s]
u = volume.read("velocity", 10)     # (nz, ny, nx, 3) at output 10
z = volume.heights(10)              # (nz, ny, nx) heights of the points
eta = store["free_surface"].series("eta", i=40, j=12)  # one point

gauges = store.tables["nhflow_wsf_hg"]     # a time series file
gauges.time, gauges.column("P1")
vertices, triangles = store.bodies["nhflow_body"].mesh(10)
```

zarr-python reads the arrays directly as well (the byte shuffle needs `numcodecs`):

```
import zarr

root = zarr.open_group("REEF3D_NHFLOW.lagoon", mode="r")
root.attrs["lagoon"]                          # what the store holds
root["volume/blocks/b0000/velocity"][10]      # one MPI block, output 10
```

The format guide, the specification and the description of the run log are in the `lagoon-format` folder of the repository (`GUIDE.md`, `SPEC.md`, `RUN.md`).

18. Scripting

Everything the window does goes through one object, `lagoonviz.core.Session`: the window calls its methods, and so can a script, a notebook or a batch job on a cluster, without Qt and without a screen.

18.1 A Session in Python

```
from lagoonviz.core import Session

session = Session()
case = session.open_case("~/runs/flume")   # the case, its surface and bed
session.set_property("free_surface-1", "colormap", "ocean")
session.set_exaggeration(case, 8.0)         # the case's vertical scale
session.add_slice("y", 2.0)                 # a slice at y = 2 m
session.set_time(12.5)
session.save("flume.lagoon-session")        # open it in the window later

mesh = session.data.load("free_surface-1")  # the output as NumPy arrays
print(mesh.points.shape, mesh.point_data["eta"].max())
```

Item ids are the kind and a number (`free_surface-1`, `slice-1`); `session.scene` lists them. Every change is a command: `session.undo()` and `session.redo()` work as in the window. The property names are listed in Section 19.3, the methods of `Session` in Section 19.4. The set-up, the maps and the sea at a case are scripted the same way (Chapters 5 to 8).

18.2 The Window Writes the Script

The Python console shows every command the window runs, and runs Python with `session` in scope. *File → Export script...* writes the whole session as a script:

```
# LAGOON session script, recorded from the command history
from lagoonviz.core import Session

# figures, films and screenshots: made again only when run as a script
MAKE_OUTPUTS = __name__ == "__main__"

session = Session()
session.open_case('/Users/me/runs/flume')
session.set_property('free_surface-1', 'colormap', 'ocean')
if MAKE_OUTPUTS:  # File > Export animation
    from lagoonviz.render.movie import frame_times, write_movie
    times = frame_times(session, None, None, 1)
    write_movie(session, '/Users/me/runs/flume/flume.mp4', times,
                (1920, 1080), 25.0, 'studio')
```

The figures, films and screenshots made in the window are in it, each after the change it followed. `python flume.py` makes them all again, without the window; for another run of the same set-up, change the path of `open_case`. `Session.from_script(text)` rebuilds only the scene, without the outputs.

18.3 Pictures, Films and Figures

```
from lagoonviz.render.offscreen import render_session, write_png
from lagoonviz.render.movie import frame_times, write_movie
from lagoonviz.render.figure import FigureSpec, save_figure, series_figure
from lagoonviz.render.pathtrace import path_trace

write_png("view.png", render_session(session, (1920, 1080), style="ocean"))
write_movie(session, "flume.mp4", frame_times(session), (1920, 1080), 25.0)

spec = FigureSpec.from_style("Journal (serif)", width_mm=90)
save_figure(series_figure(session, spec), "gauges.pdf", spec)

session.orbit_camera(degrees=360)           # a camera path around the scene
session.set_sun(azimuth=200, elevation=20)  # where the light comes from
path_trace(session, "showcase.mp4", size=(1920, 1080),
           look="open_ocean", samples=64)
```

The command line (Section 19.1) does the common cases without Python.

19. Reference

19.1 Command Line

lagoon

The application.

```
lagoon [PATH] [--version] [--dev-remote DIR]
```

PATH a session (`.lagoon-session`), a REEF3D case folder, an output folder or file, or a LAGOON store to open.

--**version** print the version.

--**dev-remote DIR** developer testing: take screenshots and simple actions from `DIR/request.json`.

`python -m lagoonviz` does the same.

lagoon-render

Images and films without a screen: on a cluster, in a batch job, or for a picture at a size the window does not have.

```
lagoon-render SOURCE OUTPUT [options]
```

`SOURCE`	a case folder, an output file, a `.lagoon-session` or a `.py` script
`OUTPUT`	an image (`.png`) or a film (`.mp4`, `.mov`)
`--time TIME`	the time (or output) to show
`--size SIZE`	image size, e.g. 1920x1080 (default 1600x900)
`--view VIEW`	`perspective`, `top`, `front`, `right`, or the 2D views `2d`, `2d-front`, `2d-right` (default: the session's view)
`--style STYLE`	the look: `studio`, `ocean`, `open_ocean`, `wave_flume`, `underwater`, `diorama`, `diorama_classic`
`--fps FPS`	films: frames per second (default 25)
`--start START`	films: the first output (default: the first with data)
`--end END`	films: the last output (default: the last)
`--every EVERY`	films: every n-th output
`--engine ENGINE`	`pygfx` (LAGOON's own renderer, the default) or `cycles` (path traced by Blender)
`--samples SAMPLES`	Cycles: samples per pixel (default 64)

`--blender BLENDER`	Cycles: the Blender executable (default: found)
`--fresh`	Cycles: render every frame again, instead of keeping the frames rendered before from the same data, camera and settings
`--white-water LEVEL`	Cycles: how much foam, spray and bubbles where the free surface breaks: `none`, `light`, `medium` (default) or `heavy`
`--no-foam`	Cycles: no white water (as `--white-water none`)

```
lagoon-render ~/runs/flume view.png --time 120 --size 1920x1080
lagoon-render flume.lagoon-session flume.mp4 --start 100 --end 400 \
    --every 2
lagoon-render flume.lagoon-session still.png --engine cycles \
    --style open_ocean
```

lagoon-convert

REEF3D's VTU/VTP output to a LAGOON store (Chapter 17). Run again, it appends the outputs written since.

```
lagoon-convert CASE [options]
```

`CASE`	the REEF3D case folder
`--out OUT`	the store to write (default `CASE/REEF3D_<SOLVER>.lagoon`)
`--outputs OUTPUTS`	what to convert, comma separated: `volume`, `free_surface`, `bed`, `bodies`, `particles`, `dem`, `ice`, `rods`, `mooring`, `series`, `profiles`, `amr` (default: everything there is)
`--limit LIMIT`	convert at most this many outputs per stream
`--shard-time N`	outputs per shard file (default 16)

lagoon-setup

A case's set-up, from its input files, in plan and side view: the domain and its sides, relaxation zones, gauges, probes, bodies, structures, solids (Section 5.12).

```
lagoon-setup CASE [OUTPUT] [options]
```

`CASE`	the case folder, with `control.txt` and/or `ctrl.txt`
`OUTPUT`	the figure (default `<case>-setup.pdf` beside the case); PDF, SVG, EPS, PNG, TIFF
`--width WIDTH`	width in mm (default 140)
`--style STYLE`	`REEF3D` (default), `Journal (serif)`, `Presentation`

`--describe`	print the main facts of the input instead

Environment variables

`LAGOON_BLENDER`	the Blender executable for path tracing
`LAGOON_DIVEMESH`	the DIVEMesh program (`DiveMESH`) the set-up runs
`LAGOON_MEMORY`	memory use: `performance`, `balanced` (the default), `low` or `streaming` (Section 10.9)
`LAGOON_DRAFTS`	the folder of draft cases started on the map
`LAGOON_CACHE`	the cache of map tiles and downloaded bathymetry
`EARTHDATA_TOKEN`	a NASA Earthdata token, to download ICESat-2 soundings
`LAGOON_LOG`	also write the application's log to this file
`LAGOON_NO_GPU`	start the window without the 3D view (e.g. over a remote desktop)

19.2 Menus and Keyboard Shortcuts

The menus of the main window, with their keyboard shortcuts. On macOS, `Ctrl` is the command key `Cmd`.

File

Open case...	`Ctrl+Shift+O`
New case...	
New case from map...	
Open output file...	
Reload case	`F5`
Follow running simulation (on/off)	
Save input files	
Run DIVEMesh	
New session	`Ctrl+N`
Open session...	`Ctrl+O`
Save session	`Ctrl+S`
Save session as...	`Ctrl+Shift+S`
Convert to LAGOON format...	
Clean up case folder...	
Export script...	
Export figure...	`Ctrl+Shift+P`
Figure editor...	
Export image...	`Ctrl+Shift+I`
Export animation...	`Ctrl+Shift+E`
Render still with Cycles...	

Save screenshot...	F12
Quit	

Edit

Undo	Ctrl+Z
Redo	Ctrl+Y
Remove	Del

Add

Every kind of item (Table 9.1); the entries marked ▷ open a list of *Automatic* and the outputs of the open cases the item can show (Section 9.6).

Case	Free surface ▷	Free Surface AMR
Bed ▷	Volume	Time series
Slice ▷	Iso-surface ▷	Iso-volume ▷
Streamlines ▷	Free surface line	Gauge
Bodies	AMR blocks	Mooring lines
Structures	DEM elements	Ice floes
Rod trees	Case set-up	River corridor
Multi-block grid	DIVEMesh grid	Background map
geo.dat points	Sea state	Statistics ▷
Particles		

View

Set-up (on/off)	Ctrl+Shift+U
Results (on/off)	Ctrl+Shift+R
2D plots (on/off)	Ctrl+Shift+D
Scene, Properties, Set-up, Plots, Timeline, Python console, Free surface lines, Run monitor (on/off)	
Wave analysis...	
Theme → Light, Dark	
Viewport look → Studio (light), Ocean (dark blue), Open ocean, Wave flume, Underwater, Diorama, Diorama classic (cinematic)	
Memory use → Performance (in memory), Balanced, Low memory, Streaming (very large cases)	
White water in Cycles → None, Light, Medium, Heavy	

Sun...	
Legend...	
Compare side by side (on/off)	`Ctrl+Shift+2`
Full screen viewer (on/off)	`Ctrl+Shift+F`
Play / pause	`Space`
Previous frame	`Ctrl+Left`
Next frame	`Ctrl+Right`
Fit range to this frame	`Ctrl+F`
Camera path → Add camera keyframe	`Ctrl+K`
Camera path → Remove camera keyframe	
Camera path → Follow camera path (on/off)	
Camera path → Orbit around the scene	
Camera path → Clear camera keyframes	
Save screenshot...	`F12`
Reset layout	

Help

Record a performance report (on/off)
About LAGOON

19.3 Item Properties

The properties of every kind of scene item, as the Properties panel shows them and as scripts set them:

```
session.set_property("free_surface-1", "colormap", "rainbow")
session.set_properties("slice-1", normal="x", position=12.0)
```

Colour by, *Contour lines of*, *Vectors of* and the other variable properties take the names of the output's variables (`none` for none), e.g. `eta`, `velocity` or one of its components `velocity.x`. *Case* and *Dataset* empty mean the first case and the item's own output. The colouring properties that every coloured item shares are listed once, in Table 19.10.

Table 19.10: The colouring properties of every coloured item.

Property	Name in scripts	Default	Choices
Automatic range	`auto_range`	`True`	
Range from	`range_from`	`'1st - 99th percentile'`	`1st - 99th percentile, lowest to highest`
Range min	`vmin`	`0.0`	

Property	Name in scripts	Default	Choices
Range max	vmax	1.0	
Range [%]	range_percent	100.0	
Opacity	opacity	1.0	
Colour bar	legend	True	
Bar title	legend_title	--	
Bar numbers	legend_format	'auto'	auto, integer, 1 decimal, 2 decimals, 3 decimals, scientific
Bar ticks	legend_ticks	0	

Case (case)

Property	Name in scripts	Default	Choices
Path	path	--	
Output interval [s]	print_interval	0.0	
Vertical scale	exaggeration	1.0	
Same for all items	sync_exaggeration	True	

Free surface (free_surface)

Property	Name in scripts	Default	Choices
Case	case	--	
Dataset	dataset	--	
Colour by	variable	'eta'	
Colour map	colormap	'reef3d'	the colour maps (Table 9.2)
Automatic range ... *Bar ticks*: the colouring properties (Table 19.10)			
Grid lines	grid_lines	'off'	off, dark, white
Contour lines of	contour_by	'none'	
Contour lines	contour_levels	10	
Vectors of	vectors_of	'none'	
Arrows	vector_count	400	
Arrow length	vector_scale	1.0	
Vertical scale	exaggeration	1.0	
Hide dry land	hide_dry	True	

Free Surface AMR (`amr`)

Property	Name in scripts	Default	Choices
Case	`case`	`--`	
Dataset	`dataset`	`--`	
Colour by	`variable`	`'eta'`	
Colour map	`colormap`	`'reef3d'`	the colour maps (Table 9.2)
Automatic range ...	*Bar ticks*: the colouring properties (Table 19.10)		
Contour lines of	`contour_by`	`'none'`	
Contour lines	`contour_levels`	`10`	
Vertical scale	`exaggeration`	`1.0`	
AMR patches	`patches`	`True`	
Patch outlines	`outlines`	`'dark'`	`dark, white, off`
Grid lines	`grid_lines`	`'off'`	`off, dark, white`
Hide dry land	`hide_dry`	`True`	

Bed (`bathymetry`)

Property	Name in scripts	Default	Choices
Case	`case`	`--`	
Dataset	`dataset`	`--`	
Colour by	`variable`	`'none'`	
Colour map	`colormap`	`'sand'`	the colour maps (Table 9.2)
Automatic range ...	*Bar ticks*: the colouring properties (Table 19.10)		
Grid lines	`grid_lines`	`'off'`	`off, dark, white`
Contour lines of	`contour_by`	`'none'`	
Contour lines	`contour_levels`	`10`	
Vectors of	`vectors_of`	`'none'`	
Arrows	`vector_count`	`400`	
Arrow length	`vector_scale`	`1.0`	
Vertical scale	`exaggeration`	`1.0`	
Hide wet land	`hide_wet`	`False`	

Property	Name in scripts	Default	Choices

Volume (`volume`)

Property	Name in scripts	Default	Choices
Case	`case`	`--`	
Dataset	`dataset`	`--`	
Colour by	`variable`	`'velocity'`	
Colour map	`colormap`	`'ocean'`	the colour maps (Table 9.2)
Automatic range ... *Bar ticks*: the colouring properties (Table 19.10)			
Grid lines	`grid_lines`	`'off'`	`off, dark, white`

Time series (`series`)

Property	Name in scripts	Default	Choices
Case	`case`	`--`	
File	`source`	`--`	
Columns	`columns`	`--`	
Gauges in 3D	`gauges_3d`	`True`	
Line names	`labels`	`--`	
Legend	`plot_legend`	`'above the plot'`	`above the plot, in the plot, off`
Values at the cursor	`legend_values`	`True`	
Measured file	`measured`	`--`	
Measured columns	`measured_columns`	`--`	
Measured time shift [s]	`measured_shift`	`0.0`	

Slice (`slice`)

Property	Name in scripts	Default	Choices
Case	`case`	`--`	
Dataset	`dataset`	`--`	
Normal	`normal`	`'y'`	`x, y, z`
Position [m]	`position`	`0.0`	
Colour by	`variable`	`'velocity'`	
Colour map	`colormap`	`'ocean'`	the colour maps (Table 9.2)
Automatic range ... *Bar ticks*: the colouring properties (Table 19.10)			
Grid lines	`grid_lines`	`'off'`	`off, dark, white`
Contour lines of	`contour_by`	`'none'`	

Property	Name in scripts	Default	Choices
Contour lines	contour_levels	10	
Vectors of	vectors_of	'none'	
Arrows	vector_count	400	
Arrow length	vector_scale	1.0	

Iso-surface (iso_surface)

Property	Name in scripts	Default	Choices
Case	case	--	
Dataset	dataset	--	
Iso-surface of	iso_by	'none'	
Value	iso_value	0.0	
Colour by	variable	'none'	
Colour map	colormap	'ocean'	the colour maps (Table 9.2)
Automatic range ... *Bar ticks*: the colouring properties (Table 19.10)			
Grid lines	grid_lines	'off'	off, dark, white

Iso-volume (iso_volume)

Property	Name in scripts	Default	Choices
Case	case	--	
Dataset	dataset	--	
Iso-volume of	iso_by	'none'	
From	iso_min	0.0	
To	iso_max	1.0	
Colour by	variable	'none'	
Colour map	colormap	'ocean'	the colour maps (Table 9.2)
Automatic range ... *Bar ticks*: the colouring properties (Table 19.10)			

Property	Name in scripts	Default	Choices
Case	`case`	`--`	
Dataset	`dataset`	`--`	
Streamlines of	`streamlines_of`	`'velocity'`	
Lines in	`lines_in`	`'volume'`	`volume`, `seed plane`, `free surface`
Seeds on plane	`seed_normal`	`'y'`	`x`, `y`, `z`
Plane at [m]	`seed_position`	`0.0`	
Lines	`seeds`	`60`	
Colour by	`variable`	`'velocity'`	
Colour map	`colormap`	`'ocean'`	the colour maps (Table 9.2)
Automatic range ... *Bar ticks*: the colouring properties (Table 19.10)			
Line width [px]	`line_width`	`2.0`	

Free surface line (`profile`)

Property	Name in scripts	Default	Choices
Case	`case`	`--`	
Lines from	`source`	`--`	
Lines	`lines`	`--`	
Wave theory	`theory`	`True`	
Lines in 3D	`profile_3d`	`True`	

Gauge (`gauge`)

Property	Name in scripts	Default	Choices
x [m]	`x`	`0.0`	
y [m]	`y`	`0.0`	
Show in plot	`show_in_plot`	`True`	

Bodies (`body`)

Property	Name in scripts	Default	Choices
Case	`case`	`--`	
Dataset	`dataset`	`--`	
Material	`material`	`'steel'`	`steel, concrete, rock, ice, wood`
Colour by	`variable`	`'none'`	
Colour map	`colormap`	`'viridis'`	the colour maps (Table 9.2)
Automatic range ... *Bar ticks*: the colouring properties (Table 19.10)			

AMR blocks (`amr_blocks`)

Property	Name in scripts	Default	Choices
Case	`case`	`--`	
Dataset	`dataset`	`--`	
Colour by	`variable`	`'level'`	
Colour map	`colormap`	`'viridis'`	the colour maps (Table 9.2)
Automatic range ... *Bar ticks*: the colouring properties (Table 19.10)			
Line width [px]	`line_width`	`1.5`	

Mooring lines (`mooring`)

Property	Name in scripts	Default	Choices
Case	`case`	`--`	
Dataset	`dataset`	`--`	
Colour by	`variable`	`'Tension'`	
Colour map	`colormap`	`'tension'`	the colour maps (Table 9.2)
Automatic range ... *Bar ticks*: the colouring properties (Table 19.10)			
Line width [px]	`line_width`	`3.0`	

Structures (`structure`)

Property	Name in scripts	Default	Choices
Property	**Name in scripts**	**Default**	**Choices**
Case	`case`	`--`	
Dataset	`dataset`	`--`	
Material	`material`	`'concrete'`	`steel, concrete, rock, ice, wood`
Colour by	`variable`	`'none'`	
Colour map	`colormap`	`'greys'`	the colour maps (Table 9.2)
Automatic range ... *Bar ticks*: the colouring properties (Table 19.10)			

DEM elements (`dem`)

Property	Name in scripts	Default	Choices
Case	`case`	`--`	
Dataset	`dataset`	`--`	
Material	`material`	`'rock'`	`steel, concrete, rock, ice, wood`
Colour by	`variable`	`'none'`	
Colour map	`colormap`	`'viridis'`	the colour maps (Table 9.2)
Automatic range ... *Bar ticks*: the colouring properties (Table 19.10)			
Trail [outputs]	`trail_outputs`	0	

Ice floes (`ice`)

Property	Name in scripts	Default	Choices
Case	`case`	`--`	
Dataset	`dataset`	`--`	
Material	`material`	`'ice'`	`steel, concrete, rock, ice, wood`
Colour by	`variable`	`'none'`	
Colour map	`colormap`	`'viridis'`	the colour maps (Table 9.2)
Automatic range ... *Bar ticks*: the colouring properties (Table 19.10)			

Rod trees (`rods`)

Property	Name in scripts	Default	Choices
Case	`case`	`--`	
Dataset	`dataset`	`--`	
Material	`material`	`'wood'`	`steel, concrete, rock, ice, wood`
Colour by	`variable`	`'none'`	
Colour map	`colormap`	`'viridis'`	the colour maps (Table 9.2)
Automatic range ... *Bar ticks*: the colouring properties (Table 19.10)			

Case set-up (`setup`)

Property	Name in scripts	Default	Choices
Case	`case`	`--`	
Domain	`show_domain`	`True`	
Sides	`show_sides`	`True`	
Still water	`show_water`	`True`	
Relaxation zones	`show_zones`	`True`	
Waves	`show_waves`	`True`	
Gauges	`show_gauges`	`True`	
Probes	`show_probes`	`True`	
Sections	`show_sections`	`True`	
Forces	`show_forces`	`True`	
Floating bodies	`show_bodies`	`True`	
Structures	`show_structures`	`True`	
Regions	`show_regions`	`True`	
Solids	`show_solids`	`False`	
Bed	`show_bed`	`True`	
Curvilinear grid	`show_curvgrid`	`True`	
River drawn	`show_river`	`True`	
Labels	`labels`	`True`	

River corridor (`corridor`)

Property	Name in scripts	Default	Choices
Case	`case`	`--`	
Points	`show_points`	`True`	
Grid	`show_grid`	`True`	

Multi-block grid (`multiblock`)

Property	Name in scripts	Default	Choices
Case	`case`	`--`	
Points	`show_points`	`True`	
Grid	`show_grid`	`True`	
Quality	`show_quality`	`True`	
Rules	`show_issues`	`True`	

DIVEMesh grid (`grid`)

Property	Name in scripts	Default	Choices
Case	`case`	`--`	
Faces	`faces`	`'bottom and back'`	`bottom and back, bottom, back, outside`
Every n-th line	`every`	`0`	
MPI partition	`partition`	`True`	

Background map (`basemap`)

Property	Name in scripts	Default	Choices
Case	`case`	`--`	
Map	`source`	`'Kartverket topo'`	`Kartverket topo, Kartverket grey, Kartverket nautical chart, Sentinel-2 (satellite), OpenStreetMap, GeoTIFF file, Tile server URL`
File or URL	`file`	`--`	
Margin	`margin`	`0.5`	
Placed	`placement`	`'below the model'`	`below the model, at the still water level`
Opacity	`opacity`	`1.0`	
Resolution [px]	`pixels`	`2048`	

geo.dat points (`geodat`)

Property	Name in scripts	Default	Choices
Case	`case`	`--`	
Dataset	`dataset`	`--`	
Colour by	`variable`	`'elevation'`	
Colour map	`colormap`	`'viridis'`	the colour maps (Table 9.2)
Automatic range ...	*Bar ticks*: the colouring properties (Table 19.10)		
Point size [px]	`point_size`	`5.0`	

Sea state (`seastate`)

Property	Name in scripts	Default	Choices
Case	`case`	`--`	
Dataset	`dataset`	`--`	
Colour by	`variable`	`'Hs'`	
Colour map	`colormap`	`'rainbow'`	the colour maps (Table 9.2)
Automatic range ...	*Bar ticks*: the colouring properties (Table 19.10)		
Grid lines	`grid_lines`	`'off'`	`off, dark, white`
Contour lines of	`contour_by`	`'none'`	
Contour lines	`contour_levels`	`10`	
Vectors of	`vectors_of`	`'none'`	
Arrows	`vector_count`	`400`	
Arrow length	`vector_scale`	`1.0`	
Hide dry land	`hide_dry`	`True`	

Statistics (`statistics`)

Property	Name in scripts	Default	Choices
Case	`case`	`--`	
Of	`source`	`--`	
From [s]	`start`	`0.0`	
To [s]	`end`	`0.0`	
Variables	`of_variables`	`--`	
Wet from [m]	`wet_depth`	`0.01`	
Colour by	`variable`	`'Hs'`	
Colour map	`colormap`	`'rainbow'`	the colour maps (Table 9.2)
Automatic range ...	*Bar ticks*: the colouring properties (Table 19.10)		
Contour lines of	`contour_by`	`'none'`	

Property	Name in scripts	Default	Choices
Contour lines	`contour_levels`	`10`	
Vectors of	`vectors_of`	`'none'`	
Arrows	`vector_count`	`400`	
Arrow length	`vector_scale`	`1.0`	
Hide never wet	`hide_dry`	`True`	

Particles (`particles`)

Property	Name in scripts	Default	Choices
Case	`case`	`--`	
Dataset	`dataset`	`--`	
Colour by	`variable`	`'velocity'`	
Colour map	`colormap`	`'viridis'`	the colour maps (Table 9.2)
Automatic range ... *Bar ticks*: the colouring properties (Table 19.10)			
Point size [px]	`point_size`	`4.0`	
Trail [outputs]	`trail_outputs`	`0`	

19.4 Session API

The methods of `lagoonviz.core.Session` (Chapter 18). Changes to the scene and to the input files are commands that can be undone and are recorded in the script; view state (time, camera, camera path, 2D view, play range, sun, legend) is not an undo step. The online documentation lists the full signatures and the rendering and figure functions.

Method	What it does
`add_background_map(source, **properties)`	A background map round a georeferenced case: map tiles (`source`, see `kinds.BASEMAP_SOURCES`), or `source="GeoTIFF file", file=...`.
`add_block(area, cells, case)`	One more block over a rectangle (x0, x1, y0, y1), with its own edges.
`add_camera_key(time, state)`	A camera keyframe: the camera `state` (default: the saved view) at `time` (default: now); a key at the same time is replaced. Turns the camera path on. Returns the number of keys. View state: not an undo step.
`add_corridor(case)`	A "River corridor" item: the case's corridor, laid out first if it has none (along the thalweg with geo.dat, else straight along the domain). Returns the item's id.
`add_input(file, code, values, case)`	One more line of a command (another gauge, solid, ...); returns its occurrence.

Method	What it does
`add_iso_surface(iso_by, iso_value, **properties)`	Add an iso-surface of the volume where `iso_by` equals `iso_value`; without a variable, a sensible first one (see DataManager.iso_default), recorded so the script repeats it. Other properties (e.g. `variable` to colour by, `case`) as for add_item.
`add_iso_volume(iso_by, iso_min, iso_max, **properties)`	Add an iso-volume: the part of the volume where `iso_by` lies between `iso_min` and `iso_max`; without a variable a first one (phi ¿= 0, the water of a CFD run; see DataManager.iso_volume_default), recorded so the script repeats it.
`add_item(kind, name, **properties)`	Add a scene item and return its id (a "River corridor": the case's corridor, laid out first if it has none, add_corridor).
`add_multiblock(case)`	A "Multi-block grid" item: the case's blocks, started first if it has none (from its river corridor, else one block over the domain). Returns its id.
`add_of(kind, dataset, case)`	Add an item that shows a chosen output (see DataManager.addable): an iso-surface or iso-volume of a volume, a free surface or a bed; a free surface of SFLOW beside NHFLOW's; a slice of an earlier run's volume. `case`: the case item (several cases open). Returns the item id.
`add_slice(normal, position, **properties)`	Add a slice through the volume; without a position it cuts the middle. With several cases open, `case=` names the one to cut (by default the first that has a volume).
`add_statistics(source, start, end, variables, wet_depth, case, progress, compute, **properties)`	A statistics map of a surface output over a time window (`source`: its key, default the free surface; `end` 0: to the last output): per point the max, min, mean, time of the maximum and change of `variables` (default eta and velocity; a bed's elevation), the water depth's maximum, the significant wave height Hs from eta, when the water first arrives, how long it stays and the maximum inundation. Worked out now (`compute`) and kept in the case's `LAGOON_Statistics` folder. Returns the item.
`add_streamlines(seed_normal, seed_position, **properties)`	Add streamlines of the volume's velocity, seeded on a plane through the middle of the volume (the position is recorded, like a slice's).

Method	What it does
`add_structure(code, values, case)`	A porous structure or a solid, one undo step: `code` one of lagoonviz.core.structures.KINDS – porous (ctrl.txt): B 270 box, B 274 cylinder, B 281/B 282 wedges, B 291 plate for CFD, B 210 – B 218 for NHFLOW; solids: DIVEMesh's S 10 box, S 32/S 33 cylinders, S 51 sphere, S 61/S 62 wedges (control.txt), NHFLOW's direct forcing A 581 – A 588 (ctrl.txt). `values`: its line (default: structure_default). NHFLOW's porous material B 201 is added too when the case has none. Returns the structure's occurrence.
`attach_river(edge, line, cells, width, step, discharge, case)`	A river block on a part of an edge on the outline (lagoonviz.core.multiblock.Topology.attached_river): along `line` (upstream first; default the river drawn on the map, else the corridor's line), `cells` of the edge's cells wide (or about `width` m), cells about `step` m long, the `discharge` (m3/s) in at its upstream face.
`auto_corridor(margin, level, points, width, sides, case)`	A corridor that follows the thalweg of the bed (geo.dat): from the deepest point of the upstream side to the deepest of the downstream side (`sides`, default: the inflow and outflow sides, C 1x = 1 and 2, else the ends of the domain's longer axis), its banks where the water below `level` (default: F 60, else halfway up the banks) reaches, plus `margin` m (or `width` m wide), `points` points on each bank. Returns the thalweg found.
`balance_blocks(processes, case)`	Cut the blocks 1.. into pieces so that the processes (default M 10) get about as many cells each (REEF3D puts each piece on one process, block 0 over all).
`camera_at(time)`	The camera on the path at a time (default: now); None when the camera path is off or there are no keys.
`case_file_clashes(folder, case)`	The files and folders of a case that a folder already has (what create_case would overwrite there).
`check_input(case_id)`	Check a case's input before a run (the first case by default): ctrl.txt and control.txt together, DIVEMesh's grid for them, and what the run will cost; a lagoonviz.core.input_check.InputReport (`print(report.text())`). Edits not saved yet are checked as they are on disk.

Method	What it does
`clear_camera_keys()`	Remove all camera keyframes.
`clear_georef(case)`	Remove a case's georef.json.
`compute_statistics(item_id, progress, again)`	Work a statistics item's map out (reading every output of its window) and keep it in the case's `LAGOON_Statistics` folder; a map kept for the same recipe is used as it is unless `again`. Not an undo step. Returns the file.
`corridor(case)`	A case's river corridor (lagoonviz.core.corridor.Corridor, from its corridor.json) or None.
`corridor_domain(margin, width, case)`	The domain (B 10's x and y) around the river drawn (`width` m wide; default: the corridor's width) or else around the corridor, `margin` m beyond it on every side (default: half the river's width), its edges on DIVEMesh's cells (B 1): where geo.dat is made for the river. Returns (x0, x1, y0, y1).
`corridor_grid(case)`	The grid the corridor makes: (x, y, zb, quality).
`create_case(folder, case, overwrite)`	Put a case (a draft started on the map) in its folder: its input files (edits saved first), georef.json, geo.dat and what DIVEMesh made are copied there, and the case is that folder from now on (named after it). The folder is made if needed; files of the same names already in it are kept unless `overwrite` (then an error says which). Returns the folder.
`create_river(width, points, margin, level, smoothing, case)`	The river corridor and its grid along the river drawn (draw_river): `width` m wide, `points` points on each bank; with geo.dat fitted to the river in the bed (its thalweg from the upstream end to the downstream end, its banks where the water below `level` reaches plus `margin` m, at most half as far again as `width`). `smoothing`: Winslow sweeps of the grid. One undo step. Returns the thalweg found (None without geo.dat).
`data_centre()`	The middle of the box of the visible data at the current time, or None.
`divemesh_done(case_id)`	After DIVEMesh has run on a case: read its outputs again and give the case a grid item if it has none (the GUI runs DIVEMesh itself and calls this).

Method	What it does
`draw_river(line, case)`	The river drawn on the map: a line ((x, y) points, upstream first; None removes it), kept in river.json, one undo step. Then corridor_domain sets the domain around it (for geo.dat) and create_river makes the corridor and its grid along it.
`extrude_block(edge, length, cells, case)`	One more block outside a boundary edge, `length` m deep, `cells` cells.
`find_gauges(case, sources, count, radius_km)`	The tide gauges nearest to a case (Kartverket, NOAA CO-OPS, IOC): lagoonviz.core.geo.gauges.Station s, the nearest first.
`find_storms(first_year, last_year, count, case, progress, source)`	The largest wave events at a case (default the last ten years): lagoonviz.core.geo.metocean.Storm s, the highest first. `source`: "MET Norway" (NORA3), "Open-Meteo" (ERA5) or "Open-Meteo, CERRA wind" (ERA5's waves, CERRA's wind); default MET Norway in its waters, Open-Meteo elsewhere. Read at the middle of the wave generation side (else the domain's centre); the case needs a georeference. Not an undo step.
`fit_block_cells(case)`	The cells of every edge with a spacing from its length as it is now (after corners were moved), carried across the blocks.
`fit_corridor(margin, level, case)`	The corridor (drawn by eye, or dragged) made to follow the river in the bed (geo.dat): along its thalweg from the upstream end of the corridor to the downstream end, its banks where the water below `level` (default: F 60, else halfway up the banks) reaches, plus `margin` m; its points, curve and grid kept. Returns the thalweg found.
`fit_range(item_id)`	Fix the colour range of an item to the values of the current frame (their min and max times the item's "range_percent").
`follow_water(edge, margin, radius, level, corners, case)`	Lay an edge along the water (lagoonviz.core.waterline): on land `margin` m from the water below `level` (default F 60), bending no tighter than `radius` m, between the points of the shore nearest its two corners (they move there; `corners` False: they stay, the edge joining the shore from them), the way round nearer to where it is now. Needs geo.dat. Moving one of its corners later lays it again.

Method	What it does
`from_script(source)`	Run a recorded script and return the session it builds.
`gauge_boundaries(station, start, end, sides, case, apply)`	A gauge's observed water level as the water level at sides of a case's domain (`hydrograph_FSF_<n>.dat`; with `apply` B 440, B 422: one undo step). Returns the files written.
`gauge_levels(station, start, end, case, progress)`	The water levels measured at a gauge (find_gauges; or its source and code as "NOAA:8518750") from `start` to `end` (UTC), with the tide predicted where its source has it, as a table in `LAGOON_Sea` shown as a time series (time 0 at `start`): to compare a run with. Returns the new items.
`geo_dat_job(source, crs, method, spacing, depths, case, lat_below_msl, most_points, nearshore, dean_a, trust_depth, overlays, icesat2)`	make_geo_dat in two parts, for a GUI: `job.run(progress)` does the work (reading, downloading, writing geo.dat) and may run in a worker thread; `job.finish()` then, in the thread of the session, turns G 10 on and records it. Checks what it can at once (raises ValueError).
`geo_dat_recipe(case)`	How the case's geo.dat was made (its source, spacing, the georeference and domain it was made for), or None when LAGOON did not make it.
`geo_dat_state(case)`	Why the case's geo.dat no longer fits it, or "" (fits, or not known): made before the model was turned or moved on the map, in another CRS, or for a smaller domain. DIVEMesh would then mesh the bed of the old place; remake_geo_dat makes it again from the same source.
`georef(case)`	Where a case's model coordinates lie on the Earth (a lagoon_format.geo.Georef: CRS, origin, rotation, vertical datum), or None. From its `georef.json`, else its store or run log.
`import_blocks(path, smoothing, case)`	A blocks.dat made elsewhere (default: the case's) as the case's blocks: their faces as edges of their own nodes, the interfaces where faces share nodes, the boundaries from control.txt (C 11 – C 14) and ctrl.txt (G 54), `smoothing` sweeps inside each block. Without geo.dat the nodes' bed is written as geo.dat, so Make grid gives it back. One undo step; returns the number of blocks.

Method	What it does
input_file(file, case)	A case's ctrl.txt or control.txt as it is being edited (a lagoonviz.core.data.controlfile.ControlFile; empty if the file does not exist yet). case: the case item (default: the first).
inputs_modified(case)	The input files of a case with edits not saved yet.
is_draft(case)	Whether a case is a draft started on the map, not yet in a folder of its own (create_case).
legend_layout()	The whole legend layout (set_legend).
load(path)	A session saved with save (a class method).
location(case)	Latitude and longitude of the centre of a case's domain (the open case's; None without a georeference or a case). A place search puts the places close to it first.
make_corridor_grid(case)	Write the corridor's grid as curvgrid.dat and set the case up for it, one undo step: G 50 2 in ctrl.txt, and DIVEMesh's grid the logical box of ni x nj cells (B 10's x and y, B 1; its heights kept). Refused if cells fold. Returns the grid's lagoonviz.core.corridor.Quality.
make_geo_dat(source, crs, method, spacing, depths, case, progress, lat_below_msl, most_points, nearshore, dean_a, trust_depth, overlays, icesat2)	Bathymetry for DIVEMesh: a GeoTIFF, ASCII grid or x y z file (in crs, or its own; "model": already in model coordinates) interpolated ("tin", "idw", "nearest") onto a grid over the domain and written as geo.dat in model coordinates, every spacing metres (default: DIVEMesh's cell size, B 1; "finest": the finest the data has, Kartverket's 1 m in Norway, Copernicus' 30 m on land elsewhere, a file's own cells), coarser where it would be more than most_points points (default 4 million; 0: no limit). depths: the file gives depths (positive down). Turns G 10 on.
make_multiblock_grid(case)	Write the blocks as blocks.dat and set the case up for them, one undo step: R 20 1 and the sides of block 0 (C 11 – C 14) in control.txt, its box (B 10's x and y; its heights and B 1 kept); G 50 1, the native in- and outflow (B 60 1, W 10, F 62) and the open faces of the other blocks (G 54) in ctrl.txt; blocks cut into pieces (for the processes) are written piece by piece, and the G 54 lines name the pieces. Refused while a rule is broken. Returns the grid's quality.
make_native_block(block, case)	Make a block block 0, the grid of the run (DIVEMesh's box).

Method	What it does
`move_block_point(edge, index, x, y, case)`	Move a point that shapes an edge (index 0: the first after its start).
`move_block_vertex(name, x, y, case)`	Move a corner (vertex) of the blocks: every edge from it follows; an edge that follows the water is laid along it again (the corner onto it).
`move_corridor_point(side, index, x, y, case)`	Move a point of a bank ("left" or "right" looking downstream; index 0 upstream) to (x, y): the bank follows it.
`multiblock(case)`	A case's multi-block grid (lagoonviz.core.multiblock.Topology, from its blocks.json) or None.
`multiblock_balance(processes, case)`	How the processes share the blocks' cells (lagoonviz.core. multiblock.Balance; default M 10's processes).
`multiblock_check(case)`	What DIVEMesh and REEF3D would say about the blocks: (issues, quality) (lagoonviz.core.multiblock.check, lagoonviz.core.multiblock.quality).
`multiblock_from_corridor(case)`	The case's river corridor as block 0 of a multi-block grid (its banks the j faces, its ends the i faces; cells and smoothing of its grid).
`new_case(path, name)`	Start a case in a folder (made if needed): its control.txt and ctrl.txt are started with a comment line if they are not there, and the case opens in the set-up, ready for the commands. Returns the case id.
`new_case_from_map(place, length, width, name)`	Start a case on the map: a draft case (in LAGOON's drafts folder, see drafts_folder) with a domain `length` x `width` metres (B 10, cells of a hundredth of the length, B 1) centred on `place` (a name or "lat, lon" to search, a lagoonviz.core.geo.places.Place, or (lat, lon)), a background map round it, in the set-up's 2D view. Draw the domain, add the bathymetry, then create_case puts it in its folder. Returns the case id.
`open(path)`	Open a `.lagoon-session` file (a class method).
`open_case(path, name)`	Open a REEF3D case and show its free surface and bed (its volume, time series and free surface lines are there, switched off; a CFD run's free surface is the level set phi at 0, an iso-surface of its volume), and its bodies. Returns the case id.

Method	What it does
`orbit_camera(degrees, start, end, centre)`	Camera keyframes that take the saved view around the scene: `degrees` about the vertical axis through `centre` (default: the middle of the data shown now) from `start` to `end` (default: the timeline's first and last frame). Replaces the keyframes there were; turns the camera path on. Returns the number of keys.
`place_case(place, case, crs)`	Put the centre of a case's domain at a place on the Earth (a name or "lat, lon" searched for, the first place found; a Place; or (lat, lon)): its georeference, in its CRS (the UTM zone of the place when it has none yet, or when the place lies outside the area its CRS is meant for: another UTM zone, ETRS89 outside Europe), the rotation kept; `crs`: another CRS in metres to use. Gives the case a background map if it has none. Returns the place.
`processes(case)`	The processes a case runs on: M 10 of ctrl.txt (else control.txt), else 1.
`record_output(code)`	Note a figure, film or screenshot made now, as the Python that makes it again (`lagoonviz.render.script` writes it). It goes into the recorded script after the change it followed; not an undo step.
`redo()`	Do again what was undone.
`reload(case_id)`	Read the case folders again, keeping the scene, the time and the camera (new results of a running simulation). Not an undo step.
`remake_geo_dat(case, progress)`	geo.dat again from the source it was made from, for the case as it is now (turned, moved, its domain): the same spacing, method and file. Returns the number of points.
`remake_geo_dat_job(case)`	remake_geo_dat in two parts (`job.run`, `job.finish`).
`remove_block(block, case)`	Take a block out (with the edges and corners only it used).
`remove_camera_key(time)`	Remove the keyframe at `time` (default: now); False if there is none.
`remove_corridor(case)`	Remove a case's corridor.json (curvgrid.dat and the input files stay).
`remove_input(file, code, occurrence, case)`	Remove a command's line (all its lines with `occurrence` None).
`remove_item(item_id)`	Remove an item from the scene (a case with the items that show it).
`remove_multiblock(case)`	Remove a case's blocks.json (blocks.dat and the input files stay).
`rename(item_id, name)`	Give an item another name.

Method	What it does
`reset()`	Start an empty session.
`reset_corridor(case)`	Lay the corridor out again the way it was made (along the domain, over the area drawn, or along the thalweg with its margin): the dragged points are dropped, the grid's settings (cells, smoothing, curve, bed) kept.
`reverse_corridor(case)`	The river flowing the other way (upstream and downstream swapped).
`river_discharge(start, end, target, case, apply, at)`	GloFAS's discharge of the river at a case (Open-Meteo) from `start` to `end` (UTC), hourly, time 0 at `start`: `hydrograph.dat` for ioFlow (`target` "B 60", B 60 2) or `hydrograph_Q_<n>.dat` for side n as a patch (B 440, B 421); a table shown as a time series. Read at the river corridor's upstream end, else the inflow side's middle, else the domain's centre; `at` = (x, y) to choose. With `apply` ctrl.txt follows (one undo step).
`river_line(case)`	The river drawn on the map (river.json): its points, upstream first; else the line the case's corridor was made along; None if neither.
`run_divemesh(case_id, executable, on_line)`	Run DIVEMesh on a case (the first by default) and show what it made: its grid, solids and bed. Edits of control.txt not saved yet are written first (DIVEMesh reads the file). Returns DIVEMesh's exit code.
`save(path)`	Save the session to a `.lagoon-session` file.
`save_inputs(case)`	Write the edited input files of a case (the first by default); returns the files written. Not an undo step.
`scale_multiblock(factor, case)`	Every edge with `factor` times its cells (2: twice as fine).
`sea_conditions(start, end, case, progress, source)`	The sea at a case from `start` to `end` (UTC: ISO text, a datetime or seconds since 1970): the waves and wind and the water level and current (NORA3 and NorKyst, or ERA5 and Météo-France's sea level with the tide from Open-Meteo; `source` as for find_storms), as tables in the case's `LAGOON_Sea` folder, shown as time series items (time 0 at `start`). Returns the new items.
`set_block_edge(edge, case, **options)`	Change an edge: `cells` (carried on to the opposite faces of the blocks), `curve` ("straight", "through", "bspline"), `ratio` (last cell / first).

Method	What it does
`set_block_face(block, face, kind, value, case)`	The boundary of a face without an interface: "wall", "inflow" (a discharge in m3/s) or "outflow" (a water level in m).
`set_block_options(block, case, **options)`	Change a block: `name`, `smoothing` (Winslow sweeps).
`set_block_spacing(edge, start, end, largest, growth, across, case)`	Space an edge's nodes by the cells at its ends (m; `end` default `start`), growing evenly from one to the other, or, with a `largest` cell, by `growth` from each end up to it; its cells follow from its length (and go on across the blocks). `across`: the edges across the blocks from it without a spacing get it too. `start` None: no spacing.
`set_camera(state)`	Remember the 3D view (saved with the session). View state: not an undo step.
`set_camera_path(on)`	Follow the camera path through the keyframes (views and films), or not.
`set_corridor(corridor, case)`	Give a case a river corridor (a Corridor or its dict as in corridor.json; None removes it). Written to corridor.json at once; one undo step.
`set_corridor_options(points, case, **options)`	Change the corridor: `points` per bank (the banks resampled), `curve` ("through", "bspline"), `cells_along`, `cells_across`, `smoothing` (Winslow iterations), `bed` ("slope", "geo"), `z_start`, `slope`.
`set_domain(x0, x1, y0, y1, case)`	The domain's horizontal extent (B 10 of control.txt, model coordinates); its heights stay as they are (0 to 1 m if there are none).
`set_exaggeration(item_id, value, merge)`	Vertical exaggeration as an item's slider sets it: while its case has sync on (the default) the case's value, so every item of the case follows; else the item's own.
`set_georef(crs, origin, rotation_deg, vertical_datum, z_offset, case)`	Georeference a case: its model's (0, 0) lies at `origin` in `crs` ("EPSG:25833"), its x-axis turned `rotation_deg` counter-clockwise from east, its $z = 0$ at `z_offset` in the vertical datum. Written to `georef.json` in the case folder at once (REEF3D and DIVEMesh read it from there).
`set_input(file, code, values, occurrence, case)`	Give a command of an input file new values: its `occurrence`-th line (a new line if there are fewer), e.g. `set_input("ctrl.txt", "P 51", [1.0, 0.5], occurrence=1)` moves the second wave gauge. Edits are kept until save_inputs.

Method	What it does
`set_input_text(file, text, case)`	Replace the whole text of an input file (the text editor).
`set_legend(**settings)`	Where films and saved images put the colour bars and the time, and how they look; the 3D view's bars on or off (lagoonviz.core.legend.LAYOUT): `bars` (a corner: "lower left", "lower right", "upper left", "upper right"), `time` (a corner or "off"), `length` (of the image's width), `thickness` and `text` (1: as by default), `box` (a light box behind them), `view` ("below the view" or "off"). `reset=True` first goes back to the defaults. Each bar's title, numbers and ticks are the item's (`legend_*`). View state: not an undo step. Returns the whole layout.
`set_mode(mode)`	Work on the results ("results") or on the set-up ("setup") of the cases.
`set_multiblock(topology, case)`	Give a case a multi-block grid (a Topology or its dict as in blocks.json; None removes it). Written to blocks.json at once; one undo step.
`set_plan_view(on, side)`	2D: orthographic, no rotation. From the top (`side="top"`) every item is drawn flat (the free surface as a plane coloured by its values over the bed); from the "front", "back", "right" or "left" it is an elevation of the shapes. `side=None` keeps the side. View state: not an undo step.
`set_play_range(start, end)`	Play only the part of the timeline from `start` to `end` (times); both None: all of it. Playback, films and the export dialog keep to it. View state: not an undo step.
`set_properties(item_id, **values)`	Set several properties of one item as one undo step.
`set_property(item_id, key, value, merge)`	Set one property.
`set_rotation(rotation_deg, about, case)`	Turn a case's georeference to `rotation_deg` about a model point (x, y): that point stays where it is on the map (the Map tab's slider: the middle of the view). Default: the model's origin.
`set_sun(azimuth, elevation)`	Put the sun at `azimuth` degrees (from +x towards +y: 0 shines from +x, 90 from +y) and `elevation` degrees above the horizon; both None: where the look puts it. It lights the 3D view, films and path tracing alike. View state: not an undo step.

Method	What it does
`set_time(time)`	Move the current time. View state: not an undo step.
`set_visible(item_id, visible)`	Show or hide an item.
`split_block_edge(edge, x, y, case)`	Cut an edge in two at its node nearest to (x, y) (a block then meets only one part, e.g. a river entering a harbour).
`start_corridor(area, points, case)`	A straight river corridor over a plan area (x0, x1, y0, y1; default the domain without a tenth on each side across), flowing along its longer side, `points` points on each bank.
`start_multiblock(area, cells, case)`	One block over a plan area (x0, x1, y0, y1; default the domain), i along x, `cells` (ni, nj) (default: about 40 along the longer side, square cells).
`statistics_job(item_id, again)`	compute_statistics in two parts, for a GUI: `job.run(progress)` reads the outputs and writes the map (it may run in a worker thread); `job.finish()` then, in the session's thread, shows it.
`structure_default(code, case, area)`	The line of a structure code over a plan area (x0, x1, y0, y1; default the middle of the domain), from the bed (the domain's bottom) up: a porous one to the still water level (F 60), of rubble; a solid to the domain's top (for NHFLOW's σ-grid, half the water depth above the still water).
`subscribe_log(callback)`	Call `callback` with every command run (the Python console uses it).
`sun_angles(look)`	(azimuth, elevation) of the sun in degrees: the session's, else the look's.
`sun_direction()`	The unit vector towards the session's sun, or None (the look's own).
`sync_exaggeration(item_id, on)`	Switch the case-wide vertical exaggeration of an item's case on or off.
`to_script()`	The session's history as a runnable Python script. Figures, films and screenshots made are in it too, each where it was made; they are made again when the script is run (`python script.py`), not by `from_script`.

Method	What it does
`try_rotation(rotation_deg, about, case)`	Show a case on the map as if its model were turned to `rotation_deg` (degrees counter-clockwise from east), about a model point (default: its origin, as the georeference's rotation turns it) – the maps turn and move, north follows – without writing anything: what a rotation slider shows while it is dragged. None: as it is. Not an undo step; set_georef or turn_model then keeps it.
`turn_model(rotation_deg, about, case)`	Turn a case's model on the map to a new rotation (degrees counter-clockwise from east), about a model point (default: the centre of its domain), which stays where it is on the map. The domain keeps its model coordinates.
`undo()`	Take back the last change.
`water_level(case)`	The still water level of a case: F 60 of its ctrl.txt, else 0.
`write_block_series(block, start, days, file, glofas, outflow, at, scale, case)`	An open face of a block 1.. over time (REEF3D's G 54 types 3, 4): the discharge in at its -i face, or with `outflow` the water level at its +i face, from a `file` ("time value" a line: seconds from the start, or a date and time with `start`) or from GloFAS (`glofas`: the river's daily discharge at the face's middle, or at `at` = (x, y), hourly from `start` over `days`); times 0 – `days` when both are given, the values times `scale`. Writes discharge-¡block¿.dat (waterlevel-¡block¿.dat), the face's boundary and, if the grid was made (G 50 1), its G 54 line. One undo step; returns the lagoonviz.core.multiblock.Series.
`write_boundaries(start, end, sides, case, apply, progress, source)`	The water level and discharge along the sides of a case's domain from `start` to `end` (UTC; NorKyst, or Open-Meteo's sea level and current with the depth from the case's bed; `source` as for find_storms): REEF3D's `hydrograph_FSF_<n>.dat` and `hydrograph_Q_<n>.dat` per side in the water (n: 1 x min, 2 y max, 3 y min, 4 x max; `sides` to choose), a table of them shown as a time series; with `apply` ctrl.txt gets the patches (B 440, B 421, B 422; one undo step). Returns the sides written.

Method	What it does
`write_spectrum(when, case, apply, half_width)`	NORA3's 2D wave spectrum at a case at `when` (UTC) as REEF3D's `spectrum-file-2d.dat` in the case folder; with `apply` the wave generation of ctrl.txt follows it (B 85 11, B 92 31, B 93, B 105, B 130 1; one undo step, saved with the input files). Returns Hs, Tp, the direction in the model and the file.
`write_tide(start, days, step, constants, levels, gauge, predicted, harmonic, ramp, direction, reference, depth, current, case)`	The tide at the open sea for a run from `start` (UTC) over `days` (lagoonviz.core.tide): from tidal `constants` ("M2 1.00 61" a line: amplitude m, Greenwich phase deg; or a file), a file of `levels` or a tide gauge (find_gauges, or "SOURCE:CODE"; `predicted` instead of observed), as REEF3D's background 1: background-1.dat ("t eta 0 0", or "t eta" with `current`) and B 510 (mode 2, `ramp` s, travelling along `direction` deg), or with `harmonic` the constituents as B 511; with a `reference` (x, y) B 515 makes it travel from there at sqrt(g `depth`). Block 0's tide faces (Blocks tab) take it. One undo step; returns the lagoonviz.core.tide.Tide.
`write_waves(when, case, apply)`	The waves at a case at the hour `when` (UTC) from ERA5 (Open-Meteo): Hs, Tp and the mean direction for a JONSWAP spectrum; with `apply` the wave generation of ctrl.txt follows them (B 85 2, B 92 31, B 93, B 105; one undo step). For NORA3's 2D spectrum: write_spectrum.

Bibliography

Blender Online Community. Blender – a 3D modelling and rendering package. Blender Foundation, `https://www.blender.org`, 2026.

Robert G. Dean. Equilibrium beach profiles: characteristics and applications. *Journal of Coastal Research*, 7(1):53–84, 1991.

Yoshimi Goda and Yasumasa Suzuki. Estimation of incident and reflected waves in random wave experiments. In *Proceedings of the 15th International Conference on Coastal Engineering*, pages 828–845, Honolulu, 1976.

Markus Ihmsen, Nadir Akinci, Gizem Akinci, and Matthias Teschner. Unified spray, foam and air bubbles for particle-based fluids. *The Visual Computer*, 28(6–8):669–677, 2012.

Etienne P. D. Mansard and E. R. Funke. The measurement of incident and reflected spectra using a least squares method. In *Proceedings of the 17th International Conference on Coastal Engineering*, pages 154–172, Sydney, 1980.

Anton Mikhailov. Turbo, an improved rainbow colormap for visualization. Google Research blog, `https://research.google/blog/turbo-an-improved-rainbow-colormap-for-visualization/`, 2019.

NTNU. ÆgirFrames: rendering of REEF3D results with Blender. `https://github.com/NTNU-MCS/aegir-frames`, MIT licence, 2026.

Peter D. Welch. The use of fast Fourier transform for the estimation of power spectra: a method based on time averaging over short, modified periodograms. *IEEE Transactions on Audio and Electroacoustics*, 15(2):70–73, 1967.

Alan M. Winslow. Numerical solution of the quasilinear Poisson equation in a nonuniform triangle mesh. *Journal of Computational Physics*, 1(2):149–172, 1966.

Zarr Developers. Zarr storage specification version 3. `https://zarr-specs.readthedocs.io/en/latest/v3/core/index.html`, 2023.